GLITTER MATH SYSTEMS — COMPLETE REPO DUMP

Systems 47-60 (14 systems, 42 source files consolidated)

Generated 2026-05-07

==========

SYSTEM 47: CUSP FLASH

==========

— DOCUMENTATION —

System 47: Cusp Flash

Overview

Shine that doesn't fade — it **explodes** at critical points. Pressure builds: nothing, nothing, BAM — full chrome burst. The cusp catastrophe = binary, dramatic, irreversible. Hysteresis means the surface remembers.

Mathematical Foundation

Cusp Catastrophe Theory (Thom, 1972): The cusp is the simplest catastrophe with two control parameters (normal factor `a`, splitting factor `b`) and one state variable `x`.

The potential function:

```
V(x) = x⁴/4 + a·x²/2 + b·x
```

Critical points where `dV/dx = 0`:

```
x³ + a·x + b = 0
```

The cusp bifurcation set (where the number of equilibria changes):

```
4a³ + 27b² = 0
```

Hysteresis: The surface remembers its history. Going up the control path vs down gives different state transitions. The system has memory.

Key Parameters:

- `control_a` (-2 to 2): normal factor — moves between single equilibrium and bistable region
- `control_b` (-1 to 1): splitting factor — asymmetry, triggers the jump
- `hysteresis_state`: 0 or 1, the memory of which branch we're on
- `flash_threshold`: critical point where catastrophe occurs

GLSL Shader

See `shader.glsl` — implements cusp catastrophe with hysteresis memory, chrome burst, and prismatic flash.

JS Module

See `module.js` — touch/pressure input mapping, hysteresis state machine, critical point detection.

Showcase Builds

1. Spark Plug Skin

Touch pressure builds `control_b`. At critical threshold: prismatic flash across surface. Release: stays on chrome branch (hysteresis) until pressure drops below lower threshold. Binary, irreversible.

2. Dam Break

Containment cells as cusp systems. Individual cells breach catastrophically when local pressure exceeds threshold. Chain reaction as breached cells increase pressure on neighbors. Domino collapse.

3. Hysteresis Tattoo

Touch history recorded as binary chrome/matte states. Each touch point is a cusp system. The pattern of chrome vs matte IS the touch history. Memory made visible.

4. Cusp Choir

Array of cusp systems with slightly different thresholds. Flash events create rhythmic patterns — some trigger early, some late. The ensemble performs catastrophe music.

5. The Sneeze

Buildup-then-release cycle. `control_a` slowly increases (pressure builds). At critical point: explosive flash (the sneeze). Then reset. Biological urgency made optical.

Implementation Notes

- Hysteresis requires storing previous state — use a texture or uniform
- Critical flash should be instantaneous (1-2 frames) for impact
- Chrome branch should have different reflectivity than matte branch
- Prismatic flash = full spectrum sweep during transition

— GLSL SHADER —

```
// System 47: Cusp Flash
// Cusp catastrophe with hysteresis — shine that explodes at critical points

uniform float u_time;
uniform vec2 u_resolution;
uniform float u_control_a;      // normal factor (-2 to 2)
uniform float u_control_b;      // splitting factor (-1 to 1)
uniform float u_hysteresis;     // 0 or 1, previous state memory
```

```
uniform float u_flash;          // flash intensity (0-1, peaks at catastrophe)

#define PI 3.14159265359

// Hash for sparkle variation
float hash(vec2 p) {
    return fract(sin(dot(p, vec2(12.9898, 78.233))) * 43758.5453);
}

// Cusp catastrophe potential: V(x) = x^4/4 + a*x^2/2 + b*x
// We solve for equilibria and determine which branch we're on
float cuspState(float a, float b, float hysteresis) {
    // Discriminant of x^3 + a*x + b = 0
    float discriminant = -(4.0 * a * a * a + 27.0 * b * b);

    // If discriminant > 0: three real roots (bistable region)
    // If discriminant < 0: one real root (monostable)

    float state;
    if (discriminant > 0.0) {
        // Bistable: hysteresis determines which branch
        state = hysteresis;
    } else {
        // Monostable: forced to single branch
        state = (b > 0.0) ? 0.0 : 1.0;
    }

    return state;
}

// Chrome reflection with cusp distortion
vec3 chromeReflect(vec2 uv, float state, float flash) {
    vec2 p = uv * 2.0 - 1.0;
    float r = length(p);
    float angle = atan(p.y, p.x);

    // State 1 = chrome (high reflectivity), State 0 = matte
    float reflectivity = mix(0.1, 0.95, state);

    // Environment reflection (simplified)
    vec3 envColor = vec3(
        0.5 + 0.5 * cos(angle * 3.0 + u_time * 0.5),
        0.5 + 0.5 * cos(angle * 2.0 + u_time * 0.3 + 1.0),
        0.5 + 0.5 * cos(angle * 4.0 + u_time * 0.7 + 2.0)
    );

    // Add flash spectrum during catastrophe
    float flashSpectrum = flash * (
        0.5 + 0.5 * cos(r * 20.0 - u_time * 10.0)
    );
```

```
    vec3 flashColor = vec3(
        1.0,
        0.8 + 0.2 * cos(u_time * 15.0),
        0.6 + 0.4 * sin(u_time * 15.0)
    );

    return mix(envColor * reflectivity, flashColor, flashSpectrum);
}

// Hysteresis boundary detection
float catastropheFlash(float a, float b, float prevState, float currState) {
    // Flash when state changes (the catastrophe event)
    float stateChange = abs(currState - prevState);

    // Also flash near the cusp boundary (anticipation)
    float boundary = -(4.0 * a * a * a + 27.0 * b * b);
    float nearBoundary = smoothstep(0.0, 0.5, boundary) * smoothstep(2.0, 0.5, boundary);

    return stateChange + nearBoundary * 0.3;
}

void main() {
    vec2 uv = gl_FragCoord.xy / u_resolution.xy;
    vec2 p = uv * 2.0 - 1.0;
    p.x *= u_resolution.x / u_resolution.y;

    // Current cusp state
    float state = cuspState(u_control_a, u_control_b, u_hysteresis);

    // Detect catastrophe
    float flash = catastropheFlash(u_control_a, u_control_b, u_hysteresis, state);
    flash = clamp(flash, 0.0, 1.0);

    // Chrome reflection
    vec3 color = chromeReflect(uv, state, flash);

    // Add sparkle noise for texture
    float sparkle = hash(gl_FragCoord.xy + u_time) * 0.1 * state;
    color += vec3(sparkle);

    // Vignette
    float vignette = 1.0 - smoothstep(0.5, 1.5, length(p));
    color *= vignette;

    gl_FragColor = vec4(color, 1.0);
}
```

— JS MODULE —

```
// System 47: Cusp Flash — JS Module
// Hysteresis state machine, critical point detection, touch mapping
```

```
export class CuspFlashSystem {
    constructor() {
        this.controlA = 0.0;        // normal factor
        this.controlB = 0.0;        // splitting factor
        this.hysteresis = 0;        // state memory: 0 or 1
        this.prevHysteresis = 0;    // for detecting change
        this.flashIntensity = 0.0; // current flash

        // Cusp parameters
        this.upperThreshold = 0.3;  // jump up threshold
        this.lowerThreshold = -0.3; // jump down threshold
        this.flashDecay = 0.9;      // flash fade per frame
    }

    // Map touch pressure to control_b
    setPressure(pressure) {
        // Pressure 0-1 maps to control_b range
        this.controlB = (pressure - 0.5) * 2.0;
    }

    // Set normal factor (can be animated for "The Sneeze" build)
    setNormalFactor(a) {
        this.controlA = a;
    }

    // Update hysteresis state based on cusp catastrophe
    update() {
        this.prevHysteresis = this.hysteresis;

        // Discriminant: 4a³ + 27b² (negated for our convention)
        const discriminant = -(4 * Math.pow(this.controlA, 3) + 27 * Math.pow(this.controlB, 2));

        if (discriminant > 0) {
            // Bistable region — hysteresis determines branch
            // Only jump if we cross thresholds
            if (this.controlB > this.upperThreshold && this.hysteresis === 0) {
                this.hysteresis = 1;
            } else if (this.controlB < this.lowerThreshold && this.hysteresis === 1) {
                this.hysteresis = 0;
            }
            // Otherwise: stay on current branch (the memory!)
        } else {
            // Monostable — forced to single equilibrium
            this.hysteresis = (this.controlB > 0) ? 0 : 1;
        }

        // Detect catastrophe (state change)
        const stateChanged = (this.hysteresis !== this.prevHysteresis);

        // Flash on state change or near boundary
```

```
        const nearBoundary = (discriminant > -0.5 && discriminant < 0.5);

        if (stateChanged) {
            this.flashIntensity = 1.0;
        } else if (nearBoundary) {
            this.flashIntensity = Math.max(this.flashIntensity, 0.3);
        }

        // Decay flash
        this.flashIntensity *= this.flashDecay;
    }

    // Get uniforms for shader
    getUniforms() {
        return {
            u_control_a: this.controlA,
            u_control_b: this.controlB,
            u_hysteresis: this.hysteresis,
            u_flash: this.flashIntensity
        };
    }

    // For "Dam Break" — array of cells
    static createCellGrid(rows, cols) {
        const cells = [];
        for (let i = 0; i < rows; i++) {
            for (let j = 0; j < cols; j++) {
                cells.push(new CuspFlashSystem());
            }
        }
        return cells;
    }

    // For "Cusp Choir" — slightly varied thresholds
    static createChoir(count) {
        const systems = [];
        for (let i = 0; i < count; i++) {
            const sys = new CuspFlashSystem();
            sys.upperThreshold = 0.3 + (Math.random() - 0.5) * 0.2;
            sys.lowerThreshold = -0.3 + (Math.random() - 0.5) * 0.2;
            sys.controlA = -1.0 + Math.random() * 0.5; // all in bistable region
            systems.push(sys);
        }
        return systems;
    }

    // For "The Sneeze" — slowly increasing pressure
    sneezeBuildup(speed = 0.01) {
        this.controlA -= speed; // Move toward bistable region
        if (this.controlA < -1.5) {
            this.controlA = 0.0; // Reset
```

```
            this.hysteresis = 0;
        }
    }
}
```

==========

SYSTEM 48: PRIME DUST

==========

— DOCUMENTATION —

System 48: Prime Dust

Overview

Every sparkle's position is a prime number. `p(n) mod width, p(n+1) mod height`. Size = prime gap. Color = residue class mod 6 (warm for $\equiv 1$, cool for $\equiv 5$). Non-repeating, non-clustering, but deterministic. Secret math message in the glitter.

Mathematical Foundation

Prime Number Theorem: The nth prime `p(n) ~ n·ln(n)`. Prime gaps grow slowly (max gap ~ $\ln^2 p$). The distribution is deterministic but appears random — "quasi-random."

Key Sequences:

- Position: `x = p(n) mod W`, `y = p(n+1) mod H`
- Size: `s = p(n+1) - p(n)` (prime gap)
- Color: `hue = 0° if p(n) ≡ 1 (mod 6)`, `hue = 240° if p(n) ≡ 5 (mod 6)`
- All primes > 3 are $\equiv 1$ or $5 \pmod 6$

Special Classes:

- Twin primes: gap = 2, connected by lines
- Mersenne primes: `2^p - 1`, only 51 known
- Goldbach pairs: even number = sum of two primes

GLSL Shader

See `shader.glsl` — prime hash for deterministic sparkle placement, mod-6 coloring, gap-based sizing.

JS Module

See `module.js` — prime sieve, nth-prime lookup, special class detection (twin, Mersenne, Goldbach).

Showcase Builds

1. Ulam Galaxy

Spiral reveals diagonal prime lines as constellations. Ulam spiral places integers in spiral — primes cluster on diagonals (Euler's n^2+n+41 and others). The galaxy IS the proof.

2. Prime Gap Rain

Gaps as raindrop sizes. Small gaps = fine mist. Large gaps = heavy drops. Deterministic but chaotic distribution. The rain pattern is a prime signature.

3. Twin Prime Constellation

Only twin pairs (gap=2), connected by glowing lines. Rare and precious. Each twin is a binary star. The constellation map shows where primes cluster.

4. Mersenne Diamonds

Massive diamonds for the 51 known Mersenne primes. Each diamond size proportional to `p` in `2^p-1`. The known giants glow brightest — unknown ones are dark holes.

5. Goldbach Glow

Even numbers as sums of two primes. Visual proof-by-light: each even number glows with intensity proportional to number of Goldbach pairs. The glow IS the theorem.

Implementation Notes

- Precompute primes up to limit (e.g., 1,000,000) for performance
- Use segmented sieve for memory efficiency
- nth-prime approximation: `n * (ln n + ln ln n - 1)` for bounds
- Twin prime detection: check `isPrime(n) && isPrime(n+2)`

— GLSL SHADER —

```
// System 48: Prime Dust
// Sparkle placement from prime sequences — secret math message in the glitter

uniform float u_time;
uniform vec2 u_resolution;
uniform float u_prime_index;    // starting prime index
uniform float u_count;          // number of sparkles
uniform float u_mode;           // 0=ulam, 1=gap rain, 2=twin, 3=mersenne, 4=goldbach

#define PI 3.14159265359
#define TAU 6.28318530718

// Deterministic pseudo-random (for non-prime variation)
float hash(float n) {
    return fract(sin(n * 12.9898) * 43758.5453);
}
```

```
// Approximate nth prime using prime number theorem
// p(n) ≈ n * (ln n + ln ln n - 1) for n >= 4
float nthPrimeApprox(float n) {
    if (n < 4.0) return n * 2.0; // rough: 2, 3, 5, 7
    float ln_n = log(n);
    return n * (ln_n + log(ln_n) - 1.0);
}

// Prime gap approximation (average gap ~ ln p)
float primeGapApprox(float n) {
    return log(nthPrimeApprox(n));
}

// Ulam spiral coordinate from integer n
vec2 ulamCoord(float n) {
    float k = ceil((sqrt(n) - 1.0) / 2.0);
    float t = 2.0 * k + 1.0;
    float m = t * t;

    vec2 pos;
    if (n >= m - t) {
        pos = vec2(k - (m - n), -k);
    } else if (n >= m - 2.0*t) {
        pos = vec2(-k, -k + (m - t - n));
    } else if (n >= m - 3.0*t) {
        pos = vec2(-k + (m - 2.0*t - n), k);
    } else {
        pos = vec2(k, k - (m - 3.0*t - n));
    }
    return pos;
}

// Color from residue mod 6
vec3 primeColor(float p) {
    float residue = mod(p, 6.0);
    // All primes > 3 are ≡ 1 or 5 (mod 6)
    if (abs(residue - 1.0) < 0.5) {
        // Warm: gold/orange
        return vec3(1.0, 0.8, 0.2);
    } else {
        // Cool: blue/cyan
        return vec3(0.2, 0.6, 1.0);
    }
}

// Mersenne diamond shape
float mersenneDiamond(vec2 uv, float size) {
    vec2 p = abs(uv * 2.0 - 1.0);
    return 1.0 - smoothstep(size * 0.8, size, p.x + p.y);
}
```

```
void main() {
    vec2 uv = gl_FragCoord.xy / u_resolution.xy;
    vec2 p = uv * 2.0 - 1.0;
    p.x *= u_resolution.x / u_resolution.y;

    vec3 color = vec3(0.0);

    // Generate sparkles from prime sequence
    float count = min(u_count, 200.0);

    for (float i = 0.0; i < 200.0; i++) {
        if (i >= count) break;

        float idx = u_prime_index + i;
        float prime = nthPrimeApprox(idx);
        float nextPrime = nthPrimeApprox(idx + 1.0);
        float gap = nextPrime - prime;

        // Position based on mode
        vec2 sparklePos;
        float size;

        if (u_mode < 0.5) {
            // Ulam spiral
            sparklePos = ulamCoord(prime) * 0.02;
            size = 0.003 + gap * 0.001;
        } else if (u_mode < 1.5) {
            // Gap rain
            sparklePos = vec2(
                hash(prime) * 2.0 - 1.0,
                fract(hash(prime * 1.618) + u_time * 0.1 * (1.0 + gap * 0.1))
            );
            size = 0.005 + gap * 0.002;
        } else if (u_mode < 2.5) {
            // Twin constellation (only gap=2)
            if (abs(gap - 2.0) > 0.5) continue;
            sparklePos = vec2(
                hash(prime) * 2.0 - 1.0,
                hash(prime * 2.0) * 2.0 - 1.0
            );
            size = 0.01;
        } else if (u_mode < 3.5) {
            // Mersenne diamonds
            // Approximate: check if prime+1 is power of 2
            float log2 = log(prime + 1.0) / log(2.0);
            if (abs(fract(log2)) > 0.01) continue;
            sparklePos = vec2(
                hash(prime) * 2.0 - 1.0,
                hash(prime * 3.0) * 2.0 - 1.0
            );
            size = 0.05 + log2 * 0.02;
```

```
        } else {
            // Goldbach glow
            sparklePos = vec2(
                hash(prime) * 2.0 - 1.0,
                hash(prime * 1.414) * 2.0 - 1.0
            );
            size = 0.008;
        }

        // Distance to sparkle
        float d = length(p - sparklePos);

        // Sparkle shape
        float sparkle = exp(-d * d / (size * size));

        // Color
        vec3 col = primeColor(prime);

        // Add to accumulation
        color += col * sparkle * 0.5;
    }

    // Tone mapping
    color = color / (1.0 + color * 0.5);

    // Vignette
    float vignette = 1.0 - smoothstep(0.5, 1.5, length(p));
    color *= vignette;

    gl_FragColor = vec4(color, 1.0);
}
```

— JS MODULE —

```
// System 48: Prime Dust — JS Module
// Prime sieve, nth-prime, special class detection

export class PrimeDustSystem {
    constructor(maxPrime = 1000000) {
        this.primes = this.sieve(maxPrime);
        this.maxPrime = maxPrime;
        this.startIndex = 0;
        this.count = 100;
        this.mode = 0; // 0=ulam, 1=gap rain, 2=twin, 3=mersenne, 4=goldbach
    }

    // Sieve of Eratosthenes
    sieve(limit) {
        const isPrime = new Uint8Array(limit + 1).fill(1);
        isPrime[0] = isPrime[1] = 0;
```

```
        for (let i = 2; i * i <= limit; i++) {
            if (isPrime[i]) {
                for (let j = i * i; j <= limit; j += i) {
                    isPrime[j] = 0;
                }
            }
        }

        const primes = [];
        for (let i = 2; i <= limit; i++) {
            if (isPrime[i]) primes.push(i);
        }
        return primes;
    }

    // Get nth prime (0-indexed)
    nthPrime(n) {
        if (n < this.primes.length) return this.primes[n];
        // Approximate for large n
        const ln_n = Math.log(n);
        return Math.floor(n * (ln_n + Math.log(ln_n) - 1));
    }

    // Check if n is prime (for numbers beyond sieve)
    isPrime(n) {
        if (n <= this.maxPrime) {
            // Binary search in primes array
            let lo = 0, hi = this.primes.length;
            while (lo < hi) {
                const mid = (lo + hi) >> 1;
                if (this.primes[mid] < n) lo = mid + 1;
                else hi = mid;
            }
            return this.primes[lo] === n;
        }
        // Trial division for large numbers
        if (n < 2) return false;
        if (n % 2 === 0) return n === 2;
        if (n % 3 === 0) return n === 3;
        for (let i = 5; i * i <= n; i += 6) {
            if (n % i === 0 || n % (i + 2) === 0) return false;
        }
        return true;
    }

    // Get twin primes (pairs with gap=2)
    getTwinPrimes(start, count) {
        const twins = [];
        let found = 0;
        let idx = start;
        while (found < count && idx < this.primes.length - 1) {
```

```
        const p = this.primes[idx];
        if (this.primes[idx + 1] === p + 2) {
            twins.push([p, p + 2]);
            found++;
        }
        idx++;
    }
    return twins;
}

// Get Mersenne primes (2^p - 1 where p is prime)
// Only 51 known — hardcoded exponents for accuracy
getMersenneExponents() {
    return [2, 3, 5, 7, 13, 17, 19, 31, 61, 89, 107, 127, 521, 607, 1279,
            2203, 2281, 3217, 4253, 4423, 9689, 9941, 11213, 19937, 21701,
            23209, 44497, 86243, 110503, 132049, 216091, 756839, 859433,
            1257787, 1398269, 2976221, 3021377, 6972593, 13466917, 20996011,
            24036583, 25964951, 30402457, 32582657, 37156667, 42643801,
            43112609, 57885161, 74207281, 77232917, 82589933];
}

// Goldbach pairs for even number n
getGoldbachPairs(n) {
    if (n % 2 !== 0 || n < 4) return [];
    const pairs = [];
    for (let p of this.primes) {
        if (p > n / 2) break;
        if (this.isPrime(n - p)) {
            pairs.push([p, n - p]);
        }
    }
    return pairs;
}

// Get uniforms for shader
getUniforms() {
    return {
        u_prime_index: this.startIndex,
        u_count: this.count,
        u_mode: this.mode,
        u_time: performance.now() * 0.001
    };
}

// Animate through primes
animate(speed = 1.0) {
    this.startIndex += speed;
    if (this.startIndex > this.primes.length - this.count) {
        this.startIndex = 0;
    }
}
```

```
    // Set mode
    setMode(mode) {
        this.mode = mode;
    }
}
```

==========

SYSTEM 49: ESCHER CHROME

==========

— DOCUMENTATION —

System 49: Escher Chrome

Overview

Chrome reflection through conformal maps — Möbius, inversion, Schwarz-Christoffel. Circles stay circles, angles preserved, but space bends. Straight lines become arcs. Parallel lines meet. The mirror shows a world with different geometry.

Mathematical Foundation

Conformal Maps: Complex functions that preserve angles locally. `f(z)` where `f'(z) ≠ 0`.

Key Maps:

- **Möbius**: `f(z) = (az+b)/(cz+d)`, maps circles→circles, lines→circles
- **Inversion**: `f(z) = R²/z̄`, turns world inside-out through sphere
- **Schwarz-Christoffel**: Maps upper half-plane to polygon interiors
- **Hyperbolic**: `f(z) = (z-a)/(1-āz)`, Poincaré disk model

Invariant: Cross-ratio `(z₁,z₂;z₃,z₄) = (z₁-z₃)(z₂-z₄)/((z₁-z₄)(z₂-z₃))` preserved under Möbius.

GLSL Shader

See `shader.glsl` — conformal map distortion of environment reflection, circle preservation, hyperbolic tiling.

JS Module

See `module.js` — Möbius transform composition, inversion center animation, Schwarz-Christoffel parameter computation.

Showcase Builds

1. Circle Limit Chrome

Infinite hyperbolic tiling in a circle (Escher's Circle Limit IV). Chrome polygons reflect distorted world. The circle boundary is infinity — objects shrink exponentially as they approach it.

2. Möbius Mirror

Mirror that inverts/rotates based on viewer distance. Funhouse mirror with real math. The distortion is a Möbius transform parameterized by viewer position.

3. Inversion Jewelry

Pendant that reflects the world inside-out through sphere inversion. The "stone" is a window into inverted space. Your face appears upside-down and reversed.

4. Schwarz-Christoffel Architecture

Building facade "unfolded" from star polygon via conformal map. The straight walls of the building are images of circular arcs in the mapped space.

5. Hyperbolic Caustics

Light through hyperbolic lens. Caustics are circular arcs not parabolas. The focusing properties of hyperbolic geometry made visible.

Implementation Notes

- Use complex arithmetic in GLSL (vec2 as complex number)
- Möbius transforms: normalize so ad-bc = 1
- Inversion: handle z=0 singularity (map to infinity)
- Hyperbolic distance: `arctanh(|(z-a)/(1-āz)|)`

— GLSL SHADER —

```glsl
// System 49: Escher Chrome
// Conformal map reflection — circles stay circles, space bends

uniform float u_time;
uniform vec2 u_resolution;
uniform float u_map_type;       // 0=mobius, 1=inversion, 2=hyperbolic, 3=schwarz
uniform vec2 u_mobius_a;        // Möbius parameter a (complex)
uniform vec2 u_mobius_b;        // Möbius parameter b (complex)
uniform vec2 u_mobius_c;        // Möbius parameter c (complex)
uniform vec2 u_mobius_d;        // Möbius parameter d (complex)
uniform float u_inversion_radius; // Inversion radius
uniform vec2 u_inversion_center;  // Inversion center

#define PI 3.14159265359

// Complex arithmetic
vec2 cmul(vec2 a, vec2 b) {
```

```
    return vec2(a.x*b.x - a.y*b.y, a.x*b.y + a.y*b.x);
}

vec2 cdiv(vec2 a, vec2 b) {
    float denom = dot(b, b);
    return vec2(dot(a, b), a.y*b.x - a.x*b.y) / denom;
}

vec2 cconj(vec2 z) {
    return vec2(z.x, -z.y);
}

float cabs(vec2 z) {
    return length(z);
}

// Möbius transform: f(z) = (az+b)/(cz+d)
vec2 mobius(vec2 z, vec2 a, vec2 b, vec2 c, vec2 d) {
    vec2 num = cmul(a, z) + b;
    vec2 den = cmul(c, z) + d;
    return cdiv(num, den);
}

// Circle inversion: f(z) = R² / (z - c)‾ + c
vec2 inversion(vec2 z, vec2 center, float R) {
    vec2 dz = z - center;
    float r2 = dot(dz, dz);
    if (r2 < 0.0001) return vec2(1e6); // singularity
    return center + (R * R / r2) * dz;
}

// Hyperbolic disk map: f(z) = (z - a)/(1 - āz)
vec2 hyperbolic(vec2 z, vec2 a) {
    vec2 num = z - a;
    vec2 den = vec2(1.0, 0.0) - cmul(cconj(a), z);
    return cdiv(num, den);
}

// Environment reflection (simplified)
vec3 envReflect(vec2 uv) {
    float angle = atan(uv.y, uv.x);
    float r = length(uv);

    return vec3(
        0.5 + 0.5 * cos(angle * 2.0 + u_time * 0.3),
        0.5 + 0.5 * cos(angle * 3.0 + u_time * 0.5 + 1.0),
        0.5 + 0.5 * cos(angle * 5.0 + u_time * 0.7 + 2.0)
    ) * (1.0 - r * 0.5);
}

// Chrome shading with conformal distortion
```

```
vec3 chromeConformal(vec2 uv, vec2 mapped_uv, float map_type) {
    // Base environment
    vec3 base = envReflect(mapped_uv);

    // Chrome reflectivity varies with map type
    float reflectivity = 0.8;

    // Add geometric pattern based on original coordinates
    float grid = 0.0;
    if (map_type < 0.5) {
        // Möbius: circles stay circles
        float circles = abs(fract(cabs(uv) * 5.0) - 0.5) * 2.0;
        grid = smoothstep(0.9, 1.0, circles);
    } else if (map_type < 1.5) {
        // Inversion: radial lines
        float angle = atan(uv.y, uv.x);
        grid = smoothstep(0.95, 1.0, abs(sin(angle * 8.0)));
    } else if (map_type < 2.5) {
        // Hyperbolic: concentric circles
        float hyp_dist = cabs(hyperbolic(uv, vec2(0.0)));
        grid = smoothstep(0.9, 1.0, abs(fract(hyp_dist * 10.0) - 0.5) * 2.0);
    } else {
        // Schwarz-Christoffel: polygon edges
        float angle = atan(uv.y, uv.x);
        float polygon = abs(sin(angle * 5.0)) * cabs(uv);
        grid = smoothstep(0.8, 1.0, polygon);
    }

    // Combine
    vec3 color = base * reflectivity;
    color += vec3(grid * 0.3);

    // Specular highlight
    float highlight = pow(max(0.0, 1.0 - length(mapped_uv)), 4.0);
    color += vec3(highlight * 0.5);

    return color;
}

// Circle Limit tiling pattern
float circleLimitPattern(vec2 z) {
    // Poincaré disk: |z| < 1
    float r = cabs(z);
    if (r >= 1.0) return 0.0;

    // Hyperbolic distance from center
    float hyp_r = atanh(r);

    // Tile boundaries (simplified)
    float tiles = abs(fract(hyp_r * 3.0) - 0.5) * 2.0;
    float angle = atan(z.y, z.x);
```

```
    tiles += abs(fract(angle * 4.0 / PI) - 0.5) * 2.0;

    return smoothstep(1.8, 2.0, tiles);
}

void main() {
    vec2 uv = gl_FragCoord.xy / u_resolution.xy;
    vec2 p = uv * 2.0 - 1.0;
    p.x *= u_resolution.x / u_resolution.y;

    // Apply conformal map
    vec2 mapped;
    if (u_map_type < 0.5) {
        mapped = mobius(p, u_mobius_a, u_mobius_b, u_mobius_c, u_mobius_d);
    } else if (u_map_type < 1.5) {
        mapped = inversion(p, u_inversion_center, u_inversion_radius);
    } else if (u_map_type < 2.5) {
        mapped = hyperbolic(p, vec2(0.3 * cos(u_time), 0.3 * sin(u_time)));
    } else {
        // Schwarz-Christoffel approximation: power map
        vec2 logz = vec2(log(cabs(p)), atan(p.y, p.x));
        mapped = vec2(cos(logz.y * 0.4) * exp(logz.x * 0.4), sin(logz.y * 0.4) * exp(logz.x * 0.4));
    }

    // Chrome reflection through distorted space
    vec3 color = chromeConformal(p, mapped, u_map_type);

    // Circle limit pattern for hyperbolic mode
    if (u_map_type > 1.5 && u_map_type < 2.5) {
        float pattern = circleLimitPattern(p);
        color = mix(color, vec3(0.9, 0.95, 1.0), pattern * 0.3);
    }

    // Vignette
    float vignette = 1.0 - smoothstep(0.5, 1.5, length(p));
    color *= vignette;

    gl_FragColor = vec4(color, 1.0);
}
```

— JS MODULE —

```
// System 49: Escher Chrome — JS Module
// Conformal map composition, Möbius transforms, inversion animation

export class EscherChromeSystem {
    constructor() {
        this.mapType = 0; // 0=mobius, 1=inversion, 2=hyperbolic, 3=schwarz

        // Möbius parameters (complex a,b,c,d with ad-bc=1)
        this.mobiusA = {x: 1, y: 0};
```

```
        this.mobiusB = {x: 0, y: 0};
        this.mobiusC = {x: 0, y: 0};
        this.mobiusD = {x: 1, y: 0};

        // Inversion
        this.inversionRadius = 1.0;
        this.inversionCenter = {x: 0, y: 0};

        // Hyperbolic
        this.hyperbolicCenter = {x: 0, y: 0};

        // Animation
        this.time = 0;
    }

    // Set Möbius transform (normalize to ad-bc=1)
    setMobius(a, b, c, d) {
        // Compute determinant
        const det = (a.x * d.x - a.y * d.y) - (b.x * c.x - b.y * c.y);
        const scale = 1 / Math.sqrt(det);

        this.mobiusA = {x: a.x * scale, y: a.y * scale};
        this.mobiusB = {x: b.x * scale, y: b.y * scale};
        this.mobiusC = {x: c.x * scale, y: c.y * scale};
        this.mobiusD = {x: d.x * scale, y: d.y * scale};
    }

    // Compose two Möbius transforms
    composeMobius(a1, b1, c1, d1, a2, b2, c2, d2) {
        // Matrix multiplication
        const a = {
            x: a1.x*a2.x - a1.y*a2.y + b1.x*c2.x - b1.y*c2.y,
            y: a1.x*a2.y + a1.y*a2.x + b1.x*c2.y + b1.y*c2.x
        };
        const b = {
            x: a1.x*b2.x - a1.y*b2.y + b1.x*d2.x - b1.y*d2.y,
            y: a1.x*b2.y + a1.y*b2.x + b1.x*d2.y + b1.y*d2.x
        };
        const c = {
            x: c1.x*a2.x - c1.y*a2.y + d1.x*c2.x - d1.y*c2.y,
            y: c1.x*a2.y + c1.y*a2.x + d1.x*c2.y + d1.y*c2.x
        };
        const d = {
            x: c1.x*b2.x - c1.y*b2.y + d1.x*d2.x - d1.y*d2.y,
            y: c1.x*b2.y + c1.y*b2.x + d1.x*d2.y + d1.y*d2.x
        };
        return {a, b, c, d};
    }

    // Animate inversion center (for "Möbius Mirror")
    animateInversion(dt) {
```

```
        this.time += dt;
        this.inversionCenter = {
            x: Math.sin(this.time * 0.5) * 0.3,
            y: Math.cos(this.time * 0.7) * 0.3
        };
        this.inversionRadius = 0.5 + Math.sin(this.time * 0.3) * 0.3;
    }

    // Animate hyperbolic center
    animateHyperbolic(dt) {
        this.time += dt;
        this.hyperbolicCenter = {
            x: 0.3 * Math.cos(this.time * 0.4),
            y: 0.3 * Math.sin(this.time * 0.4)
        };
    }

    // Set map type
    setMapType(type) {
        this.mapType = type;
    }

    // Get uniforms for shader
    getUniforms() {
        return {
            u_time: this.time,
            u_map_type: this.mapType,
            u_mobius_a: [this.mobiusA.x, this.mobiusA.y],
            u_mobius_b: [this.mobiusB.x, this.mobiusB.y],
            u_mobius_c: [this.mobiusC.x, this.mobiusC.y],
            u_mobius_d: [this.mobiusD.x, this.mobiusD.y],
            u_inversion_radius: this.inversionRadius,
            u_inversion_center: [this.inversionCenter.x, this.inversionCenter.y]
        };
    }

    // Generate random Möbius transform (for "funhouse" effect)
    randomMobius() {
        const a = {x: 1 + Math.random(), y: Math.random()};
        const b = {x: Math.random(), y: Math.random()};
        const c = {x: Math.random() * 0.5, y: Math.random() * 0.5};
        const d = {x: 1 + Math.random(), y: Math.random()};
        this.setMobius(a, b, c, d);
    }

    // Circle limit tiling vertices (for hyperbolic mode)
    generateCircleLimitTiling(p, q, maxDepth = 3) {
        // Regular {p,q} tiling in hyperbolic plane
        // p = polygon sides, q = polygons meeting at vertex
        const polygons = [];
```

```
        // Central polygon
        const centralRadius = Math.sqrt(
            (Math.cos(PI/q) - Math.cos(PI/p)) /
            (Math.cos(PI/q) + Math.cos(PI/p))
        );

        for (let i = 0; i < p; i++) {
            const angle = (2 * PI * i) / p;
            polygons.push({
                x: centralRadius * Math.cos(angle),
                y: centralRadius * Math.sin(angle),
                radius: centralRadius
            });
        }

        return polygons;
    }
}

const PI = Math.PI;
```

==========

SYSTEM 50: NETWORK SONGS

==========

— DOCUMENTATION —

System 50: Network Songs

Overview

Network shine where graph Laplacian eigenvalues = color frequencies. The Fiedler vector ( ) detects communities — positive = warm, negative = cool, boundary = white flash. Each eigenmode is a different "dance" the network performs. The graph is a musical instrument made of light.

Mathematical Foundation

Graph Laplacian: `L = D - A` where D = degree matrix, A = adjacency matrix.

Spectral Decomposition: $L = U\Lambda U^T$ where Λ = diag(, , …,), = 0 … .

Key Eigenvalues:

- $\lambda_1 = 0$: constant eigenvector, trivial
- λ_2 (Fiedler value): algebraic connectivity. Small = graph nearly disconnected.
- λ_2 eigenvector (Fiedler vector): signs define communities. Zero-crossings = cuts.

Effective Resistance: $R_{eff}(i,j) = (e_i - e_j)^T L^+ (e_i - e_j)$ — measures how "connected" two nodes are.

GLSL Shader

See `shader.glsl` — node positions from eigenvectors, edge glow from effective resistance, community coloring from Fiedler vector.

JS Module

See `module.js` — graph construction, sparse Laplacian, power iteration for eigenvectors, effective resistance computation.

Showcase Builds

1. Fiedler Choir

Communities sing different notes. Fiedler vector signs = warm vs cool. Boundary nodes (near zero) pulse white — conflicted, torn between communities. The network's social tension made color.

2. Expander Pulse

Pulse spreads instantly through well-connected graph (expander). No bottlenecks. The flash reaches all nodes in O(log n) time. Network efficiency made visible.

3. Bridge Flash

Critical edges glow proportional to effective resistance. High resistance = vulnerable bridge. The glow map IS the vulnerability map. Cut the bright edges, graph falls apart.

4. Spectral Dance

Rapid cycling through eigenmodes. Network morphs between states. Each eigenmode is a different "dance" — some nodes always move together (eigenvector entry signs), some always opposite.

5. Random Walk Glow

Walker leaves heat trail. Well-connected areas visited often = bright. Bottlenecks = dark (walker gets stuck). The heat map IS the connectivity map.

Implementation Notes

- Use sparse matrix formats for large graphs
- Power iteration for dominant eigenvectors
- Lanczos algorithm for extremal eigenvalues
- Effective resistance: solve linear system with Laplacian pseudoinverse

— GLSL SHADER —

```
// System 50: Network Songs
// Graph eigenvalues = color frequencies, Fiedler vector = community detection

uniform float u_time;
uniform vec2 u_resolution;
uniform float u_node_count;
uniform float u_edge_count;
```

```glsl
uniform float u_mode;           // 0=fiedler, 1=expander, 2=bridge, 3=spectral, 4=randomwalk
uniform sampler2D u_node_positions; // RGBA = (x, y, fiedler, degree)
uniform sampler2D u_edge_data;    // RGB = (node_i, node_j, weight)
uniform float u_eigenmode;      // which eigenmode to display (0-4)

#define PI 3.14159265359
#define MAX_NODES 100
#define MAX_EDGES 200

// Hash for deterministic variation
float hash(float n) {
    return fract(sin(n * 12.9898) * 43758.5453);
}

// Node position from texture
vec4 getNode(float idx) {
    float u = (idx + 0.5) / u_node_count;
    return texture2D(u_node_positions, vec2(u, 0.5));
}

// Edge data from texture
vec3 getEdge(float idx) {
    float u = (idx + 0.5) / u_edge_count;
    return texture2D(u_edge_data, vec2(u, 0.5));
}

// Fiedler coloring: positive = warm, negative = cool, zero = white
vec3 fiedlerColor(float f) {
    float abs_f = abs(f);
    vec3 warm = vec3(1.0, 0.6, 0.2);  // orange
    vec3 cool = vec3(0.2, 0.5, 1.0);  // blue
    vec3 white = vec3(1.0, 1.0, 1.0); // boundary

    // Near zero = white flash
    float boundary = smoothstep(0.0, 0.1, abs_f) * smoothstep(0.3, 0.1, abs_f);

    vec3 color = mix(warm, cool, step(0.0, f));
    return mix(color, white, boundary);
}

// Glow based on connectivity (degree)
float connectivityGlow(float degree, float maxDegree) {
    return degree / maxDegree;
}

// Edge glow from effective resistance
float edgeGlow(float weight, float resistance) {
    return weight * (1.0 - resistance);
}

// Pulse propagation through network
```

```
float pulsePropagation(vec2 uv, float time) {
    float pulse = 0.0;
    float nodeCount = min(u_node_count, float(MAX_NODES));

    for (float i = 0.0; i < float(MAX_NODES); i++) {
        if (i >= nodeCount) break;

        vec4 node = getNode(i);
        vec2 pos = node.xy * 2.0 - 1.0;
        pos.x *= u_resolution.x / u_resolution.y;

        float dist = length(uv - pos);
        float wave = sin(dist * 20.0 - time * 5.0) * 0.5 + 0.5;
        float envelope = exp(-dist * dist * 10.0);

        pulse += wave * envelope * node.w; // weighted by degree
    }

    return pulse;
}

// Random walk heat trail
float randomWalkHeat(vec2 uv, float time) {
    float heat = 0.0;
    float walkerCount = 5.0;

    for (float w = 0.0; w < 5.0; w++) {
        float seed = w * 123.456;
        float x = hash(seed + time * 0.1) * 2.0 - 1.0;
        float y = hash(seed + time * 0.1 + 78.233) * 2.0 - 1.0;

        vec2 walkerPos = vec2(x, y);
        float d = length(uv - walkerPos);
        heat += exp(-d * d * 50.0);
    }

    return heat;
}

void main() {
    vec2 uv = gl_FragCoord.xy / u_resolution.xy;
    vec2 p = uv * 2.0 - 1.0;
    p.x *= u_resolution.x / u_resolution.y;

    vec3 color = vec3(0.05, 0.05, 0.1); // dark background

    float nodeCount = min(u_node_count, float(MAX_NODES));
    float edgeCount = min(u_edge_count, float(MAX_EDGES));

    if (u_mode < 0.5) {
        // Fiedler Choir
```

```
        for (float i = 0.0; i < float(MAX_NODES); i++) {
            if (i >= nodeCount) break;

            vec4 node = getNode(i);
            vec2 pos = node.xy * 2.0 - 1.0;
            pos.x *= u_resolution.x / u_resolution.y;

            float d = length(p - pos);
            float fiedler = node.z;
            float degree = node.w;

            // Node glow
            float glow = exp(-d * d * 200.0) * (0.5 + degree * 0.5);
            vec3 nodeColor = fiedlerColor(fiedler);

            // Boundary flash
            float boundary = smoothstep(0.0, 0.05, abs(fiedler)) * smoothstep(0.2, 0.05, abs(fiedler));
            nodeColor = mix(nodeColor, vec3(1.0), boundary * sin(u_time * 10.0) * 0.5 + 0.5);

            color += nodeColor * glow;
        }

        // Draw edges
        for (float e = 0.0; e < float(MAX_EDGES); e++) {
            if (e >= edgeCount) break;

            vec3 edge = getEdge(e);
            vec4 nodeA = getNode(edge.x);
            vec4 nodeB = getNode(edge.y);

            vec2 a = nodeA.xy * 2.0 - 1.0;
            vec2 b = nodeB.xy * 2.0 - 1.0;
            a.x *= u_resolution.x / u_resolution.y;
            b.x *= u_resolution.x / u_resolution.y;

            // Distance to line segment
            vec2 ab = b - a;
            float t = clamp(dot(p - a, ab) / dot(ab, ab), 0.0, 1.0);
            vec2 closest = a + ab * t;
            float d = length(p - closest);

            float edgeIntensity = exp(-d * d * 500.0) * edge.z * 0.3;
            color += vec3(0.5, 0.7, 1.0) * edgeIntensity;
        }

    } else if (u_mode < 1.5) {
        // Expander Pulse
        float pulse = pulsePropagation(p, u_time);
        color = vec3(0.1, 0.2, 0.3) + vec3(0.5, 0.8, 1.0) * pulse;

    } else if (u_mode < 2.5) {
```

```
        // Bridge Flash
        for (float e = 0.0; e < float(MAX_EDGES); e++) {
            if (e >= edgeCount) break;

            vec3 edge = getEdge(e);
            vec4 nodeA = getNode(edge.x);
            vec4 nodeB = getNode(edge.y);

            vec2 a = nodeA.xy * 2.0 - 1.0;
            vec2 b = nodeB.xy * 2.0 - 1.0;
            a.x *= u_resolution.x / u_resolution.y;
            b.x *= u_resolution.x / u_resolution.y;

            vec2 ab = b - a;
            float t = clamp(dot(p - a, ab) / dot(ab, ab), 0.0, 1.0);
            vec2 closest = a + ab * t;
            float d = length(p - closest);

            // Resistance = 1 / (weight * connectivity)
            float resistance = 1.0 / (edge.z * (nodeA.w + nodeB.w) * 0.5 + 0.01);
            float bridgeGlow = exp(-d * d * 300.0) * resistance * 2.0;

            // High resistance = bright red (vulnerable)
            vec3 edgeColor = mix(vec3(0.2, 0.8, 0.2), vec3(1.0, 0.2, 0.2), smoothstep(0.5, 1.0,
            ↪  resistance));
            color += edgeColor * bridgeGlow;
        }

    } else if (u_mode < 3.5) {
        // Spectral Dance
        float mode = u_eigenmode;
        float freq = 1.0 + mode * 0.5;

        for (float i = 0.0; i < float(MAX_NODES); i++) {
            if (i >= nodeCount) break;

            vec4 node = getNode(i);
            vec2 pos = node.xy * 2.0 - 1.0;
            pos.x *= u_resolution.x / u_resolution.y;

            float d = length(p - pos);
            float phase = sin(u_time * freq + node.z * PI * 2.0) * 0.5 + 0.5;
            float glow = exp(-d * d * 150.0) * phase;

            vec3 modeColor = vec3(
                sin(mode * 0.5) * 0.5 + 0.5,
                sin(mode * 0.5 + 2.0) * 0.5 + 0.5,
                sin(mode * 0.5 + 4.0) * 0.5 + 0.5
            );

            color += modeColor * glow;
```

```
        }

    } else {
        // Random Walk Glow
        float heat = randomWalkHeat(p, u_time);
        color = vec3(0.05, 0.05, 0.1) + vec3(1.0, 0.4, 0.1) * heat;
    }

    // Tone mapping
    color = color / (1.0 + color * 0.3);

    gl_FragColor = vec4(color, 1.0);
}
```

— JS MODULE —

```
// System 50: Network Songs — JS Module
// Graph Laplacian, spectral decomposition, effective resistance

export class NetworkSongsSystem {
    constructor(nodeCount = 50) {
        this.nodeCount = nodeCount;
        this.nodes = []; // {x, y, fiedler, degree, eigenvectors[]}
        this.edges = []; // {i, j, weight}
        this.adjacency = new Array(nodeCount).fill(null).map(() => new Array(nodeCount).fill(0));
        this.laplacian = new Array(nodeCount).fill(null).map(() => new Array(nodeCount).fill(0));
        this.eigenvalues = [];
        this.eigenvectors = [];
        this.mode = 0;
        this.eigenmode = 0;
    }

    // Generate random graph (Erdős–Rényi)
    generateRandomGraph(p = 0.1) {
        this.edges = [];
        this.adjacency = new Array(this.nodeCount).fill(null).map(() => new
        ↪  Array(this.nodeCount).fill(0));

        for (let i = 0; i < this.nodeCount; i++) {
            for (let j = i + 1; j < this.nodeCount; j++) {
                if (Math.random() < p) {
                    const weight = Math.random();
                    this.edges.push({i, j, weight});
                    this.adjacency[i][j] = this.adjacency[j][i] = weight;
                }
            }
        }

        this.computeLaplacian();
        this.computeSpectral();
    }
```

```
// Generate expander graph (well-connected)
generateExpanderGraph() {
    this.edges = [];
    this.adjacency = new Array(this.nodeCount).fill(null).map(() => new
    ↪ Array(this.nodeCount).fill(0));

    // Random regular graph approximation
    const degree = 4;
    for (let i = 0; i < this.nodeCount; i++) {
        for (let d = 1; d <= degree / 2; d++) {
            const j = (i + d) % this.nodeCount;
            const weight = 1.0;
            this.edges.push({i, j, weight});
            this.adjacency[i][j] = this.adjacency[j][i] = weight;
        }
    }

    this.computeLaplacian();
    this.computeSpectral();
}

// Generate graph with communities (for Fiedler demo)
generateCommunityGraph(communities = 2) {
    this.edges = [];
    this.adjacency = new Array(this.nodeCount).fill(null).map(() => new
    ↪ Array(this.nodeCount).fill(0));

    const nodesPerCommunity = Math.floor(this.nodeCount / communities);

    for (let i = 0; i < this.nodeCount; i++) {
        for (let j = i + 1; j < this.nodeCount; j++) {
            const ci = Math.floor(i / nodesPerCommunity);
            const cj = Math.floor(j / nodesPerCommunity);

            // High probability within community, low between
            const p = (ci === cj) ? 0.3 : 0.02;

            if (Math.random() < p) {
                const weight = (ci === cj) ? 1.0 : 0.3;
                this.edges.push({i, j, weight});
                this.adjacency[i][j] = this.adjacency[j][i] = weight;
            }
        }
    }

    this.computeLaplacian();
    this.computeSpectral();
}

// Compute Laplacian matrix
```

```
    computeLaplacian() {
        const n = this.nodeCount;
        this.laplacian = new Array(n).fill(null).map(() => new Array(n).fill(0));

        for (let i = 0; i < n; i++) {
            let degree = 0;
            for (let j = 0; j < n; j++) {
                degree += this.adjacency[i][j];
            }
            this.laplacian[i][i] = degree;
            for (let j = 0; j < n; j++) {
                if (i !== j) {
                    this.laplacian[i][j] = -this.adjacency[i][j];
                }
            }
        }
    }

    // Power iteration for eigenvectors
    computeSpectral(numEigenvectors = 5) {
        const n = this.nodeCount;
        this.eigenvectors = [];
        this.eigenvalues = [];

        // Simple power iteration (for demo — use Lanczos in production)
        for (let k = 0; k < numEigenvectors; k++) {
            let v = new Array(n).fill(0).map(() => Math.random() - 0.5);

            // Gram-Schmidt against previous eigenvectors
            for (let prev = 0; prev < k; prev++) {
                const prevVec = this.eigenvectors[prev];
                let dot = 0;
                for (let i = 0; i < n; i++) dot += v[i] * prevVec[i];
                for (let i = 0; i < n; i++) v[i] -= dot * prevVec[i];
            }

            // Normalize
            let norm = Math.sqrt(v.reduce((s, x) => s + x * x, 0));
            v = v.map(x => x / norm);

            // Power iteration
            for (let iter = 0; iter < 100; iter++) {
                let newV = new Array(n).fill(0);
                for (let i = 0; i < n; i++) {
                    for (let j = 0; j < n; j++) {
                        newV[i] += this.laplacian[i][j] * v[j];
                    }
                }

                // Gram-Schmidt
                for (let prev = 0; prev < k; prev++) {
```

```
                const prevVec = this.eigenvectors[prev];
                let dot = 0;
                for (let i = 0; i < n; i++) dot += newV[i] * prevVec[i];
                for (let i = 0; i < n; i++) newV[i] -= dot * prevVec[i];
            }

            norm = Math.sqrt(newV.reduce((s, x) => s + x * x, 0));
            v = newV.map(x => x / norm);
        }

        // Compute eigenvalue
        let lambda = 0;
        for (let i = 0; i < n; i++) {
            let lv = 0;
            for (let j = 0; j < n; j++) {
                lv += this.laplacian[i][j] * v[j];
            }
            lambda += v[i] * lv;
        }

        this.eigenvectors.push(v);
        this.eigenvalues.push(lambda);
    }

    // Update node data
    this.updateNodeData();
}

// Update node positions and spectral data
updateNodeData() {
    this.nodes = [];
    const n = this.nodeCount;

    for (let i = 0; i < n; i++) {
        const degree = this.adjacency[i].reduce((s, w) => s + w, 0);
        const fiedler = this.eigenvectors[1] ? this.eigenvectors[1][i] : 0;

        // Position from first two non-trivial eigenvectors
        const x = this.eigenvectors[1] ? (this.eigenvectors[1][i] + 1) * 0.5 : Math.random();
        const y = this.eigenvectors[2] ? (this.eigenvectors[2][i] + 1) * 0.5 : Math.random();

        this.nodes.push({
            x, y,
            fiedler,
            degree,
            eigenvectors: this.eigenvectors.map(v => v[i])
        });
    }
}

// Compute effective resistance between two nodes
```

```
    effectiveResistance(i, j) {
        // Simplified: use pseudoinverse approximation
        // In production: solve Lx = e_i - e_j, return x[i] - x[j]
        const n = this.nodeCount;

        // Use spectral formula: R_eff = sum_k (v_k[i] - v_k[j])² / lambda_k
        let resistance = 0;
        for (let k = 1; k < this.eigenvectors.length; k++) {
            const diff = this.eigenvectors[k][i] - this.eigenvectors[k][j];
            resistance += diff * diff / (this.eigenvalues[k] + 0.001);
        }

        return Math.min(resistance, 1.0);
    }

    // Get node position texture data (for shader)
    getNodeTextureData() {
        const data = new Float32Array(this.nodeCount * 4);
        for (let i = 0; i < this.nodeCount; i++) {
            const node = this.nodes[i];
            data[i * 4] = node.x;
            data[i * 4 + 1] = node.y;
            data[i * 4 + 2] = node.fiedler;
            data[i * 4 + 3] = node.degree;
        }
        return data;
    }

    // Get edge texture data (for shader)
    getEdgeTextureData() {
        const data = new Float32Array(this.edges.length * 4);
        for (let i = 0; i < this.edges.length; i++) {
            const edge = this.edges[i];
            data[i * 4] = edge.i;
            data[i * 4 + 1] = edge.j;
            data[i * 4 + 2] = edge.weight;
            data[i * 4 + 3] = this.effectiveResistance(edge.i, edge.j);
        }
        return data;
    }

    // Get uniforms for shader
    getUniforms() {
        return {
            u_node_count: this.nodeCount,
            u_edge_count: this.edges.length,
            u_mode: this.mode,
            u_eigenmode: this.eigenmode,
            u_time: performance.now() * 0.001
        };
    }
```

```
    setMode(mode) {
        this.mode = mode;
    }

    setEigenmode(mode) {
        this.eigenmode = mode;
    }
}
```

==========

SYSTEM 51: OPTIMAL BLOOM

==========

— DOCUMENTATION —

System 51: Optimal Bloom

Overview

Shine that follows minimum-jerk trajectories — 5th-order polynomial birth/death. Not random spray, but smooth, natural arcs. Birth = warm orange, cruise = stable blue, death = violet fade. Elastic collisions conserve momentum. Bang-bang control = staccato rhythms. Hamiltonian orbits = phase space made color.

Mathematical Foundation

Minimum-Jerk Trajectory: The smoothest path between two points. Jerk = 3rd derivative of position. Minimizing $\int \text{jerk}^2\, dt$ gives 5th-order polynomial.

Position as function of time:

$$x(t) = x_0 + (x_1 - x_0) \cdot (10\tau^3 - 15\tau^4 + 6\tau^5)$$

where $\tau = t/T$ (normalized time).

Bang-Bang Control: All-or-nothing control. Switch between extremes. Optimal for time-minimization. Creates staccato, rhythmic patterns.

Hamiltonian Mechanics: $H = T + V$ (kinetic + potential). Conservation laws = symmetries. Phase space trajectory = orbit.

Elastic Collisions: Momentum conserved. $m_1v_1 + m_2v_2 = m_1v_1' + m_2v_2'$.

GLSL Shader

See `shader.glsl` — minimum-jerk sparkle trajectories, elastic collision response, Hamiltonian orbit coloring.

JS Module

See `module.js` — 5th-order polynomial evaluation, collision detection, bang-bang timing, phase space tracking.

Showcase Builds

1. Minimum-Jerk Fireworks

Sparks follow natural arcs, ballet not explosion. Each sparkle born with minimum-jerk trajectory. The motion looks "alive" because it follows biological movement principles.

2. Geodesic Constellation

Stars connected by shortest paths through curved space. The connections are geodesics — straight lines in curved geometry. The constellation is a map of spatial structure.

3. Elastic Garden

Colliding sparkles with fission/fusion. Population dynamics. Sparkles split when too energetic, merge when close and slow. Ecosystem of light particles.

4. Bang-Bang Bloom

All-or-nothing flashes. Optimal timing encodes information. The pattern of on/off IS the message. Morse code made optical, but with math-optimal timing.

5. Hamiltonian Orbit

Color = momentum, brightness = speed. Phase space visible. The orbit traces a contour of constant energy. The sparkle is a particle in a potential well.

Implementation Notes

- Minimum-jerk: precompute 5th-order coefficients
- Collision: grid-based spatial hashing for O(1) neighbor lookup
- Bang-bang: switch times computed from boundary conditions
- Hamiltonian: symplectic integrator (Verlet/Leapfrog) for energy conservation

— GLSL SHADER —

```
// System 51: Optimal Bloom
// Minimum-jerk trajectories, elastic collisions, Hamiltonian orbits

uniform float u_time;
uniform vec2 u_resolution;
uniform float u_sparkle_count;
uniform float u_mode;           // 0=fireworks, 1=geodesic, 2=elastic, 3=bangbang, 4=hamiltonian
uniform sampler2D u_sparkle_data; // RGBA = (x, y, vx, vy)
uniform float u_lifespan;       // sparkle lifetime

#define PI 3.14159265359
#define MAX_SPARKLES 150
```

```
// 5th-order minimum-jerk polynomial
// x(t) = x0 + (x1-x0) * (10*t^3 - 15*t^4 + 6*t^5)
float minJerk(float t) {
    float t2 = t * t;
    float t3 = t2 * t;
    return 10.0 * t3 - 15.0 * t2 * t2 + 6.0 * t3 * t2;
}

// Derivative of minimum-jerk (velocity profile)
float minJerkVel(float t) {
    float t2 = t * t;
    return 30.0 * t2 - 60.0 * t2 * t + 30.0 * t2 * t2;
}

// Sparkle position from birth params
vec2 sparklePosition(vec2 birthPos, vec2 targetPos, float birthTime, float currentTime, float lifespan)
 ↪  {
    float t = clamp((currentTime - birthTime) / lifespan, 0.0, 1.0);
    float mj = minJerk(t);
    return mix(birthPos, targetPos, mj);
}

// Sparkle velocity
vec2 sparkleVelocity(vec2 birthPos, vec2 targetPos, float birthTime, float currentTime, float lifespan)
 ↪  {
    float t = clamp((currentTime - birthTime) / lifespan, 0.0, 1.0);
    float v = minJerkVel(t);
    return (targetPos - birthPos) / lifespan * v;
}

// Life phase color: birth=warm, cruise=blue, death=violet
vec3 lifeColor(float t) {
    // t: 0=birth, 0.5=cruise, 1=death
    vec3 birth = vec3(1.0, 0.6, 0.2);   // orange
    vec3 cruise = vec3(0.2, 0.6, 1.0);  // blue
    vec3 death = vec3(0.6, 0.2, 0.8);   // violet

    if (t < 0.2) {
        return mix(birth, cruise, t / 0.2);
    } else if (t > 0.8) {
        return mix(cruise, death, (t - 0.8) / 0.2);
    } else {
        return cruise;
    }
}

// Elastic collision response (simplified)
vec2 elasticBounce(vec2 pos, vec2 vel, float radius) {
    vec2 newVel = vel;
```

```
    // Bounce off boundaries
    if (pos.x < -1.0 + radius || pos.x > 1.0 - radius) newVel.x *= -0.9;
    if (pos.y < -1.0 + radius || pos.y > 1.0 - radius) newVel.y *= -0.9;

    return newVel;
}

// Hamiltonian orbit: color from phase space (position, velocity)
vec3 hamiltonianColor(vec2 pos, vec2 vel) {
    // Momentum color
    float speed = length(vel);
    float hue = atan(vel.y, vel.x) / (2.0 * PI) + 0.5;

    // HSV to RGB
    vec3 c = vec3(hue, 0.8, speed * 2.0);
    vec3 rgb;
    float h = c.x * 6.0;
    float i = floor(h);
    float f = h - i;
    float q = c.z * (1.0 - c.y * f);
    float p = c.z * (1.0 - c.y);
    float t = c.z * (1.0 - c.y * (1.0 - f));

    if (i == 0.0) rgb = vec3(c.z, t, p);
    else if (i == 1.0) rgb = vec3(q, c.z, p);
    else if (i == 2.0) rgb = vec3(p, c.z, t);
    else if (i == 3.0) rgb = vec3(p, q, c.z);
    else if (i == 4.0) rgb = vec3(t, p, c.z);
    else rgb = vec3(c.z, p, q);

    return rgb;
}

// Bang-bang flash pattern
float bangBang(float time, float period, float dutyCycle) {
    float phase = fract(time / period);
    return step(phase, dutyCycle);
}

// Geodesic distance in curved space (simplified)
float geodesicDist(vec2 a, vec2 b) {
    // Flat space geodesic = straight line
    return length(a - b);
}

void main() {
    vec2 uv = gl_FragCoord.xy / u_resolution.xy;
    vec2 p = uv * 2.0 - 1.0;
    p.x *= u_resolution.x / u_resolution.y;

    vec3 color = vec3(0.05, 0.05, 0.1);
```

```
float sparkleCount = min(u_sparkle_count, float(MAX_SPARKLES));
float currentTime = u_time;

if (u_mode < 0.5) {
    // Minimum-Jerk Fireworks
    for (float i = 0.0; i < float(MAX_SPARKLES); i++) {
        if (i >= sparkleCount) break;

        float seed = i * 123.456;
        vec2 birthPos = vec2(hash(seed) * 2.0 - 1.0, hash(seed + 1.0) * 2.0 - 1.0);
        vec2 targetPos = vec2(hash(seed + 2.0) * 2.0 - 1.0, hash(seed + 3.0) * 2.0 - 1.0);
        float birthTime = hash(seed + 4.0) * 10.0;
        float lifespan = 2.0 + hash(seed + 5.0) * 3.0;

        vec2 pos = sparklePosition(birthPos, targetPos, birthTime, currentTime, lifespan);
        float t = clamp((currentTime - birthTime) / lifespan, 0.0, 1.0);

        // Only show if alive
        if (currentTime >= birthTime && currentTime <= birthTime + lifespan) {
            float d = length(p - pos);
            float size = 0.02 * (1.0 - t * 0.5);
            float glow = exp(-d * d / (size * size));

            vec3 col = lifeColor(t);
            color += col * glow * 0.8;
        }
    }

} else if (u_mode < 1.5) {
    // Geodesic Constellation
    float nodeCount = 8.0;
    vec2 nodes[8];
    for (float i = 0.0; i < 8.0; i++) {
        float angle = i / nodeCount * TAU + u_time * 0.1;
        nodes[int(i)] = vec2(cos(angle), sin(angle)) * 0.5;
    }

    // Draw nodes
    for (float i = 0.0; i < 8.0; i++) {
        float d = length(p - nodes[int(i)]);
        float glow = exp(-d * d * 200.0);
        color += vec3(0.8, 0.9, 1.0) * glow;
    }

    // Draw geodesic connections
    for (float i = 0.0; i < 8.0; i++) {
        for (float j = i + 1.0; j < 8.0; j++) {
            vec2 a = nodes[int(i)];
            vec2 b = nodes[int(j)];
            vec2 ab = b - a;
```

```
            float t = clamp(dot(p - a, ab) / dot(ab, ab), 0.0, 1.0);
            vec2 closest = a + ab * t;
            float d = length(p - closest);
            float line = exp(-d * d * 300.0) * 0.2;
            color += vec3(0.3, 0.5, 0.8) * line;
        }
    }

} else if (u_mode < 2.5) {
    // Elastic Garden
    for (float i = 0.0; i < float(MAX_SPARKLES); i++) {
        if (i >= sparkleCount) break;

        float seed = i * 123.456;
        float angle = hash(seed) * TAU;
        float speed = 0.2 + hash(seed + 1.0) * 0.5;
        vec2 vel = vec2(cos(angle), sin(angle)) * speed;
        vec2 pos = vec2(hash(seed + 2.0) * 2.0 - 1.0, hash(seed + 3.0) * 2.0 - 1.0);

        // Animate position
        pos += vel * fract(currentTime * 0.3 + hash(seed + 4.0));
        pos = fract(pos * 0.5 + 0.5) * 2.0 - 1.0;

        float d = length(p - pos);
        float size = 0.015 + hash(seed + 5.0) * 0.02;
        float glow = exp(-d * d / (size * size));

        // Color from velocity
        vec3 col = hamiltonianColor(pos, vel);
        color += col * glow;
    }

} else if (u_mode < 3.5) {
    // Bang-Bang Bloom
    float flash = bangBang(currentTime, 1.0, 0.3);
    float flash2 = bangBang(currentTime, 0.7, 0.5);
    float flash3 = bangBang(currentTime, 1.3, 0.2);

    vec3 flashColor = vec3(flash, flash2 * 0.5, flash3) * 0.5;

    // Stroboscopic pattern
    float pattern = step(0.5, fract(p.x * 5.0 + currentTime * 2.0)) *
                    step(0.5, fract(p.y * 5.0 + currentTime * 3.0));

    color = mix(vec3(0.05, 0.05, 0.1), flashColor, pattern);

} else {
    // Hamiltonian Orbit
    float orbitCount = 5.0;
    for (float i = 0.0; i < 5.0; i++) {
        float seed = i * 789.123;
```

```
            float energy = 0.5 + hash(seed) * 2.0;
            float phase = currentTime * (0.5 + hash(seed + 1.0));

            // Simple harmonic oscillator orbit
            vec2 pos = vec2(
                cos(phase * energy) * (0.2 + hash(seed + 2.0) * 0.3),
                sin(phase * energy * (1.0 + hash(seed + 3.0) * 0.5)) * (0.2 + hash(seed + 4.0) * 0.3)
            );

            vec2 vel = vec2(
                -sin(phase * energy) * energy,
               cos(phase * energy * (1.0 + hash(seed + 3.0) * 0.5)) * energy * (1.0 + hash(seed + 3.0) *
                ↪  0.5)
            );

            float d = length(p - pos);
            float size = 0.02;
            float glow = exp(-d * d / (size * size));

            vec3 col = hamiltonianColor(pos, vel);
            color += col * glow * 0.6;

            // Trail
            for (float t = 0.0; t < 10.0; t++) {
                float trailPhase = phase - t * 0.05;
                vec2 trailPos = vec2(
                    cos(trailPhase * energy) * (0.2 + hash(seed + 2.0) * 0.3),
                   sin(trailPhase * energy * (1.0 + hash(seed + 3.0) * 0.5)) * (0.2 + hash(seed + 4.0) *
                    ↪  0.3)
                );
                float td = length(p - trailPos);
                float trailGlow = exp(-td * td / (size * size * 2.0)) * (1.0 - t / 10.0) * 0.1;
                color += col * trailGlow;
            }
        }
    }

    // Tone mapping
    color = color / (1.0 + color * 0.3);

    gl_FragColor = vec4(color, 1.0);
}

float hash(float n) {
    return fract(sin(n * 12.9898) * 43758.5453);
}
```

— JS MODULE —

```
// System 51: Optimal Bloom — JS Module
// Minimum-jerk trajectories, collision detection, bang-bang control
```

```
export class OptimalBloomSystem {
    constructor(maxSparkles = 150) {
        this.sparkles = []; // {birthPos, targetPos, birthTime, lifespan, mass, velocity}
        this.maxSparkles = maxSparkles;
        this.mode = 0;
        this.time = 0;
        this.lifespan = 3.0;
    }

    // 5th-order minimum-jerk polynomial evaluation
    // x(t) = x0 + (x1-x0) * (10*t^3 - 15*t^4 + 6*t^5)
    minJerk(t) {
        const t2 = t * t;
        const t3 = t2 * t;
        return 10 * t3 - 15 * t2 * t2 + 6 * t3 * t2;
    }

    minJerkVelocity(t) {
        const t2 = t * t;
        return 30 * t2 - 60 * t2 * t + 30 * t2 * t2;
    }

    // Spawn a sparkle with minimum-jerk trajectory
    spawnSparkle(birthPos, targetPos, lifespan = 3.0) {
        if (this.sparkles.length >= this.maxSparkles) {
            // Remove oldest
            this.sparkles.shift();
        }

        this.sparkles.push({
            birthPos: {...birthPos},
            targetPos: {...targetPos},
            birthTime: this.time,
            lifespan,
            mass: 0.5 + Math.random() * 1.0,
            velocity: {x: 0, y: 0},
            alive: true
        });
    }

    // Update all sparkles
    update(dt) {
        this.time += dt;

        for (let s of this.sparkles) {
            if (!s.alive) continue;

            const age = this.time - s.birthTime;
            const t = Math.min(age / s.lifespan, 1.0);
```

```
            // Compute position from minimum-jerk
            const mj = this.minJerk(t);
            s.position = {
                x: s.birthPos.x + (s.targetPos.x - s.birthPos.x) * mj,
                y: s.birthPos.y + (s.targetPos.y - s.birthPos.y) * mj
            };

            // Compute velocity
            const v = this.minJerkVelocity(t);
            s.velocity = {
                x: (s.targetPos.x - s.birthPos.x) / s.lifespan * v,
                y: (s.targetPos.y - s.birthPos.y) / s.lifespan * v
            };

            // Check death
            if (age >= s.lifespan) {
                s.alive = false;
            }
        }

        // Remove dead sparkles
        this.sparkles = this.sparkles.filter(s => s.alive);

        // Handle collisions (Elastic Garden mode)
        if (this.mode === 2) {
            this.handleCollisions();
        }
    }

    // Elastic collision between two sparkles
    handleCollisions() {
        const restitution = 0.9;
        const mergeThreshold = 0.05;

        for (let i = 0; i < this.sparkles.length; i++) {
            for (let j = i + 1; j < this.sparkles.length; j++) {
                const a = this.sparkles[i];
                const b = this.sparkles[j];

                if (!a.alive || !b.alive) continue;

                const dx = b.position.x - a.position.x;
                const dy = b.position.y - a.position.y;
                const dist = Math.sqrt(dx * dx + dy * dy);
                const minDist = 0.05; // collision radius

                if (dist < minDist) {
                    // Collision response
                    const nx = dx / dist;
                    const ny = dy / dist;
```

```
                // Relative velocity
                const dvx = b.velocity.x - a.velocity.x;
                const dvy = b.velocity.y - a.velocity.y;
                const relVel = dvx * nx + dvy * ny;

                if (relVel < 0) {
                    // Elastic collision
                    const totalMass = a.mass + b.mass;
                    const impulse = 2 * relVel / totalMass * restitution;

                    a.velocity.x += impulse * b.mass * nx;
                    a.velocity.y += impulse * b.mass * ny;
                    b.velocity.x -= impulse * a.mass * nx;
                    b.velocity.y -= impulse * a.mass * ny;
                }

                // Fission/Fusion (Elastic Garden)
                const totalEnergy = 0.5 * a.mass * (a.velocity.x**2 + a.velocity.y**2) +
                                    0.5 * b.mass * (b.velocity.x**2 + b.velocity.y**2);

                if (totalEnergy > 2.0 && a.mass > 0.5) {
                    // Fission: split high-energy sparkle
                    a.mass *= 0.7;
                    this.spawnSparkle(
                        a.position,
                        {x: a.position.x + a.velocity.x, y: a.position.y + a.velocity.y},
                        a.lifespan * 0.5
                    );
                } else if (dist < mergeThreshold && a.mass + b.mass < 2.0) {
                    // Fusion: merge close, slow sparkles
                    a.mass += b.mass * 0.5;
                  a.velocity.x = (a.mass * a.velocity.x + b.mass * b.velocity.x) / (a.mass + b.mass);
                  a.velocity.y = (a.mass * a.velocity.y + b.mass * b.velocity.y) / (a.mass + b.mass);
                    b.alive = false;
                }
            }
        }
    }
}

// Bang-bang control: generate staccato flash pattern
bangBangPattern(period = 1.0, dutyCycle = 0.3) {
    const phase = (this.time % period) / period;
    return phase < dutyCycle ? 1.0 : 0.0;
}

// Hamiltonian orbit: update with symplectic integrator
hamiltonianStep(sparkle, dt) {
    // Simple harmonic oscillator: H = p²/2m + kx²/2
    const k = 1.0; // spring constant
    const m = sparkle.mass;
```

```
        // Leapfrog integration
        sparkle.velocity.x -= (k / m) * sparkle.position.x * dt * 0.5;
        sparkle.velocity.y -= (k / m) * sparkle.position.y * dt * 0.5;

        sparkle.position.x += sparkle.velocity.x * dt;
        sparkle.position.y += sparkle.velocity.y * dt;

        sparkle.velocity.x -= (k / m) * sparkle.position.x * dt * 0.5;
        sparkle.velocity.y -= (k / m) * sparkle.position.y * dt * 0.5;
    }

    // Get sparkle data for shader
    getSparkleData() {
        const data = new Float32Array(this.maxSparkles * 4);
        for (let i = 0; i < this.sparkles.length; i++) {
            const s = this.sparkles[i];
            data[i * 4] = s.position.x;
            data[i * 4 + 1] = s.position.y;
            data[i * 4 + 2] = s.velocity.x;
            data[i * 4 + 3] = s.velocity.y;
        }
        return data;
    }

    // Get uniforms for shader
    getUniforms() {
        return {
            u_sparkle_count: this.sparkles.length,
            u_mode: this.mode,
            u_lifespan: this.lifespan,
            u_time: this.time
        };
    }

    // Auto-spawn fireworks
    autoFireworks() {
        if (Math.random() < 0.05) {
            const birthPos = {
                x: (Math.random() - 0.5) * 2,
                y: (Math.random() - 0.5) * 2
            };
            const targetPos = {
                x: (Math.random() - 0.5) * 2,
                y: (Math.random() - 0.5) * 2
            };
            this.spawnSparkle(birthPos, targetPos);
        }
    }

    setMode(mode) {
```

```
        this.mode = mode;
    }
}
```

==========

SYSTEM 52: GRAVITY CHROME

==========

— DOCUMENTATION —

System 52: Gravity Chrome

Overview

Chrome reflection through curved space — geodesics follow surface geometry. Positive curvature = converging geodesics (warm, focal points). Negative curvature = diverging geodesics (cool, chaotic scattering). The distortion IS the mass distribution. Like general relativity made material.

Mathematical Foundation

Geodesic Equation: $d^2x^\mu/d\tau^2 + \Gamma^\mu_{\nu\rho} dx^\nu/d\tau dx^\rho/d\tau = 0$

Christoffel Symbols: $\Gamma^\mu_{\nu\rho} = \frac{1}{2}g^{\mu\sigma}(\partial_\nu g_{\rho\sigma} + \partial_\rho g_{\nu\sigma} - \partial_\sigma g_{\nu\rho})$

Curvature:

- Positive ($K > 0$): sphere, converging geodesics, focal points
- Zero ($K = 0$): flat, straight lines
- Negative ($K < 0$): saddle/hyperbolic, diverging geodesics

Metric Examples:

- Sphere: $ds^2 = R^2(d\theta^2 + \sin^2\theta\, d\varphi^2)$
- Poincaré disk: $ds^2 = 4R^2(dx^2+dy^2)/(1-x^2-y^2)^2$
- Schwarzschild: $ds^2 = -(1-2M/r)dt^2 + (1-2M/r)^{-1}dr^2 + r^2d\Omega^2$

GLSL Shader

See `shader.glsl` — geodesic ray tracing on curved surfaces, metric-based distortion, curvature coloring.

JS Module

See `module.js` — metric tensor computation, geodesic integration (Runge-Kutta), mass distribution editing.

Showcase Builds

1. Black Hole Mirror

Light spirals into center, redshifts, infinite reflection ring at photon sphere (r = 3M). The mirror IS the event horizon. Looking at it = looking at infinity.

2. Saddle Lens

Negative curvature tears environment apart. Exponential divergence of geodesics. The lens shows a world where parallel lines don't exist — everything diverges.

3. Gravity Well Jewelry

Wearable curved space. The "stone" is a window into different geometry. Mass distribution = gemstone cut. The wearer carries a pocket universe.

4. Geodesic Dome

Great circle chrome seams, spherical geometry made physical. Each seam is a geodesic. The dome is a map of shortest paths on a sphere.

5. Wormhole Portal

Two mouths connected by throat. See other environment through curved tunnel. The connection is an Einstein-Rosen bridge. Topology made visible.

Implementation Notes

- Geodesic integration: 4th-order Runge-Kutta
- Metric tensor: precompute for common spaces
- Redshift: `z = (λ_obs - λ_emit)/λ_emit = 1/√(1-2M/r) - 1`
- Photon sphere: unstable circular orbit at r = 3M

— GLSL SHADER —

```
// System 52: Gravity Chrome
// Geodesic reflection through curved space — general relativity made material

uniform float u_time;
uniform vec2 u_resolution;
uniform float u_curvature_type; // 0=sphere, 1=hyperbolic, 2=schwarzschild, 3=saddle, 4=wormhole
uniform float u_mass;           // mass parameter (for Schwarzschild)
uniform float u_curvature;      // curvature magnitude
uniform vec2 u_lens_center;     // center of curvature

#define PI 3.14159265359
#define EPS 0.001

// Metric tensor components for different spaces
// Sphere: ds² = R²(dθ² + sin²θ dφ²)
// Hyperbolic: ds² = R²(dθ² + sinh²θ dφ²)
// Schwarzschild: ds² = -(1-2M/r)dt² + (1-2M/r)⁻¹dr² + r²dΩ²
```

```
// Sphere metric
vec2 sphereMetric(vec2 coords, float R) {
    float theta = coords.x;
    float g_theta = R * R;
    float g_phi = R * R * sin(theta) * sin(theta);
    return vec2(g_theta, g_phi);
}

// Hyperbolic (Poincaré disk) metric
vec2 hyperbolicMetric(vec2 coords, float R) {
    float r2 = dot(coords, coords);
    float factor = 4.0 * R * R / ((1.0 - r2) * (1.0 - r2));
    return vec2(factor, factor);
}

// Schwarzschild metric (simplified for spatial part)
vec2 schwarzschildMetric(vec2 coords, float M) {
    float r = length(coords);
    float r_s = 2.0 * M; // Schwarzschild radius

    if (r < r_s + EPS) {
        // Inside event horizon — return large values
        return vec2(1e6, 1e6);
    }

    float g_r = 1.0 / (1.0 - r_s / r);
    float g_theta = r * r;
    return vec2(g_r, g_theta);
}

// Saddle (hyperbolic paraboloid) metric
vec2 saddleMetric(vec2 coords, float K) {
    float x = coords.x;
    float y = coords.y;
    float g_xx = 1.0 + K * K * x * x;
    float g_yy = 1.0 + K * K * y * y;
    return vec2(g_xx, g_yy);
}

// Wormhole metric (Morris-Thorne, simplified)
vec2 wormholeMetric(vec2 coords, float throat) {
    float r = length(coords);
    float r2 = r * r + throat * throat;
    float g_r = 1.0;
    float g_theta = r2;
    return vec2(g_r, g_theta);
}

// Geodesic step (simplified Euler integration)
// Returns new position and direction
```

```
vec4 geodesicStep(vec2 pos, vec2 dir, float dt, float curvatureType, float param) {
    vec2 metric;
    vec2 dmetric;

    if (curvatureType < 0.5) {
        // Sphere
        metric = sphereMetric(pos, param);
        dmetric = vec2(
            2.0 * param * param * sin(pos.x) * cos(pos.x),
            0.0
        );
    } else if (curvatureType < 1.5) {
        // Hyperbolic
        metric = hyperbolicMetric(pos, param);
        float r2 = dot(pos, pos);
        float factor = 16.0 * param * param * r2 / pow(1.0 - r2, 3);
        dmetric = vec2(factor, factor);
    } else if (curvatureType < 2.5) {
        // Schwarzschild
        metric = schwarzschildMetric(pos, param);
        float r = length(pos);
        float r_s = 2.0 * param;
        dmetric = vec2(
            -r_s / (r * r * pow(1.0 - r_s / r, 2)),
            2.0 * r
        );
    } else if (curvatureType < 3.5) {
        // Saddle
        metric = saddleMetric(pos, param);
        dmetric = vec2(
            2.0 * param * param * pos.x,
            2.0 * param * param * pos.y
        );
    } else {
        // Wormhole
        metric = wormholeMetric(pos, param);
        dmetric = vec2(
            2.0 * pos.x,
            2.0 * pos.y
        );
    }

    // Christoffel symbols (simplified 2D)
    vec2 gamma = dmetric / (2.0 * metric);

    // Update direction
    vec2 newDir = dir - gamma * dir * dir * dt;

    // Normalize
    float dirLen = length(newDir);
    if (dirLen > EPS) newDir = normalize(newDir);
```

```
    // Update position
    vec2 newPos = pos + newDir * dt;

    return vec4(newPos, newDir);
}

// Trace geodesic and compute color
vec3 traceGeodesic(vec2 startPos, vec2 startDir, float curvatureType, float param) {
    vec2 pos = startPos;
    vec2 dir = startDir;
    vec3 color = vec3(0.0);

    float totalDist = 0.0;
    float dt = 0.01;
    int steps = 100;

    for (int i = 0; i < 100; i++) {
        if (totalDist > 5.0) break;

        // Step geodesic
        vec4 step = geodesicStep(pos, dir, dt, curvatureType, param);
        pos = step.xy;
        dir = step.zw;
        totalDist += dt;

        // Sample environment at current position
        float angle = atan(pos.y, pos.x);
        float r = length(pos);

        vec3 envColor = vec3(
            0.5 + 0.5 * cos(angle * 2.0 + u_time * 0.3),
            0.5 + 0.5 * cos(angle * 3.0 + u_time * 0.5 + 1.0),
            0.5 + 0.5 * cos(angle * 5.0 + u_time * 0.7 + 2.0)
        );

        // Curvature-based coloring
        float curvatureColor;
        if (curvatureType < 0.5) {
            // Sphere: warm, converging
            curvatureColor = 1.0 - smoothstep(0.0, 1.0, r);
            envColor = mix(envColor, vec3(1.0, 0.6, 0.3), curvatureColor * 0.5);
        } else if (curvatureType < 1.5) {
            // Hyperbolic: cool, diverging
            curvatureColor = smoothstep(0.0, 1.0, r);
            envColor = mix(envColor, vec3(0.3, 0.6, 1.0), curvatureColor * 0.5);
        } else if (curvatureType < 2.5) {
            // Schwarzschild: redshift near horizon
            float r_s = 2.0 * param;
            float redshift = 1.0 / sqrt(max(1.0 - r_s / r, 0.01));
            envColor.r *= redshift;
```

```
            envColor.b /= redshift;
        } else if (curvatureType < 3.5) {
            // Saddle: chaotic, mixed
            curvatureColor = abs(sin(pos.x * 5.0) * cos(pos.y * 5.0));
            envColor = mix(envColor, vec3(0.5, 1.0, 0.5), curvatureColor * 0.3);
        } else {
            // Wormhole: tunnel effect
            float throat = param;
            float tunnel = exp(-pow(r - throat, 2.0) / (throat * 0.5));
            envColor = mix(envColor, vec3(0.8, 0.9, 1.0), tunnel * 0.5);
        }

        // Accumulate with distance falloff
        float falloff = exp(-totalDist * 0.5);
        color += envColor * falloff * dt;
    }

    return color;
}

// Chrome reflection with geodesic distortion
vec3 chromeGravity(vec2 uv, float curvatureType, float param) {
    vec2 p = uv * 2.0 - 1.0;
    p.x *= u_resolution.x / u_resolution.y;

    // Ray direction from view point
    vec2 dir = normalize(p - vec2(0.0, -0.5));
    vec2 startPos = vec2(0.0, -0.5);

    // Trace geodesic
    vec3 color = traceGeodesic(startPos, dir, curvatureType, param);

    // Chrome reflectivity
    color *= 0.9;

    // Specular highlight
    float highlight = pow(max(0.0, 1.0 - length(p)), 3.0);
    color += vec3(highlight * 0.3);

    return color;
}

void main() {
    vec2 uv = gl_FragCoord.xy / u_resolution.xy;
    vec2 p = uv * 2.0 - 1.0;
    p.x *= u_resolution.x / u_resolution.y;

    // Chrome reflection through curved space
    vec3 color = chromeGravity(uv, u_curvature_type, u_mass);

    // Vignette
```

```
    float vignette = 1.0 - smoothstep(0.5, 1.5, length(p));
    color *= vignette;

    // Tone mapping
    color = color / (1.0 + color * 0.5);

    gl_FragColor = vec4(color, 1.0);
}
```

— JS MODULE —

```
// System 52: Gravity Chrome — JS Module
// Metric tensors, geodesic integration, mass distribution

export class GravityChromeSystem {
    constructor() {
        this.curvatureType = 0; // 0=sphere, 1=hyperbolic, 2=schwarzschild, 3=saddle, 4=wormhole
        this.mass = 0.5;
        this.curvature = 1.0;
        this.lensCenter = {x: 0, y: 0};
        this.time = 0;

        // Mass distribution for custom metrics
        this.massPoints = []; // {x, y, mass}
    }

    // Metric tensor at point (x, y) for different curvature types
    metricTensor(x, y, type, param) {
        switch(type) {
            case 0: // Sphere
                const R = param;
                const theta = Math.atan2(y, x);
                return {
                    g_xx: R * R,
                    g_yy: R * R * Math.sin(theta) * Math.sin(theta),
                    g_xy: 0
                };

            case 1: // Hyperbolic (Poincaré disk)
                const r2 = x * x + y * y;
                const factor = 4 * param * param / ((1 - r2) * (1 - r2));
                return {
                    g_xx: factor,
                    g_yy: factor,
                    g_xy: 0
                };

            case 2: // Schwarzschild
                const r = Math.sqrt(x * x + y * y);
                const r_s = 2 * param;
                if (r < r_s + 0.001) {
```

```
                return {g_xx: 1e6, g_yy: 1e6, g_xy: 0};
            }
            return {
                g_xx: 1 / (1 - r_s / r),
                g_yy: r * r,
                g_xy: 0
            };

        case 3: // Saddle
            const K = param;
            return {
                g_xx: 1 + K * K * x * x,
                g_yy: 1 + K * K * y * y,
                g_xy: K * K * x * y
            };

        case 4: // Wormhole
            const throat = param;
            const r_w = Math.sqrt(x * x + y * y + throat * throat);
            return {
                g_xx: 1,
                g_yy: r_w * r_w,
                g_xy: 0
            };

        default:
            return {g_xx: 1, g_yy: 1, g_xy: 0};
    }
}

// Christoffel symbols (simplified for diagonal metrics)
christoffel(x, y, type, param) {
    const h = 0.001;
    const metric = this.metricTensor.bind(this);

    // Numerical derivatives
    const g = metric(x, y, type, param);
    const g_x = metric(x + h, y, type, param);
    const g_y = metric(x, y + h, type, param);

    const dg_xx_dx = (g_x.g_xx - g.g_xx) / h;
    const dg_yy_dx = (g_x.g_yy - g.g_yy) / h;
    const dg_xx_dy = (g_y.g_xx - g.g_xx) / h;
    const dg_yy_dy = (g_y.g_yy - g.g_yy) / h;

    // For diagonal metric: Γ^x_xx = g^xx * ∂x g_xx / 2
    const gInv_xx = 1 / g.g_xx;
    const gInv_yy = 1 / g.g_yy;

    return {
        Gx_xx: gInv_xx * dg_xx_dx / 2,
```

```
            Gx_yy: -gInv_xx * dg_yy_dx / 2,
            Gy_xx: -gInv_yy * dg_xx_dy / 2,
            Gy_yy: gInv_yy * dg_yy_dy / 2
        };
    }

    // 4th-order Runge-Kutta geodesic integration
    integrateGeodesic(startPos, startDir, steps, dt, type, param) {
        let pos = {...startPos};
        let dir = {...startDir};
        const path = [{...pos}];

        for (let i = 0; i < steps; i++) {
            const gamma = this.christoffel(pos.x, pos.y, type, param);

            // k1
            const ddir1 = {
                x: -gamma.Gx_xx * dir.x * dir.x - gamma.Gx_yy * dir.y * dir.y,
                y: -gamma.Gy_xx * dir.x * dir.x - gamma.Gy_yy * dir.y * dir.y
            };

            // k2
            const pos2 = {
                x: pos.x + dir.x * dt * 0.5,
                y: pos.y + dir.y * dt * 0.5
            };
            const dir2 = {
                x: dir.x + ddir1.x * dt * 0.5,
                y: dir.y + ddir1.y * dt * 0.5
            };
            const gamma2 = this.christoffel(pos2.x, pos2.y, type, param);
            const ddir2 = {
                x: -gamma2.Gx_xx * dir2.x * dir2.x - gamma2.Gx_yy * dir2.y * dir2.y,
                y: -gamma2.Gy_xx * dir2.x * dir2.x - gamma2.Gy_yy * dir2.y * dir2.y
            };

            // k3
            const pos3 = {
                x: pos.x + dir2.x * dt * 0.5,
                y: pos.y + dir2.y * dt * 0.5
            };
            const dir3 = {
                x: dir.x + ddir2.x * dt * 0.5,
                y: dir.y + ddir2.y * dt * 0.5
            };
            const gamma3 = this.christoffel(pos3.x, pos3.y, type, param);
            const ddir3 = {
                x: -gamma3.Gx_xx * dir3.x * dir3.x - gamma3.Gx_yy * dir3.y * dir3.y,
                y: -gamma3.Gy_xx * dir3.x * dir3.x - gamma3.Gy_yy * dir3.y * dir3.y
            };
```

```
        // k4
        const pos4 = {
            x: pos.x + dir3.x * dt,
            y: pos.y + dir3.y * dt
        };
        const dir4 = {
            x: dir.x + ddir3.x * dt,
            y: dir.y + ddir3.y * dt
        };
        const gamma4 = this.christoffel(pos4.x, pos4.y, type, param);
        const ddir4 = {
            x: -gamma4.Gx_xx * dir4.x * dir4.x - gamma4.Gx_yy * dir4.y * dir4.y,
            y: -gamma4.Gy_xx * dir4.x * dir4.x - gamma4.Gy_yy * dir4.y * dir4.y
        };

        // Update
        pos.x += (dir.x + 2 * dir2.x + 2 * dir3.x + dir4.x) * dt / 6;
        pos.y += (dir.y + 2 * dir2.y + 2 * dir3.y + dir4.y) * dt / 6;

        dir.x += (ddir1.x + 2 * ddir2.x + 2 * ddir3.x + ddir4.x) * dt / 6;
        dir.y += (ddir1.y + 2 * ddir2.y + 2 * ddir3.y + ddir4.y) * dt / 6;

        // Normalize direction
        const dirLen = Math.sqrt(dir.x * dir.x + dir.y * dir.y);
        if (dirLen > 0) {
            dir.x /= dirLen;
            dir.y /= dirLen;
        }

        path.push({...pos});
    }

    return path;
}

// Add mass point for custom gravity well
addMassPoint(x, y, mass) {
    this.massPoints.push({x, y, mass});
}

// Clear mass points
clearMassPoints() {
    this.massPoints = [];
}

// Compute custom metric from mass points (superposition)
customMetric(x, y) {
    let g_xx = 1, g_yy = 1, g_xy = 0;

    for (let mp of this.massPoints) {
        const dx = x - mp.x;
```

```
            const dy = y - mp.y;
            const r = Math.sqrt(dx * dx + dy * dy);
            const r_s = 2 * mp.mass;

            if (r > r_s) {
                const factor = 1 / (1 - r_s / r);
                g_xx += factor - 1;
                g_yy += r * r;
            }
        }

        return {g_xx, g_yy, g_xy};
    }

    // Schwarzschild redshift factor
    redshift(r, M) {
        const r_s = 2 * M;
        if (r <= r_s) return Infinity;
        return 1 / Math.sqrt(1 - r_s / r);
    }

    // Photon sphere radius (unstable circular orbit)
    photonSphereRadius(M) {
        return 3 * M;
    }

    // Get uniforms for shader
    getUniforms() {
        return {
            u_time: this.time,
            u_curvature_type: this.curvatureType,
            u_mass: this.mass,
            u_curvature: this.curvature,
            u_lens_center: [this.lensCenter.x, this.lensCenter.y]
        };
    }

    // Animate lens center
    animate(dt) {
        this.time += dt;
        this.lensCenter = {
            x: Math.sin(this.time * 0.3) * 0.2,
            y: Math.cos(this.time * 0.5) * 0.2
        };
    }

    setCurvatureType(type) {
        this.curvatureType = type;
    }

    setMass(mass) {
```

```
        this.mass = mass;
    }
}
```

==========

SYSTEM 53: MINIMAL SPARKLE

==========

— DOCUMENTATION —

System 53: Minimal Sparkle

Overview

Single hash function = infinite sparkle. `fract(sin(x*12.9898+y*78.233)*43758.5453)` — 80 characters, infinite output. fBm for natural variation. Domain warp for organic patterns. Three hashes = RGB. The code is tiny, the output is a universe.

Mathematical Foundation

The Hash: `fract(sin(dot(p, vec2(12.9898, 78.233))) * 43758.5453)`

Properties:

- Deterministic: same input → same output
- Pseudo-random: output appears uniform in [0,1]
- Cheap: one sin, one dot, one fract

fBm (fractional Brownian motion): Sum of noise octaves with decreasing amplitude and increasing frequency.

```
fBm(x) = Σ 2^(-i·H) · noise(2^i · x)
```

H = Hurst exponent ($0 < H < 1$). H = 0.5 = classic Brownian motion.

Domain Warping: Distort coordinates before sampling noise.

```
warp(x) = x + fBm(x)
color = fBm(warp(x))
```

Creates flowing, organic patterns from rigid noise.

GLSL Shader

See `shader.glsl` — single-line hash, fBm layering, domain warp, temporal crystal.

JS Module

See `module.js` — hash parameter exploration, fBm configuration, domain warp animation.

Showcase Builds

1. One Line Universe

Entire visual system in single GLSL line. The hash IS the universe. Every pixel determined by 80 characters. Emergent patterns from minimal rules.

2. Hash DNA

Different magic numbers = different species. Crossbreeding and mutation of hash parameters. The parameter space IS the genome. Evolution of visual forms.

3. Fractal Dust

fBm from hash creates terrain, clouds, infinite landscape. Each octave adds detail. The dust settles into mountains and valleys. Procedural world from one function.

4. Domain Warp Nebula

Recursive coordinate distortion creates flowing, living patterns. The warp field itself is warped. Self-referential beauty. The nebula breathes.

5. Temporal Crystal

Time-quantized hash creates stroboscopic beats, waves of sync. Discrete time steps create standing wave patterns. The crystal has temporal structure, not just spatial.

Implementation Notes

- Hash magic numbers: 12.9898, 78.233, 43758.5453 (classic values from IQ)
- fBm: 4-8 octaves typical, lacunarity = 2.0, gain = 0.5
- Domain warp: 1-2 levels of recursion for organic look
- Temporal: quantize time with `floor(time * fps) / fps` for stroboscopic effect

— GLSL SHADER —

```glsl
// System 53: Minimal Sparkle
// Single hash = infinite sparkle. 80 characters, one universe.

uniform float u_time;
uniform vec2 u_resolution;
uniform float u_mode;           // 0=one line, 1=hash dna, 2=fractal dust, 3=domain warp, 4=temporal
uniform vec2 u_hash_seed;       // magic numbers for hash DNA
uniform float u_octaves;        // fBm octaves
uniform float u_warp_amount;    // domain warp strength
uniform float u_time_quant;     // temporal quantization

#define PI 3.14159265359

// THE HASH — 80 characters, infinite output
float hash(vec2 p) {
    return fract(sin(dot(p, vec2(12.9898, 78.233))) * 43758.5453);
}
```

```
// Custom hash with DNA parameters
float hashDNA(vec2 p, vec2 seed, float multiplier) {
    return fract(sin(dot(p, seed)) * multiplier);
}

// Value noise (smooth hash)
float noise(vec2 p) {
    vec2 i = floor(p);
    vec2 f = fract(p);
    f = f * f * (3.0 - 2.0 * f); // smoothstep

    float a = hash(i);
    float b = hash(i + vec2(1.0, 0.0));
    float c = hash(i + vec2(0.0, 1.0));
    float d = hash(i + vec2(1.0, 1.0));

    return mix(mix(a, b, f.x), mix(c, d, f.x), f.y);
}

// fBm (fractional Brownian motion)
float fbm(vec2 p, float octaves) {
    float value = 0.0;
    float amplitude = 0.5;
    float frequency = 1.0;

    for (float i = 0.0; i < 10.0; i++) {
        if (i >= octaves) break;
        value += amplitude * noise(p * frequency);
        amplitude *= 0.5;
        frequency *= 2.0;
    }

    return value;
}

// Domain warp: distort coordinates before sampling
vec2 domainWarp(vec2 p, float amount, float time) {
    vec2 q = vec2(
        fbm(p + vec2(0.0, 0.0), 4.0),
        fbm(p + vec2(5.2, 1.3), 4.0)
    );

    vec2 r = vec2(
        fbm(p + amount * q + vec2(1.7, 9.2) + time * 0.15, 4.0),
        fbm(p + amount * q + vec2(8.3, 2.8) + time * 0.126, 4.0)
    );

    return p + amount * r;
}
```

```
// Temporal crystal: quantized time
float temporalCrystal(float time, float quant) {
    return floor(time * quant) / quant;
}

// Sparkle from hash
float sparkle(vec2 uv, float time, vec2 seed) {
    float h = hashDNA(uv * 100.0, seed, 43758.5453);
    float pulse = sin(time * 5.0 + h * 10.0) * 0.5 + 0.5;
    return h * pulse;
}

void main() {
    vec2 uv = gl_FragCoord.xy / u_resolution.xy;
    vec2 p = uv * 2.0 - 1.0;
    p.x *= u_resolution.x / u_resolution.y;

    vec3 color = vec3(0.0);

    if (u_mode < 0.5) {
        // One Line Universe
        float t = u_time;
        vec2 q = p * 3.0;

        // Single hash line = entire visual
        float h = hash(q + t);
        float h2 = hash(q * 2.0 - t * 0.5);
        float h3 = hash(q * 0.5 + t * 0.3);

        color = vec3(h, h2, h3);

        // Add sparkle
        float s = sparkle(q, t, vec2(12.9898, 78.233));
        color += vec3(s * 0.5);

    } else if (u_mode < 1.5) {
        // Hash DNA
        vec2 seed = u_hash_seed;
        float mult = 43758.5453;

        // Three species from different hash parameters
        float species1 = hashDNA(p * 5.0 + u_time * 0.1, seed, mult);
        float species2 = hashDNA(p * 5.0 + u_time * 0.15, seed * 1.618, mult * 1.414);
        float species3 = hashDNA(p * 5.0 + u_time * 0.2, seed * 2.718, mult * 2.236);

        color = vec3(species1, species2, species3);

        // Crossbreed in center
        float blend = smoothstep(0.5, 0.0, length(p));
        color = mix(color, (color + vec3(species1 * species2, species2 * species3, species3 * species1))
        ↪  * 0.5, blend);
```

```
} else if (u_mode < 2.5) {
    // Fractal Dust
    float octaves = u_octaves;
    float terrain = fbm(p * 3.0 + u_time * 0.05, octaves);
    float clouds = fbm(p * 2.0 + vec2(u_time * 0.1, 0.0), octaves * 0.7);
    float detail = fbm(p * 10.0, octaves * 0.5);

    // Terrain coloring
    vec3 ground = mix(
        vec3(0.2, 0.1, 0.05), // low
        vec3(0.4, 0.6, 0.2),  // high
        terrain
    );

    // Clouds
    vec3 sky = mix(
        vec3(0.1, 0.2, 0.4),
        vec3(0.8, 0.9, 1.0),
        clouds
    );

    // Combine
    float height = terrain + clouds * 0.3;
    color = mix(ground, sky, smoothstep(0.3, 0.7, height));

    // Sparkle dust
    float dust = hash(floor(p * 50.0) + u_time * 0.01);
    color += vec3(dust * 0.1);

} else if (u_mode < 3.5) {
    // Domain Warp Nebula
    float warp = u_warp_amount;
    vec2 warped = domainWarp(p * 2.0, warp, u_time);

    float nebula = fbm(warped, 5.0);
    float nebula2 = fbm(warped + vec2(5.0), 5.0);
    float nebula3 = fbm(warped * 0.5, 3.0);

    // Nebula colors
    vec3 core = vec3(0.8, 0.3, 0.1);
    vec3 mid = vec3(0.2, 0.5, 0.8);
    vec3 edge = vec3(0.1, 0.1, 0.3);

    color = mix(edge, mid, nebula);
    color = mix(color, core, nebula2 * nebula3);

    // Stars
    float stars = hash(floor(warped * 30.0));
    stars = step(0.97, stars);
    color += vec3(stars);
```

```
    } else {
        // Temporal Crystal
        float quant = u_time_quant;
        float t = temporalCrystal(u_time, quant);

        // Quantized movement creates standing waves
        vec2 q = p * 5.0;
        float wave1 = sin(q.x * 3.0 + t * 10.0) * sin(q.y * 2.0 + t * 8.0);
        float wave2 = sin(length(q) * 5.0 - t * 15.0);
        float wave3 = sin(q.x * q.y + t * 5.0);

        // Hash adds sparkle to waves
        float h = hash(floor(q + t * 5.0));

        color = vec3(
            wave1 * 0.5 + 0.5,
            wave2 * 0.5 + 0.5,
            wave3 * 0.5 + 0.5
        ) * (0.7 + h * 0.3);

        // Stroboscopic flash
        float flash = step(0.8, sin(t * quant * PI));
        color += vec3(flash * 0.2);
    }

    // Tone mapping
    color = color / (1.0 + color * 0.3);

    // Vignette
    float vignette = 1.0 - smoothstep(0.5, 1.5, length(p));
    color *= vignette;

    gl_FragColor = vec4(color, 1.0);
}
```

— JS MODULE —

```
// System 53: Minimal Sparkle — JS Module
// Hash exploration, fBm configuration, domain warp animation

export class MinimalSparkleSystem {
    constructor() {
        this.mode = 0;
        this.time = 0;

        // Hash DNA parameters
        this.hashSeed = {x: 12.9898, y: 78.233};
        this.hashMultiplier = 43758.5453;

        // fBm parameters
```

```
        this.octaves = 6;
        this.lacunarity = 2.0;
        this.gain = 0.5;
        this.hurst = 0.5;

        // Domain warp
        this.warpAmount = 1.0;
        this.warpOctaves = 4;

        // Temporal
        this.timeQuant = 10.0; // fps for quantization

        // DNA population for evolution
        this.population = [];
    }

    // THE HASH — 80 characters, infinite output
    hash(x, y, seedX = 12.9898, seedY = 78.233, multiplier = 43758.5453) {
        const dot = x * seedX + y * seedY;
        return ((Math.sin(dot) * multiplier) % 1 + 1) % 1;
    }

    // Value noise (smooth hash)
    noise(x, y) {
        const ix = Math.floor(x);
        const iy = Math.floor(y);
        const fx = x - ix;
        const fy = y - iy;

        // Smoothstep
        const sx = fx * fx * (3 - 2 * fx);
        const sy = fy * fy * (3 - 2 * fy);

        const a = this.hash(ix, iy);
        const b = this.hash(ix + 1, iy);
        const c = this.hash(ix, iy + 1);
        const d = this.hash(ix + 1, iy + 1);

        return this.mix(this.mix(a, b, sx), this.mix(c, d, sx), sy);
    }

    mix(a, b, t) {
        return a + (b - a) * t;
    }

    // fBm (fractional Brownian motion)
    fbm(x, y, octaves = this.octaves) {
        let value = 0;
        let amplitude = 0.5;
        let frequency = 1;
```

```
        for (let i = 0; i < octaves; i++) {
            value += amplitude * this.noise(x * frequency, y * frequency);
            amplitude *= this.gain;
            frequency *= this.lacunarity;
        }

        return value;
    }

    // Domain warp
    domainWarp(x, y, amount = this.warpAmount, time = 0) {
        const qx = this.fbm(x + 0, y + 0, this.warpOctaves);
        const qy = this.fbm(x + 5.2, y + 1.3, this.warpOctaves);

       const rx = this.fbm(x + amount * qx + 1.7, y + amount * qy + 9.2 + time * 0.15, this.warpOctaves);
        const ry = this.fbm(x + amount * qx + 8.3, y + amount * qy + 2.8 + time * 0.126,
        ↪ this.warpOctaves);

        return {x: x + amount * rx, y: y + amount * ry};
    }

    // Temporal crystal: quantized time
    temporalCrystal(time, quant = this.timeQuant) {
        return Math.floor(time * quant) / quant;
    }

    // Generate DNA population (for evolution)
    generatePopulation(size = 10) {
        this.population = [];
        for (let i = 0; i < size; i++) {
            this.population.push({
                seedX: 10 + Math.random() * 10,
                seedY: 70 + Math.random() * 15,
                multiplier: 40000 + Math.random() * 10000,
                octaves: 3 + Math.floor(Math.random() * 6),
                gain: 0.3 + Math.random() * 0.4,
                fitness: 0
            });
        }
    }

    // Crossbreed two DNA specimens
    crossbreed(a, b) {
        return {
            seedX: Math.random() < 0.5 ? a.seedX : b.seedX,
            seedY: Math.random() < 0.5 ? a.seedY : b.seedY,
            multiplier: Math.random() < 0.5 ? a.multiplier : b.multiplier,
            octaves: Math.random() < 0.5 ? a.octaves : b.octaves,
            gain: Math.random() < 0.5 ? a.gain : b.gain,
            fitness: 0
        };
```

```
}

// Mutate DNA
mutate(specimen, rate = 0.1) {
    return {
        seedX: specimen.seedX + (Math.random() - 0.5) * rate * 10,
        seedY: specimen.seedY + (Math.random() - 0.5) * rate * 15,
        multiplier: specimen.multiplier + (Math.random() - 0.5) * rate * 10000,
        octaves: Math.max(1, specimen.octaves + Math.floor((Math.random() - 0.5) * rate * 4)),
        gain: Math.max(0.1, Math.min(0.9, specimen.gain + (Math.random() - 0.5) * rate)),
        fitness: 0
    };
}

// Evaluate fitness (visual interest metric)
evaluateFitness(specimen, samples = 100) {
    let variance = 0;
    let mean = 0;

    for (let i = 0; i < samples; i++) {
        const x = Math.random() * 10;
        const y = Math.random() * 10;
        const h = this.hash(x, y, specimen.seedX, specimen.seedY, specimen.multiplier);
        mean += h;
        variance += h * h;
    }

    mean /= samples;
    variance = variance / samples - mean * mean;

    // Fitness = variance (we want interesting, not flat)
    // Penalize too uniform or too chaotic
    const targetVariance = 0.08;
    specimen.fitness = 1 / (1 + Math.abs(variance - targetVariance) * 10);

    return specimen.fitness;
}

// Evolve one generation
evolve() {
    // Evaluate fitness
    for (let s of this.population) {
        this.evaluateFitness(s);
    }

    // Sort by fitness
    this.population.sort((a, b) => b.fitness - a.fitness);

    // Keep top half, breed new half
    const newPopulation = this.population.slice(0, Math.floor(this.population.length / 2));
```

```
        while (newPopulation.length < this.population.length) {
            const parentA = this.population[Math.floor(Math.random() * newPopulation.length)];
            const parentB = this.population[Math.floor(Math.random() * newPopulation.length)];
            let child = this.crossbreed(parentA, parentB);
            child = this.mutate(child, 0.1);
            newPopulation.push(child);
        }

        this.population = newPopulation;

        // Return best
        return this.population[0];
    }

    // Get uniforms for shader
    getUniforms() {
        return {
            u_time: this.time,
            u_mode: this.mode,
            u_hash_seed: [this.hashSeed.x, this.hashSeed.y],
            u_octaves: this.octaves,
            u_warp_amount: this.warpAmount,
            u_time_quant: this.timeQuant
        };
    }

    // Animate
    update(dt) {
        this.time += dt;
    }

    setMode(mode) {
        this.mode = mode;
    }

    // Set hash parameters (for DNA exploration)
    setHashParams(seedX, seedY, multiplier) {
        this.hashSeed = {x: seedX, y: seedY};
        this.hashMultiplier = multiplier;
    }

    // Randomize hash (discover new species)
    randomizeHash() {
        this.hashSeed = {
            x: 10 + Math.random() * 20,
            y: 60 + Math.random() * 40
        };
        this.hashMultiplier = 30000 + Math.random() * 30000;
    }
}
```

==========

SYSTEM 54: FOURIER SHIMMER

==========

— DOCUMENTATION —

System 54: Fourier Shimmer

Overview

Every sparkle is a frequency bin. The FFT of the visual field IS the image. Time-domain sparkles = inverse-transformed spectrum. You see the music, literally — each frequency component is a point of light with amplitude = brightness, phase = hue.

Mathematical Foundation

Discrete Fourier Transform: `X[k] = Σ x[n] · e^(-2πikn/N)`

Inverse DFT: `x[n] = (1/N) Σ X[k] · e^(2πikn/N)`

Key Insight: The sparkle field IS the time-domain signal. The spectrum (frequency bins) controls the pattern. Low frequencies = large-scale structure. High frequencies = fine detail/sparkle.

Phase Information: `arg(X[k])` determines spatial position of frequency component. Phase coherence = structured pattern. Random phase = noise.

GLSL Shader

See `shader.glsl` — real-time FFT approximation, frequency-bin sparkle placement, phase-color mapping.

JS Module

See `module.js` — Web Audio API integration, FFT computation, frequency bin → sparkle mapping, beat detection.

Showcase Builds

1. Audio Prism

Real-time audio FFT drives sparkle field. Bass = large warm sparkles. Treble = small cool sparkles. The music IS the visual. Every note = a birth event.

2. Harmonic Lattice

Frequencies arranged as harmonic series lattice. Integer ratios = consonance = connected sparkles. Dissonance = chaotic, unconnected. Musical theory made spatial.

3. Phase Sculpture

Interactive phase manipulation. Drag to rotate phase of frequency bins. The image morphs as phase shifts. You sculpt with phase, not geometry.

4. Spectral Freeze

Capture FFT snapshot, hold it. The frozen spectrum becomes a static sparkle constellation. Release = morph back to live audio. Memory of a sound.

5. Convolution Bloom

Sparkle field convolved with impulse response. The reverb of a space applied to light. Cathedral = long trailing sparkles. Bathroom = short, bright bursts.

Implementation Notes

- Use Web Audio API AnalyserNode for real-time FFT
- Approximate inverse-DFT in shader with sum of sinusoids
- Phase coherence crucial for structured patterns
- Beat detection: sudden energy increase in low-frequency bins

— GLSL SHADER —

```
// System 54: Fourier Shimmer
// FFT frequency bins = sparkle field. The spectrum IS the image.

uniform float u_time;
uniform vec2 u_resolution;
uniform float u_mode;           // 0=audio, 1=harmonic, 2=phase, 3=freeze, 4=convolution
uniform sampler2D u_spectrum;   // FFT magnitude data (RGBA = freq bins)
uniform sampler2D u_phase;      // FFT phase data
uniform float u_beat;           // beat detection intensity
uniform float u_spectrum_size;  // number of frequency bins

#define PI 3.14159265359
#define TAU 6.28318530718
#define MAX_BINS 128

// Approximate inverse-DFT: sum of sinusoids
// x[n] = (1/N) Σ X[k]·e^(2πikn/N)
vec3 inverseDFT(vec2 uv, sampler2D spectrum, sampler2D phase, float numBins) {
    vec3 color = vec3(0.0);
    float N = numBins;

    for (float k = 0.0; k < float(MAX_BINS); k++) {
        if (k >= N) break;

        float u = (k + 0.5) / N;
        float mag = texture2D(spectrum, vec2(u, 0.5)).r;
        float ph = texture2D(phase, vec2(u, 0.5)).r * TAU;

        // Frequency as spatial wave
        float freq = k + 1.0;
        float wave = sin(uv.x * freq * TAU + ph) * sin(uv.y * freq * TAU + ph);

        // Amplitude = brightness, frequency = hue
```

```
        vec3 waveColor = vec3(
            sin(freq * 0.1) * 0.5 + 0.5,
            sin(freq * 0.1 + 2.0) * 0.5 + 0.5,
            sin(freq * 0.1 + 4.0) * 0.5 + 0.5
        );

        color += waveColor * mag * wave * 0.5;
    }

    return color / N;
}

// Beat-driven sparkle burst
float beatSparkle(vec2 uv, float beat, float time) {
    float burst = exp(-beat * 3.0);
    float ring = sin(length(uv) * 20.0 - time * 10.0) * 0.5 + 0.5;
    return burst * ring * beat;
}

// Harmonic lattice: integer ratio frequencies
vec3 harmonicLattice(vec2 uv, float time) {
    vec3 color = vec3(0.0);

    // Just intonation ratios
    float ratios[12];
    ratios[0] = 1.0;    ratios[1] = 1.059;  ratios[2] = 1.122;
    ratios[3] = 1.189;  ratios[4] = 1.260;  ratios[5] = 1.335;
    ratios[6] = 1.414;  ratios[7] = 1.498;  ratios[8] = 1.587;
    ratios[9] = 1.682;  ratios[10] = 1.782; ratios[11] = 1.888;

    for (int i = 0; i < 12; i++) {
        float freq = ratios[i] * 5.0;
        float wave = sin(uv.x * freq + time * ratios[i]) *
                     sin(uv.y * freq + time * ratios[i] * 1.618);

        float consonance = 1.0 - abs(fract(ratios[i]) - 0.5) * 2.0;

        vec3 noteColor = vec3(
            sin(float(i) * 0.5) * 0.5 + 0.5,
            sin(float(i) * 0.5 + 2.0) * 0.5 + 0.5,
            sin(float(i) * 0.5 + 4.0) * 0.5 + 0.5
        );

        color += noteColor * wave * consonance * 0.1;
    }

    return color;
}

// Phase sculpture: interactive phase rotation
vec3 phaseSculpture(vec2 uv, float time, float phaseShift) {
```

```
    vec3 color = vec3(0.0);

    for (float k = 1.0; k < 20.0; k++) {
        float ph = phaseShift * k + time * 0.5;
        float wave = sin(uv.x * k * TAU + ph) * cos(uv.y * k * TAU + ph * 1.618);

        float hue = fract(ph / TAU);
        vec3 waveColor = vec3(
            sin(hue * TAU) * 0.5 + 0.5,
            sin(hue * TAU + 2.0) * 0.5 + 0.5,
            sin(hue * TAU + 4.0) * 0.5 + 0.5
        );

        color += waveColor * wave * (1.0 / k) * 0.3;
    }

    return color;
}

// Convolution bloom: reverb applied to light
vec3 convolutionBloom(vec2 uv, float time) {
    vec3 color = vec3(0.0);

    for (int i = 0; i < 20; i++) {
        float t = float(i) * 0.05;
        float decay = exp(-t * 3.0);
        vec2 offset = vec2(
            sin(t * 5.0 + time) * t * 0.1,
            cos(t * 3.0 + time) * t * 0.1
        );

        float sparkle = sin((uv + offset).x * 10.0) * sin((uv + offset).y * 10.0);
        sparkle = pow(abs(sparkle), 3.0);

        color += vec3(0.5 + decay * 0.5, 0.3 + decay * 0.3, decay) * sparkle * decay * 0.1;
    }

    return color;
}

void main() {
    vec2 uv = gl_FragCoord.xy / u_resolution.xy;
    vec2 p = uv * 2.0 - 1.0;
    p.x *= u_resolution.x / u_resolution.y;

    vec3 color = vec3(0.05, 0.05, 0.1);

    if (u_mode < 0.5) {
        color += inverseDFT(p, u_spectrum, u_phase, u_spectrum_size);
        color += vec3(beatSparkle(p, u_beat, u_time));
    } else if (u_mode < 1.5) {
```

```
        color += harmonicLattice(p, u_time);
    } else if (u_mode < 2.5) {
        float phaseShift = sin(u_time * 0.5) * PI;
        color += phaseSculpture(p, u_time, phaseShift);
    } else if (u_mode < 3.5) {
        float freezeTime = floor(u_time * 0.5) * 2.0;
        float morph = fract(u_time * 0.5);
        vec3 frozen = inverseDFT(p, u_spectrum, u_phase, u_spectrum_size);
        vec3 live = harmonicLattice(p, u_time);
        color += mix(frozen, live, smoothstep(0.7, 1.0, morph));
    } else {
        color += convolutionBloom(p, u_time);
    }

    color = color / (1.0 + color * 0.3);

    float vignette = 1.0 - smoothstep(0.5, 1.5, length(p));
    color *= vignette;

    gl_FragColor = vec4(color, 1.0);
}
```

— JS MODULE —

```
// System 54: Fourier Shimmer — JS Module
// Web Audio FFT, frequency bin mapping, beat detection

export class FourierShimmerSystem {
    constructor(audioContext = null) {
        this.audioContext = audioContext;
        this.analyser = null;
        this.fftSize = 256;
        this.spectrumData = new Uint8Array(this.fftSize);
        this.phaseData = new Float32Array(this.fftSize);
        this.beatIntensity = 0;
        this.mode = 0;
        this.time = 0;
        this.useSyntheticAudio = false;

        this.energyHistory = [];
        this.historySize = 43;

        this.phaseShift = 0;
        this.frozenSpectrum = null;
        this.isFrozen = false;
    }

    async initAudio() {
        if (!this.audioContext) {
            this.audioContext = new (window.AudioContext || window.webkitAudioContext)();
        }
```

```
        try {
            const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
            const source = this.audioContext.createMediaStreamSource(stream);
            this.analyser = this.audioContext.createAnalyser();
            this.analyser.fftSize = this.fftSize;
            source.connect(this.analyser);
        } catch (e) {
            console.log('Audio input failed, using synthetic');
            this.useSyntheticAudio = true;
        }
    }

    generateSyntheticSpectrum(time) {
        for (let i = 0; i < this.fftSize; i++) {
            const freq = i / this.fftSize;
            const kick = Math.exp(-Math.pow((freq - 0.05) / 0.02, 2)) * (Math.sin(time * 10) > 0.8 ? 1 :
             ↪  0);
            const snare = Math.exp(-Math.pow((freq - 0.3) / 0.1, 2)) * (Math.sin(time * 10 + 3) > 0.9 ?
             ↪  0.7 : 0);
            const hihat = Math.exp(-Math.pow((freq - 0.8) / 0.2, 2)) * (Math.sin(time * 20) > 0.95 ? 0.5
             ↪  : 0);
            const harmonics = Math.sin(freq * 100) * 0.3 * Math.exp(-freq * 2);

            this.spectrumData[i] = (kick + snare + hihat + harmonics) * 255;
            this.phaseData[i] = Math.sin(freq * 50 + time) * Math.PI;
        }
    }

    detectBeat() {
        let energy = 0;
        const bassBins = Math.floor(this.fftSize * 0.1);
        for (let i = 0; i < bassBins; i++) {
            energy += this.spectrumData[i] / 255;
        }
        energy /= bassBins;

        this.energyHistory.push(energy);
        if (this.energyHistory.length > this.historySize) {
            this.energyHistory.shift();
        }

        const avgEnergy = this.energyHistory.reduce((a, b) => a + b, 0) / this.energyHistory.length;

        if (energy > avgEnergy * 1.3 && energy > 0.3) {
            this.beatIntensity = 1.0;
        }
        this.beatIntensity *= 0.9;
    }

    updateAudio() {
        if (this.useSyntheticAudio) {
```

```
            this.generateSyntheticSpectrum(this.time);
        } else if (this.analyser) {
            this.analyser.getByteFrequencyData(this.spectrumData);
            const timeData = new Uint8Array(this.fftSize);
            this.analyser.getByteTimeDomainData(timeData);
            for (let i = 0; i < this.fftSize; i++) {
                this.phaseData[i] = (timeData[i] - 128) / 128 * Math.PI;
            }
        }
        this.detectBeat();
    }

    freeze() {
        this.frozenSpectrum = new Uint8Array(this.spectrumData);
        this.isFrozen = true;
    }

    unfreeze() {
        this.isFrozen = false;
    }

    setPhaseShift(shift) {
        this.phaseShift = shift;
    }

    getSpectrumData() {
        return this.isFrozen ? this.frozenSpectrum : this.spectrumData;
    }

    getPhaseData() {
        return this.phaseData;
    }

    getUniforms() {
        return {
            u_time: this.time,
            u_mode: this.mode,
            u_beat: this.beatIntensity,
            u_spectrum_size: this.fftSize / 2,
            u_phase: this.phaseShift
        };
    }

    update(dt) {
        this.time += dt;
        this.updateAudio();
    }

    setMode(mode) {
        this.mode = mode;
    }
```

```
}
```

==========

SYSTEM 55: CELLULAR AUTOMATA CHROME

==========

— DOCUMENTATION —

System 55: Cellular Automata Chrome

Overview

Chrome that evolves. Each pixel is a cell in a continuous-state automaton. Conway's Game of Life but for light — birth, survival, death rules applied to brightness values. The chrome surface is alive, breathing, dying, being reborn. Emergent patterns = emergent beauty.

Mathematical Foundation

Continuous Cellular Automata: Cell state [0,1] (brightness), not binary.

Update Rule: `state' = f(state, neighbor_average)`

Lenia (Chan, 2019): Generalized continuous CA with bell-shaped growth function.

```
g(U) = 2·exp(-(U-μ)²/(2σ²)) - 1
state' = clamp(state + dt · g(U), 0, 1)
```

SmoothLife: Continuous space + continuous time CA. Patterns include gliders, oscillators, self-replicators.

GLSL Shader

See `shader.glsl` — ping-pong framebuffer CA update, bell-shaped growth, chrome reflection on living surface.

JS Module

See `module.js` — CA parameter space exploration, rule breeding, pattern seeding, chrome state management.

Showcase Builds

1. Living Chrome

Standard Lenia on chrome surface. The reflection evolves. You see yourself in a mirror that is alive, changing, forgetting and remembering.

2. Glider Constellation

Seed glider patterns. Each glider is a moving sparkle. Collisions create new patterns. The chrome sky is full of migrating light creatures.

3. Mitosis Bloom

Self-replicator rules. Sparkles split when they reach critical mass. Population explosion = bloom event. Then death from overcrowding. Cycle repeats.

4. Rule Breeding

Two CA rules breed offspring. Visual characteristics mix. The chrome inherits traits from both parents. Evolution of shine.

5. Frozen Fire

Cooling CA — high temperature = chaos, low = crystalline order. The chrome passes through phase transitions. Fire that freezes into snowflakes.

Implementation Notes

- Use ping-pong framebuffers for CA state
- Gaussian kernel for neighbor averaging
- Bell-shaped growth function for smooth dynamics
- Chrome reflection applied to CA state as height map

— GLSL SHADER —

```glsl
// System 55: Cellular Automata Chrome
// Living chrome — continuous CA (Lenia-style) on reflective surface

uniform float u_time;
uniform vec2 u_resolution;
uniform sampler2D u_prevState;  // Previous frame CA state
uniform float u_mode;           // 0=living chrome, 1=gliders, 2=mitosis, 3=breeding, 4=frozen fire
uniform float u_mu;             // growth center
uniform float u_sigma;          // growth width
uniform float u_dt;             // time step

#define PI 3.14159265359

// Gaussian kernel for neighbor averaging
float gaussianKernel(vec2 offset, float radius) {
    float r = length(offset);
    return exp(-r * r / (2.0 * radius * radius));
}

// Bell-shaped growth function (Lenia)
float growth(float u, float mu, float sigma) {
    return 2.0 * exp(-pow(u - mu, 2.0) / (2.0 * sigma * sigma)) - 1.0;
}

// Sample neighbor average with kernel
float neighborAverage(sampler2D state, vec2 uv, vec2 resolution, float radius) {
    float sum = 0.0;
    float weight = 0.0;
```

```
    // Sample in 3x3 neighborhood
    for (float x = -1.0; x <= 1.0; x += 1.0) {
        for (float y = -1.0; y <= 1.0; y += 1.0) {
            vec2 offset = vec2(x, y) / resolution * radius;
            float w = gaussianKernel(vec2(x, y), 1.0);
            sum += texture2D(state, uv + offset).r * w;
            weight += w;
        }
    }

    return sum / weight;
}

// Chrome reflection from CA state
vec3 chromeFromState(float state, vec2 uv, float time) {
    // State = height map for chrome
    float height = state;

    // Normal from height gradient
    vec2 grad = vec2(
        dFdx(height),
        dFdy(height)
    );
    vec3 normal = normalize(vec3(-grad, 1.0));

    // Environment reflection
    vec3 envColor = vec3(
        0.5 + 0.5 * sin(uv.x * 3.0 + time * 0.3),
        0.5 + 0.5 * sin(uv.y * 2.0 + time * 0.5 + 1.0),
        0.5 + 0.5 * sin((uv.x + uv.y) * 4.0 + time * 0.7 + 2.0)
    );

    // Fresnel
    float fresnel = pow(1.0 - abs(normal.z), 3.0);

    // State-based color: low = cool, high = warm
    vec3 stateColor = mix(
        vec3(0.1, 0.2, 0.5),  // dead: deep blue
        vec3(1.0, 0.8, 0.3),  // alive: warm gold
        height
    );

    return mix(stateColor, envColor, fresnel * 0.5);
}

// Seed glider pattern
float gliderSeed(vec2 uv, vec2 center) {
    vec2 d = (uv - center) * u_resolution;
    float pattern = 0.0;
```

```
    // Glider pattern (5 cells)
    pattern += smoothstep(2.0, 0.0, length(d - vec2(0.0, 0.0)));
    pattern += smoothstep(2.0, 0.0, length(d - vec2(1.0, 0.0)));
    pattern += smoothstep(2.0, 0.0, length(d - vec2(2.0, 0.0)));
    pattern += smoothstep(2.0, 0.0, length(d - vec2(2.0, 1.0)));
    pattern += smoothstep(2.0, 0.0, length(d - vec2(1.0, 2.0)));

    return clamp(pattern, 0.0, 1.0);
}

// Mitosis: split when mass > threshold
float mitosisRule(float state, float neighbors, float time) {
    float growthRate = growth(neighbors, 0.35, 0.15);
    float newState = state + u_dt * growthRate;

    // Splitting: if too big, create two smaller
    if (state > 0.8 && neighbors < 0.3) {
        newState = 0.4; // Split into half
    }

    return clamp(newState, 0.0, 1.0);
}

// Frozen fire: temperature-based phase transition
float frozenFire(float state, float neighbors, float time) {
    float temp = 0.5 + 0.3 * sin(time * 0.5); // oscillating temperature

    float mu = mix(0.2, 0.5, temp); // high temp = chaotic, low = ordered
    float sigma = mix(0.05, 0.2, temp);

    float growthRate = growth(neighbors, mu, sigma);
    return clamp(state + u_dt * growthRate, 0.0, 1.0);
}

void main() {
    vec2 uv = gl_FragCoord.xy / u_resolution.xy;
    vec2 p = uv * 2.0 - 1.0;
    p.x *= u_resolution.x / u_resolution.y;

    float state = texture2D(u_prevState, uv).r;
    float neighbors = neighborAverage(u_prevState, uv, u_resolution, 3.0);

    if (u_mode < 0.5) {
        // Living Chrome
        float growthRate = growth(neighbors, u_mu, u_sigma);
        state = clamp(state + u_dt * growthRate, 0.0, 1.0);

    } else if (u_mode < 1.5) {
        // Glider Constellation
        float growthRate = growth(neighbors, 0.35, 0.15);
        state = clamp(state + u_dt * growthRate, 0.0, 1.0);
```

```
        // Seed new gliders periodically
        if (fract(u_time * 0.3) < 0.01) {
            vec2 seedPos = vec2(
                sin(u_time * 0.7) * 0.5 + 0.5,
                cos(u_time * 0.5) * 0.5 + 0.5
            );
            state = max(state, gliderSeed(uv, seedPos));
        }

    } else if (u_mode < 2.5) {
        // Mitosis Bloom
        state = mitosisRule(state, neighbors, u_time);

    } else if (u_mode < 3.5) {
        // Rule Breeding: mix two rules
        float rule1 = growth(neighbors, 0.35, 0.15);
        float rule2 = growth(neighbors, 0.25, 0.1);
        float mixFactor = 0.5 + 0.5 * sin(u_time * 0.2);
        float growthRate = mix(rule1, rule2, mixFactor);
        state = clamp(state + u_dt * growthRate, 0.0, 1.0);

    } else {
        // Frozen Fire
        state = frozenFire(state, neighbors, u_time);
    }

    // Chrome reflection
    vec3 color = chromeFromState(state, uv, u_time);

    // Output state to framebuffer for next frame
    gl_FragColor = vec4(color, state);
}
```

— JS MODULE —

```
// System 55: Cellular Automata Chrome — JS Module
// Lenia CA, ping-pong buffers, rule breeding, pattern seeding

export class CellularAutomataChromeSystem {
    constructor(width, height) {
        this.width = width;
        this.height = height;
        this.mode = 0;
        this.time = 0;

        // CA parameters
        this.mu = 0.35;
        this.sigma = 0.15;
        this.dt = 0.1;
```

```
        // Framebuffers for ping-pong
        this.framebuffers = [];
        this.currentBuffer = 0;

        // Rules for breeding
        this.rules = [
            {mu: 0.35, sigma: 0.15, name: 'standard'},
            {mu: 0.25, sigma: 0.10, name: 'crystal'},
            {mu: 0.45, sigma: 0.20, name: 'chaos'}
        ];

        // Pattern library
        this.patterns = {
            glider: [
                [0,1,0],
                [0,0,1],
                [1,1,1]
            ],
            oscillator: [
                [0,1,0],
                [0,1,0],
                [0,1,0]
            ],
            pulsar: [
                [0,0,1,1,1,0,0,0,1,1,1,0,0],
                [0,0,0,0,0,0,0,0,0,0,0,0,0],
                [1,0,0,0,0,1,0,1,0,0,0,0,1],
                [1,0,0,0,0,1,0,1,0,0,0,0,1],
                [1,0,0,0,0,1,0,1,0,0,0,0,1],
                [0,0,1,1,1,0,0,0,1,1,1,0,0]
            ]
        };

        this.initFramebuffers();
    }

    initFramebuffers() {
        // Create two framebuffers for ping-pong
        // In practice, you'd use WebGL framebuffers
        // Here we store state as Float32Arrays
        this.states = [
            new Float32Array(this.width * this.height),
            new Float32Array(this.width * this.height)
        ];

        // Random initial state
        this.randomizeState();
    }

    randomizeState() {
        const state = this.states[this.currentBuffer];
```

```
        for (let i = 0; i < state.length; i++) {
            state[i] = Math.random() > 0.8 ? Math.random() : 0;
        }
    }

    // Seed a pattern at position
    seedPattern(patternName, x, y) {
        const pattern = this.patterns[patternName];
        if (!pattern) return;

        const state = this.states[this.currentBuffer];
        const h = pattern.length;
        const w = pattern[0].length;

        for (let py = 0; py < h; py++) {
            for (let px = 0; px < w; px++) {
                const sx = Math.floor(x + px - w/2);
                const sy = Math.floor(y + py - h/2);
                if (sx >= 0 && sx < this.width && sy >= 0 && sy < this.height) {
                    state[sy * this.width + sx] = pattern[py][px];
                }
            }
        }
    }

    // Get neighbor average with Gaussian kernel
    getNeighborAverage(state, x, y, radius = 3) {
        let sum = 0;
        let weight = 0;

        for (let dy = -radius; dy <= radius; dy++) {
            for (let dx = -radius; dx <= radius; dx++) {
                const nx = (x + dx + this.width) % this.width;
                const ny = (y + dy + this.height) % this.height;
                const r2 = dx * dx + dy * dy;
                const w = Math.exp(-r2 / (2 * radius * radius));

                sum += state[ny * this.width + nx] * w;
                weight += w;
            }
        }

        return sum / weight;
    }

    // Bell-shaped growth function
    growth(u, mu, sigma) {
        return 2 * Math.exp(-Math.pow(u - mu, 2) / (2 * sigma * sigma)) - 1;
    }

    // Update CA state
```

```
    update() {
        const current = this.states[this.currentBuffer];
        const next = this.states[1 - this.currentBuffer];

        for (let y = 0; y < this.height; y++) {
            for (let x = 0; x < this.width; x++) {
                const idx = y * this.width + x;
                const state = current[idx];
                const neighbors = this.getNeighborAverage(current, x, y, 3);

                let newState;

                if (this.mode === 2) {
                    // Mitosis
                    const growthRate = this.growth(neighbors, 0.35, 0.15);
                    newState = state + this.dt * growthRate;
                    if (state > 0.8 && neighbors < 0.3) {
                        newState = 0.4;
                    }
                } else if (this.mode === 3) {
                    // Breeding: mix rules
                    const rule1 = this.growth(neighbors, this.rules[0].mu, this.rules[0].sigma);
                    const rule2 = this.growth(neighbors, this.rules[1].mu, this.rules[1].sigma);
                    const mixFactor = 0.5 + 0.5 * Math.sin(this.time * 0.2);
                    newState = state + this.dt * (rule1 * mixFactor + rule2 * (1 - mixFactor));
                } else if (this.mode === 4) {
                    // Frozen fire
                    const temp = 0.5 + 0.3 * Math.sin(this.time * 0.5);
                    const mu = 0.2 + 0.3 * temp;
                    const sigma = 0.05 + 0.15 * temp;
                    const growthRate = this.growth(neighbors, mu, sigma);
                    newState = state + this.dt * growthRate;
                } else {
                    // Standard / Glider
                    const growthRate = this.growth(neighbors, this.mu, this.sigma);
                    newState = state + this.dt * growthRate;
                }

                next[idx] = Math.max(0, Math.min(1, newState));
            }
        }

        // Swap buffers
        this.currentBuffer = 1 - this.currentBuffer;
    }

    // Auto-seed gliders
    autoSeedGliders() {
        if (Math.random() < 0.02) {
            const x = Math.floor(Math.random() * this.width);
            const y = Math.floor(Math.random() * this.height);
```

```
            this.seedPattern('glider', x, y);
        }
    }

    // Get current state as texture data
    getStateData() {
        return this.states[this.currentBuffer];
    }

    // Get uniforms for shader
    getUniforms() {
        return {
            u_time: this.time,
            u_mode: this.mode,
            u_mu: this.mu,
            u_sigma: this.sigma,
            u_dt: this.dt
        };
    }

    // Main update
    update(dt) {
        this.time += dt;
        this.update();

        if (this.mode === 1) {
            this.autoSeedGliders();
        }
    }

    setMode(mode) {
        this.mode = mode;
    }

    setParams(mu, sigma, dt) {
        this.mu = mu;
        this.sigma = sigma;
        this.dt = dt;
    }

    // Breed new rule from two parents
    breedRules(parent1, parent2) {
        return {
            mu: (parent1.mu + parent2.mu) / 2 + (Math.random() - 0.5) * 0.05,
            sigma: (parent1.sigma + parent2.sigma) / 2 + (Math.random() - 0.5) * 0.03,
            name: `breed_${Math.floor(Math.random() * 1000)}`
        };
    }
}
```

==========

SYSTEM 56: REACTION-DIFFUSION GLITTER

==========

— DOCUMENTATION —

System 56: Reaction-Diffusion Glitter

Overview

Turing patterns made of light. Two chemicals (activator/inhibitor) react and diffuse on the chrome surface. Spots, stripes, labyrinths, chaos — all emergent from simple local rules. The chrome is a chemical skin that patterns itself.

Mathematical Foundation

Gray-Scott Model:

```
∂U/∂t = D_u ∇²U - UV² + F(1-U)
∂V/∂t = D_v ∇²V + UV² - (F+k)V
```

Turing Instability: Diffusion-driven instability. The inhibitor diffuses faster than the activator, creating stable patterns from homogeneous equilibrium.

Pattern Zoo (Pearson, 1993): For different (F, k) parameters:

- : spots, amoeba, holes
- : stripes, worms, spirals
- : chaos, Turing turbulence
- : solitons, pulsating
- : waves, spirals
- : spatiotemporal chaos
- : Worms, U-skate world
- : U-skate, long worms
- : self-replicating spots
- : pulsating spots
- : worms, maze
- : coral, fingerprint
- : spots, moving
- : U-skate, Liesegang rings
- : spots, moving, worms
- : waves, target patterns
- : fission, spots

- : solitons
- : long worms
- : U-skate
- : self-replicating
- : chaos
- : spots, worms
- : waves

GLSL Shader

See `shader.glsl` — Gray-Scott reaction-diffusion on ping-pong buffers, pattern parameter mapping, chrome reflection from chemical concentrations.

JS Module

See `module.js` — parameter space exploration, pattern classification, feed/kill rate animation, seed injection.

Showcase Builds

1. Coral Chrome

Coral-like growth patterns on chrome surface. The metal develops organic texture. F = 0.0545, k = 0.062. Fingerprint of the sea on metal.

2. Leopard Spots

Self-replicating spots that move, split, die. Like leopard skin but alive. F = 0.035, k = 0.065. Each spot is a sparkle that breeds.

3. Maze Labyrinth

Maze-like patterns that grow and dissolve. The chrome is a labyrinth that solves itself. F = 0.029, k = 0.057. Ariadne's thread made of light.

4. Turing Waves

Traveling waves and target patterns. The chrome ripples with chemical waves. F = 0.018, k = 0.050. Heartbeat of the metal.

5. Parameter Morph

Smoothly interpolate between pattern types. The chrome transforms from spots to stripes to chaos. A metamorphosis of mathematical species.

Implementation Notes

- Use ping-pong framebuffers for U and V fields
- Laplacian: 3x3 kernel [0.05, 0.2, 0.05, 0.2, -1, 0.2, 0.05, 0.2, 0.05]

- Time step: dt = 1.0 (simulation time)
- Grid size: 256x256 minimum for interesting patterns

— GLSL SHADER —

```glsl
// System 56: Reaction-Diffusion Glitter
// Gray-Scott model on chrome — Turing patterns made of light

uniform float u_time;
uniform vec2 u_resolution;
uniform sampler2D u_prevU;      // Previous U field (activator)
uniform sampler2D u_prevV;      // Previous V field (inhibitor)
uniform float u_mode;           // 0=coral, 1=leopard, 2=maze, 3=waves, 4=morph
uniform float u_feed;           // Feed rate F
uniform float u_kill;           // Kill rate k
uniform float u_du;             // Diffusion rate U
uniform float u_dv;             // Diffusion rate V

#define PI 3.14159265359

// Laplacian with 3x3 kernel
vec2 laplacian(sampler2D texU, sampler2D texV, vec2 uv, vec2 resolution) {
    vec2 sum = vec2(0.0);

    // Center weight: -1.0
    sum += texture2D(texU, uv).rg * -1.0;

    // Neighbors: 0.2 (cardinal), 0.05 (diagonal)
    vec2 offset = 1.0 / resolution;

    sum += texture2D(texU, uv + vec2(offset.x, 0.0)).rg * 0.2;
    sum += texture2D(texU, uv - vec2(offset.x, 0.0)).rg * 0.2;
    sum += texture2D(texU, uv + vec2(0.0, offset.y)).rg * 0.2;
    sum += texture2D(texU, uv - vec2(0.0, offset.y)).rg * 0.2;

    sum += texture2D(texU, uv + offset).rg * 0.05;
    sum += texture2D(texU, uv - offset).rg * 0.05;
    sum += texture2D(texU, uv + vec2(offset.x, -offset.y)).rg * 0.05;
    sum += texture2D(texU, uv + vec2(-offset.x, offset.y)).rg * 0.05;

    return sum;
}

// Chrome reflection from chemical concentrations
vec3 chromeFromChemicals(float u, float v, vec2 uv, float time) {
    // U = activator (high = pattern), V = inhibitor (high = suppresses)

    // Pattern height map
    float height = u * (1.0 - v * 0.5);

    // Normal from height
```

```
    vec2 grad = vec2(dFdx(height), dFdy(height));
    vec3 normal = normalize(vec3(-grad * 2.0, 1.0));

    // Environment
    vec3 envColor = vec3(
        0.5 + 0.5 * sin(uv.x * 2.0 + time * 0.2),
        0.5 + 0.5 * sin(uv.y * 3.0 + time * 0.3 + 1.0),
        0.5 + 0.5 * sin((uv.x + uv.y) * 4.0 + time * 0.4 + 2.0)
    );

    // Chemical-based color
    vec3 uColor = mix(
        vec3(0.1, 0.05, 0.2),  // low U: deep purple
        vec3(1.0, 0.9, 0.4),   // high U: warm gold
        u
    );

    vec3 vColor = mix(
        vec3(0.0, 0.0, 0.0),   // low V: nothing
        vec3(0.2, 0.6, 1.0),  // high V: cool blue
        v
    );

    vec3 patternColor = mix(uColor, vColor, v * 0.3);

    // Fresnel
    float fresnel = pow(1.0 - abs(normal.z), 2.0);

    return mix(patternColor, envColor, fresnel * 0.4);
}

// Parameter presets for pattern zoo
vec2 getPatternParams(float mode, float time) {
    if (mode < 0.5) {
        // Coral: F = 0.0545, k = 0.062
        return vec2(0.0545, 0.062);
    } else if (mode < 1.5) {
        // Leopard spots: F = 0.035, k = 0.065
        return vec2(0.035, 0.065);
    } else if (mode < 2.5) {
        // Maze: F = 0.029, k = 0.057
        return vec2(0.029, 0.057);
    } else if (mode < 3.5) {
        // Waves: F = 0.018, k = 0.050
        return vec2(0.018, 0.050);
    } else {
        // Morph: interpolate through parameter space
        float t = fract(time * 0.1);
        vec2 coral = vec2(0.0545, 0.062);
        vec2 leopard = vec2(0.035, 0.065);
        vec2 maze = vec2(0.029, 0.057);
```

```
        vec2 waves = vec2(0.018, 0.050);

        if (t < 0.25) return mix(coral, leopard, t * 4.0);
        else if (t < 0.5) return mix(leopard, maze, (t - 0.25) * 4.0);
        else if (t < 0.75) return mix(maze, waves, (t - 0.5) * 4.0);
        else return mix(waves, coral, (t - 0.75) * 4.0);
    }
}

void main() {
    vec2 uv = gl_FragCoord.xy / u_resolution.xy;
    vec2 p = uv * 2.0 - 1.0;
    p.x *= u_resolution.x / u_resolution.y;

    // Get current chemical concentrations
    vec2 chemicals = texture2D(u_prevU, uv).rg;
    float u = chemicals.r;
    float v = chemicals.g;

    // Get parameters
    vec2 params = getPatternParams(u_mode, u_time);
    float F = params.x;
    float k = params.y;
    float Du = u_du;
    float Dv = u_dv;

    // Compute Laplacian
    vec2 lapl = laplacian(u_prevU, u_prevV, uv, u_resolution);

    // Gray-Scott reaction
    float reaction = u * v * v;

    float newU = u + (Du * lapl.x - reaction + F * (1.0 - u)) * 1.0;
    float newV = v + (Dv * lapl.y + reaction - (F + k) * v) * 1.0;

    newU = clamp(newU, 0.0, 1.0);
    newV = clamp(newV, 0.0, 1.0);

    // Chrome reflection
    vec3 color = chromeFromChemicals(newU, newV, uv, u_time);

    // Output chemicals to framebuffer for next frame
    // Store U in R, V in G
    gl_FragColor = vec4(color, newU);
}
```

— JS MODULE —

```
// System 56: Reaction-Diffusion Glitter — JS Module
// Gray-Scott model, parameter space, pattern seeding
```

```
export class ReactionDiffusionGlitterSystem {
    constructor(width, height) {
        this.width = width;
        this.height = height;
        this.mode = 0;
        this.time = 0;

        // Diffusion rates
        this.Du = 0.16;
        this.Dv = 0.08;

        // Current feed/kill rates
        this.F = 0.035;
        this.k = 0.065;

        // Chemical fields
        this.U = new Float32Array(width * height).fill(1.0);
        this.V = new Float32Array(width * height).fill(0.0);

        // Pattern presets
        this.patterns = {
            coral: {F: 0.0545, k: 0.062, Du: 0.16, Dv: 0.08},
            leopard: {F: 0.035, k: 0.065, Du: 0.16, Dv: 0.08},
            maze: {F: 0.029, k: 0.057, Du: 0.16, Dv: 0.08},
            waves: {F: 0.018, k: 0.050, Du: 0.16, Dv: 0.08},
            chaos: {F: 0.026, k: 0.051, Du: 0.16, Dv: 0.08},
            solitons: {F: 0.030, k: 0.062, Du: 0.16, Dv: 0.08}
        };

        this.initSeed();
    }

    // Seed initial pattern
    initSeed() {
        // Small square of V in center
        const cx = Math.floor(this.width / 2);
        const cy = Math.floor(this.height / 2);
        const size = 10;

        for (let y = cy - size; y < cy + size; y++) {
            for (let x = cx - size; x < cx + size; x++) {
                if (x >= 0 && x < this.width && y >= 0 && y < this.height) {
                    this.V[y * this.width + x] = 0.5 + Math.random() * 0.5;
                    this.U[y * this.width + x] = 0.5;
                }
            }
        }

        // Add noise
        for (let i = 0; i < this.U.length; i++) {
            this.U[i] += (Math.random() - 0.5) * 0.01;
```

```
            this.V[i] += (Math.random() - 0.5) * 0.01;
            this.U[i] = Math.max(0, Math.min(1, this.U[i]));
            this.V[i] = Math.max(0, Math.min(1, this.V[i]));
        }
    }

    // Inject V at position (mouse/touch)
    injectV(x, y, radius = 5, amount = 0.5) {
        for (let dy = -radius; dy <= radius; dy++) {
            for (let dx = -radius; dx <= radius; dx++) {
                const px = x + dx;
                const py = y + dy;
                if (px >= 0 && px < this.width && py >= 0 && py < this.height) {
                    const dist = Math.sqrt(dx * dx + dy * dy);
                    if (dist <= radius) {
                        const idx = py * this.width + px;
                        this.V[idx] = Math.min(1, this.V[idx] + amount * (1 - dist / radius));
                        this.U[idx] = Math.max(0, this.U[idx] - amount * 0.5 * (1 - dist / radius));
                    }
                }
            }
        }
    }

    // Laplacian with 3x3 kernel
    laplacian(field, x, y) {
        const w = this.width;
        const h = this.height;

        const center = field[y * w + x];
        const n = field[((y - 1 + h) % h) * w + x];
        const s = field[((y + 1) % h) * w + x];
        const e = field[y * w + ((x + 1) % w)];
        const w2 = field[y * w + ((x - 1 + w) % w)];
        const ne = field[((y - 1 + h) % h) * w + ((x + 1) % w)];
        const nw = field[((y - 1 + h) % h) * w + ((x - 1 + w) % w)];
        const se = field[((y + 1) % h) * w + ((x + 1) % w)];
        const sw = field[((y + 1) % h) * w + ((x - 1 + w) % w)];

        return (n + s + e + w2) * 0.2 + (ne + nw + se + sw) * 0.05 - center;
    }

    // Update Gray-Scott
    update() {
        const newU = new Float32Array(this.U.length);
        const newV = new Float32Array(this.V.length);

        for (let y = 0; y < this.height; y++) {
            for (let x = 0; x < this.width; x++) {
                const idx = y * this.width + x;
                const u = this.U[idx];
```

```
            const v = this.V[idx];

            const lapU = this.laplacian(this.U, x, y);
            const lapV = this.laplacian(this.V, x, y);

            const reaction = u * v * v;

            newU[idx] = u + (this.Du * lapU - reaction + this.F * (1.0 - u));
            newV[idx] = v + (this.Dv * lapV + reaction - (this.F + this.k) * v);

            newU[idx] = Math.max(0, Math.min(1, newU[idx]));
            newV[idx] = Math.max(0, Math.min(1, newV[idx]));
        }
    }

    this.U = newU;
    this.V = newV;
}

// Get parameters for current mode
getModeParams() {
    const modes = ['coral', 'leopard', 'maze', 'waves', 'chaos'];
    const modeName = modes[this.mode] || 'leopard';

    if (this.mode === 4) {
        // Morph mode: interpolate
        const t = (this.time * 0.1) % 1;
        const keys = Object.keys(this.patterns);
        const idx = Math.floor(t * keys.length) % keys.length;
        const nextIdx = (idx + 1) % keys.length;
        const localT = (t * keys.length) % 1;

        const p1 = this.patterns[keys[idx]];
        const p2 = this.patterns[keys[nextIdx]];

        return {
            F: p1.F + (p2.F - p1.F) * localT,
            k: p1.k + (p2.k - p1.k) * localT,
            Du: p1.Du,
            Dv: p1.Dv
        };
    }

    return this.patterns[modeName];
}

// Apply mode parameters
applyModeParams() {
    const params = this.getModeParams();
    this.F = params.F;
    this.k = params.k;
```

```
        this.Du = params.Du;
        this.Dv = params.Dv;
    }

    // Get chemical data as texture
    getChemicalData() {
        // Pack U and V into RGBA
        const data = new Float32Array(this.width * this.height * 4);
        for (let i = 0; i < this.U.length; i++) {
            data[i * 4] = this.U[i];
            data[i * 4 + 1] = this.V[i];
            data[i * 4 + 2] = 0;
            data[i * 4 + 3] = 1;
        }
        return data;
    }

    // Get uniforms for shader
    getUniforms() {
        return {
            u_time: this.time,
            u_mode: this.mode,
            u_feed: this.F,
            u_kill: this.k,
            u_du: this.Du,
            u_dv: this.Dv
        };
    }

    // Main update
    update(dt) {
        this.time += dt;
        this.applyModeParams();
        this.update();
    }

    setMode(mode) {
        this.mode = mode;
        // Reset with new seed for dramatic pattern change
        if (mode !== 4) {
            this.initSeed();
        }
    }

    setParams(F, k, Du = 0.16, Dv = 0.08) {
        this.F = F;
        this.k = k;
        this.Du = Du;
        this.Dv = Dv;
    }
```

```
    // Classify current pattern (simple heuristic)
    classifyPattern() {
        // Compute statistics
        let uMean = 0, vMean = 0;
        let uVar = 0, vVar = 0;

        for (let i = 0; i < this.U.length; i++) {
            uMean += this.U[i];
            vMean += this.V[i];
        }
        uMean /= this.U.length;
        vMean /= this.V.length;

        for (let i = 0; i < this.U.length; i++) {
            uVar += Math.pow(this.U[i] - uMean, 2);
            vVar += Math.pow(this.V[i] - vMean, 2);
        }
        uVar /= this.U.length;
        vVar /= this.V.length;

        // Simple classification
        if (uVar < 0.01) return 'uniform';
        if (uVar < 0.05) return 'waves';
        if (uVar < 0.1) return 'spots';
        return 'chaos';
    }
}
```

==========

SYSTEM 57: STRANGE ATTRACTOR SPARKLE

==========

— DOCUMENTATION —

System 57: Strange Attractor Sparkle

Overview

Chaotic systems with structure. Lorenz, Rössler, Aizawa — classic strange attractors rendered as sparkle trails. The deterministic chaos creates organic, never-repeating patterns. Each point is a moment in time, the trail is memory. The attractor is a ghost made of light.

Mathematical Foundation

Lorenz Attractor (1963):

```
dx/dt = σ(y - x)
dy/dt = x(ρ - z) - y
```

```
dz/dt = xy - βz
```

$= 10, \quad = 28, \quad = 8/3$

Rössler Attractor (1976):

```
dx/dt = -y - z
dy/dt = x + ay
dz/dt = b + z(x - c)
```

a = 0.2, b = 0.2, c = 5.7

Aizawa Attractor:

```
dx/dt = (z - b)x - dy
dy/dt = (z - b)y + dx
dz/dt = c + az - z³/3 - (x² + y²)(1 + ez) + fz x³
```

Strange Attractor Properties:

- Deterministic: no randomness
- Aperiodic: never repeats
- Bounded: stays in finite region
- Sensitive: tiny changes → huge divergence
- Fractal dimension: non-integer

GLSL Shader

See `shader.glsl` — precomputed attractor point cloud, trail rendering, parameter animation.

JS Module

See `module.js` — ODE integration (Runge-Kutta), attractor point generation, trail management, parameter morphing.

Showcase Builds

1. Lorenz Butterfly

The iconic butterfly wings as sparkle trails. Two lobes, never the same path twice. The wings flap but the butterfly is made of light. =10, =28, =8/3.

2. Rössler Ribbon

Single-lobed attractor with a "fold". The ribbon twists and never closes. Like a Möbius strip in time. a=0.2, b=0.2, c=5.7.

3. Aizawa Sphere

Spherical strange attractor. Points orbit a center but never settle. The sphere breathes. Parameters create different "planets".

4. Parameter Morph

Smoothly change attractor parameters. The shape morphs between different strange attractors. One ghost becomes another.

5. Attractor Orchestra

Multiple attractors, different scales, different colors. The ensemble performs chaotic music. Each attractor is an instrument.

Implementation Notes

- Precompute 10,000+ points per attractor
- Use 4th-order Runge-Kutta for integration
- Trail rendering: point sprites with motion blur
- Parameter morphing: interpolate between attractor parameter sets
- Color from velocity magnitude or position

— GLSL SHADER —

```
// System 57: Strange Attractor Sparkle
// Chaotic systems rendered as sparkle trails — deterministic ghosts

uniform float u_time;
uniform vec2 u_resolution;
uniform float u_mode;           // 0=lorenz, 1=rossler, 2=aizawa, 3=morph, 4=orchestra
uniform sampler2D u_attractor_points; // Precomputed points (x,y,z,velocity)
uniform float u_point_count;
uniform float u_trail_length;   // how many previous points to show
uniform float u_point_size;

#define PI 3.14159265359
#define MAX_POINTS 5000

// Hash for variation
float hash(float n) {
    return fract(sin(n * 12.9898) * 43758.5453);
}

// Project 3D point to 2D with rotation
vec2 project3D(vec3 p, float time) {
    float rotX = time * 0.1;
    float rotY = time * 0.15;

    // Rotate around Y
    float x1 = p.x * cos(rotY) - p.z * sin(rotY);
    float z1 = p.x * sin(rotY) + p.z * cos(rotY);

    // Rotate around X
    float y2 = p.y * cos(rotX) - z1 * sin(rotX);
    float z2 = p.y * sin(rotX) + z1 * cos(rotX);
```

```
    // Perspective
    float perspective = 1.0 / (1.0 + z2 * 0.1);
    return vec2(x1, y2) * perspective;
}

// Color from velocity
vec3 velocityColor(float vel) {
    float hue = fract(vel * 0.5 + 0.3);
    vec3 c;
    float h = hue * 6.0;
    float i = floor(h);
    float f = h - i;
    float q = 1.0 - f;

    if (i == 0.0) c = vec3(1.0, f, 0.0);
    else if (i == 1.0) c = vec3(q, 1.0, 0.0);
    else if (i == 2.0) c = vec3(0.0, 1.0, f);
    else if (i == 3.0) c = vec3(0.0, q, 1.0);
    else if (i == 4.0) c = vec3(f, 0.0, 1.0);
    else c = vec3(1.0, 0.0, q);

    return c;
}

// Position-based color (for Aizawa sphere)
vec3 positionColor(vec3 p) {
    float r = length(p);
    float hue = fract(atan(p.y, p.x) / (2.0 * PI) + 0.5);
    float sat = smoothstep(0.0, 2.0, r);
    float val = 0.5 + 0.5 * sin(p.z * 2.0);

    vec3 c;
    float h = hue * 6.0;
    float i = floor(h);
    float f = h - i;
    float q = 1.0 - f;

    if (i == 0.0) c = vec3(1.0, f, 0.0);
    else if (i == 1.0) c = vec3(q, 1.0, 0.0);
    else if (i == 2.0) c = vec3(0.0, 1.0, f);
    else if (i == 3.0) c = vec3(0.0, q, 1.0);
    else if (i == 4.0) c = vec3(f, 0.0, 1.0);
    else c = vec3(1.0, 0.0, q);

    return c * sat * val;
}

// Trail fade
float trailFade(float pointIndex, float currentIndex, float trailLength) {
    float dist = abs(currentIndex - pointIndex);
    if (dist > trailLength) return 0.0;
```

```
    return 1.0 - dist / trailLength;
}

void main() {
    vec2 uv = gl_FragCoord.xy / u_resolution.xy;
    vec2 p = uv * 2.0 - 1.0;
    p.x *= u_resolution.x / u_resolution.y;

    vec3 color = vec3(0.02, 0.02, 0.05);

    float pointCount = min(u_point_count, float(MAX_POINTS));
    float currentIndex = mod(u_time * 50.0, pointCount);

    if (u_mode < 3.5) {
        // Single attractor modes
        for (float i = 0.0; i < float(MAX_POINTS); i++) {
            if (i >= pointCount) break;

            float u = (i + 0.5) / pointCount;
            vec4 pointData = texture2D(u_attractor_points, vec2(u, 0.5));
            vec3 pos = pointData.xyz;
            float vel = pointData.w;

            vec2 proj = project3D(pos, u_time);
            float d = length(p - proj);

            // Trail effect
            float trail = trailFade(i, currentIndex, u_trail_length);
            float size = u_point_size * (1.0 + vel * 0.5);
            float glow = exp(-d * d / (size * size)) * trail;

            // Color
            vec3 pointColor;
            if (u_mode < 0.5 || u_mode > 2.5) {
                // Lorenz / Morph: velocity color
                pointColor = velocityColor(vel);
            } else if (u_mode < 1.5) {
                // Rössler: warm ribbon
                pointColor = mix(vec3(1.0, 0.3, 0.1), vec3(0.9, 0.8, 0.2), vel * 0.5);
            } else {
                // Aizawa: position color
                pointColor = positionColor(pos);
            }

            color += pointColor * glow * 0.5;
        }

    } else {
        // Orchestra: multiple attractors
        for (float a = 0.0; a < 5.0; a++) {
            float offset = a * 1000.0;
```

```
            float scale = 0.3 + a * 0.15;
            vec2 center = vec2(
                cos(a * 1.256) * 0.6,
                sin(a * 1.256) * 0.6
            );

            for (float i = 0.0; i < 1000.0; i++) {
                float idx = offset + i;
                if (idx >= pointCount) break;

                float u = (idx + 0.5) / pointCount;
                vec4 pointData = texture2D(u_attractor_points, vec2(u, 0.5));
                vec3 pos = pointData.xyz * scale;
                float vel = pointData.w;

                vec2 proj = project3D(pos, u_time + a) + center;
                float d = length(p - proj);

                float trail = trailFade(idx, currentIndex, u_trail_length * 0.5);
                float size = u_point_size * scale;
                float glow = exp(-d * d / (size * size)) * trail;

                vec3 attractorColor = vec3(
                    sin(a * 1.2) * 0.5 + 0.5,
                    sin(a * 1.2 + 2.0) * 0.5 + 0.5,
                    sin(a * 1.2 + 4.0) * 0.5 + 0.5
                );

                color += attractorColor * glow * 0.3;
            }
        }
    }

    // Tone mapping
    color = color / (1.0 + color * 0.5);

    // Vignette
    float vignette = 1.0 - smoothstep(0.5, 1.5, length(p));
    color *= vignette;

    gl_FragColor = vec4(color, 1.0);
}
```

— JS MODULE —

```
// System 57: Strange Attractor Sparkle — JS Module
// ODE integration, attractor generation, trail management

export class StrangeAttractorSparkleSystem {
    constructor(numPoints = 10000) {
        this.numPoints = numPoints;
```

```
        this.mode = 0;
        this.time = 0;

        // Attractor parameters
        this.lorenzParams = {sigma: 10, rho: 28, beta: 8/3};
        this.rosslerParams = {a: 0.2, b: 0.2, c: 5.7};
        this.aizawaParams = {a: 0.95, b: 0.7, c: 0.6, d: 3.5, e: 0.25, f: 0.1};

        // Current parameters (for morphing)
        this.currentParams = {...this.lorenzParams};
        this.targetParams = {...this.lorenzParams};

        // Point data: [x, y, z, velocity] for each point
        this.points = new Float32Array(numPoints * 4);

        // Integration
        this.dt = 0.01;
        this.currentState = {x: 0.1, y: 0, z: 0};

        this.generatePoints();
    }

    // Lorenz ODE
    lorenzDeriv(state, params) {
        const {sigma, rho, beta} = params;
        return {
            x: sigma * (state.y - state.x),
            y: state.x * (rho - state.z) - state.y,
            z: state.x * state.y - beta * state.z
        };
    }

    // Rössler ODE
    rosslerDeriv(state, params) {
        const {a, b, c} = params;
        return {
            x: -state.y - state.z,
            y: state.x + a * state.y,
            z: b + state.z * (state.x - c)
        };
    }

    // Aizawa ODE
    aizawaDeriv(state, params) {
        const {a, b, c, d, e, f} = params;
        const {x, y, z} = state;
        const r = Math.sqrt(x*x + y*y);
        return {
            x: (z - b) * x - d * y,
            y: (z - b) * y + d * x,
            z: c + a * z - z*z*z/3 - r*r * (1 + e * z) + f * z * x*x*x
```

```
        };
    }

    // 4th-order Runge-Kutta step
    rk4Step(state, derivFn, params, dt) {
        const k1 = derivFn(state, params);
        const s2 = {
            x: state.x + k1.x * dt * 0.5,
            y: state.y + k1.y * dt * 0.5,
            z: state.z + k1.z * dt * 0.5
        };
        const k2 = derivFn(s2, params);
        const s3 = {
            x: state.x + k2.x * dt * 0.5,
            y: state.y + k2.y * dt * 0.5,
            z: state.z + k2.z * dt * 0.5
        };
        const k3 = derivFn(s3, params);
        const s4 = {
            x: state.x + k3.x * dt,
            y: state.y + k3.y * dt,
            z: state.z + k3.z * dt
        };
        const k4 = derivFn(s4, params);

        return {
            x: state.x + (k1.x + 2*k2.x + 2*k3.x + k4.x) * dt / 6,
            y: state.y + (k1.y + 2*k2.y + 2*k3.y + k4.y) * dt / 6,
            z: state.z + (k1.z + 2*k2.z + 2*k3.z + k4.z) * dt / 6
        };
    }

    // Generate attractor points
    generatePoints() {
        let state = {x: 0.1, y: 0, z: 0};

        // Burn-in: let attractor settle
        for (let i = 0; i < 1000; i++) {
            const derivFn = this.getDerivFn();
            state = this.rk4Step(state, derivFn, this.currentParams, this.dt);
        }

        // Generate points
        for (let i = 0; i < this.numPoints; i++) {
            const derivFn = this.getDerivFn();
            const prevState = {...state};
            state = this.rk4Step(state, derivFn, this.currentParams, this.dt);

            // Compute velocity
            const vel = Math.sqrt(
                Math.pow(state.x - prevState.x, 2) +
```

```
            Math.pow(state.y - prevState.y, 2) +
            Math.pow(state.z - prevState.z, 2)
        ) / this.dt;

        // Normalize for storage
        const idx = i * 4;
        this.points[idx] = state.x * 0.03;     // scale down
        this.points[idx + 1] = state.y * 0.03;
        this.points[idx + 2] = state.z * 0.03;
        this.points[idx + 3] = Math.min(vel * 0.1, 1.0); // normalized velocity
    }

    this.currentState = state;
}

// Get derivative function for current mode
getDerivFn() {
    if (this.mode === 0 || this.mode === 3) {
        return this.lorenzDeriv.bind(this);
    } else if (this.mode === 1) {
        return this.rosslerDeriv.bind(this);
    } else {
        return this.aizawaDeriv.bind(this);
    }
}

// Get current parameters
getCurrentParams() {
    if (this.mode === 0 || this.mode === 3) return this.currentParams;
    if (this.mode === 1) return this.rosslerParams;
    return this.aizawaParams;
}

// Morph parameters toward target
morphParams(dt) {
    const speed = 0.5 * dt;
    const keys = Object.keys(this.currentParams);
    for (let key of keys) {
        if (this.targetParams[key] !== undefined) {
            this.currentParams[key] += (this.targetParams[key] - this.currentParams[key]) * speed;
        }
    }
}

// Set target parameters for morphing
setTargetParams(params) {
    this.targetParams = {...params};
}

// Get point data for shader
getPointData() {
```

```
        return this.points;
    }

    // Get uniforms for shader
    getUniforms() {
        return {
            u_time: this.time,
            u_mode: this.mode,
            u_point_count: this.numPoints,
            u_trail_length: 500,
            u_point_size: 0.005
        };
    }

    // Main update
    update(dt) {
        this.time += dt;

        if (this.mode === 3) {
            // Morph mode
            this.morphParams(dt);
            // Regenerate points periodically
            if (Math.floor(this.time * 10) % 50 === 0) {
                this.generatePoints();
            }
        }
    }

    setMode(mode) {
        this.mode = mode;

        // Reset parameters for new mode
        if (mode === 0) {
            this.currentParams = {...this.lorenzParams};
        } else if (mode === 1) {
            this.currentParams = {...this.rosslerParams};
        } else if (mode === 2) {
            this.currentParams = {...this.aizawaParams};
        }

        this.generatePoints();
    }

    // Randomize parameters (discover new attractors)
    randomizeParams() {
        this.currentParams = {
            sigma: 5 + Math.random() * 15,
            rho: 10 + Math.random() * 30,
            beta: 2 + Math.random() * 3
        };
        this.generatePoints();
```

```
    }

    // Compute Lyapunov exponent (measure of chaos)
    computeLyapunov(iterations = 1000) {
        let state = {x: 0.1, y: 0, z: 0};
        let perturbed = {x: 0.10001, y: 0, z: 0};
        let sum = 0;

        const derivFn = this.getDerivFn();
        const params = this.getCurrentParams();

        for (let i = 0; i < iterations; i++) {
            state = this.rk4Step(state, derivFn, params, this.dt);
            perturbed = this.rk4Step(perturbed, derivFn, params, this.dt);

            const dist = Math.sqrt(
                Math.pow(state.x - perturbed.x, 2) +
                Math.pow(state.y - perturbed.y, 2) +
                Math.pow(state.z - perturbed.z, 2)
            );

            if (dist > 0) {
                sum += Math.log(dist / 0.00001);
            }

            // Renormalize
            const scale = 0.00001 / dist;
            perturbed = {
                x: state.x + (perturbed.x - state.x) * scale,
                y: state.y + (perturbed.y - state.y) * scale,
                z: state.z + (perturbed.z - state.z) * scale
            };
        }

        return sum / (iterations * this.dt);
    }
}
```

==========

SYSTEM 58: QUANTUM INTERFERENCE GLITTER

==========

— DOCUMENTATION —

System 58: Quantum Interference Glitter

Overview

Wave functions made visible. Double-slit, quantum harmonic oscillator, particle in a box — the probability amplitudes become brightness. Where waves constructively interfere = bright sparkles. Destructive interference = darkness. The quantum world, but you can see it.

Mathematical Foundation

Schrödinger Equation: `iℏ ∂ψ/∂t = Ĥψ`

Wave Function: `ψ(x,t) = |ψ| · e^(iφ)`. Probability density = `|ψ|²`.

Double-Slit: Two point sources, interference pattern:

```
ψ = ψ₁ + ψ₂ = A · e^(ikr₁)/r₁ + A · e^(ikr₂)/r₂
|ψ|² = A²(1/r₁² + 1/r₂² + 2 · cos(k(r₁-r₂))/(r₁ · r₂))
```

Quantum Harmonic Oscillator: Energy levels `E_n = ℏω(n + ½)`. Wave functions: Hermite polynomials × Gaussian.

GLSL Shader

See `shader.glsl` — wave function evaluation, interference patterns, probability density visualization.

JS Module

See `module.js` — wave packet initialization, time evolution, measurement simulation, eigenstate decomposition.

Showcase Builds

1. Double-Slit Sparkle

Classic interference. Two sources, bright and dark bands. Each band is a line of sparkles. The pattern shifts as you "observe" — wavefunction collapse made visual.

2. Harmonic Oscillator

Quantum energy levels as horizontal sparkle bands. n=0 = single central band. n=1 = two bands. n=2 = three bands. The quantization IS the banding.

3. Particle in a Box

Infinite square well. Standing wave patterns. Each mode is a different sparkle pattern. The box boundaries = nodes (darkness). The center = antinodes (brightness).

4. Wave Packet

Gaussian wave packet spreading and interfering with itself. The packet is a cloud of sparkles that disperses over time. Uncertainty principle made visible.

5. Measurement Flash

Simulate quantum measurement. The wavefunction collapses to an eigenstate — flash of light at measured position. Then re-spreads. Observation = destruction and rebirth.

Implementation Notes

- Use complex numbers in GLSL (vec2 as real/imaginary)
- Time evolution: multiply by phase factor e^(-iEt/)
- Interference: sum amplitudes, square magnitude
- Measurement: project onto position eigenstate, then re-initialize

— GLSL SHADER —

```
// System 58: Quantum Interference Glitter
// Wave functions made visible — probability amplitudes as brightness

uniform float u_time;
uniform vec2 u_resolution;
uniform float u_mode;           // 0=double-slit, 1=harmonic, 2=box, 3=wavepacket, 4=measurement
uniform float u_wavelength;     // de Broglie wavelength
uniform float u_slit_separation; // for double-slit
uniform float u_quantum_n;      // energy level for harmonic/box
uniform float u_spread;         // wave packet spread

#define PI 3.14159265359
#define TAU 6.28318530718

// Complex number operations (vec2 = real, imag)
vec2 cmul(vec2 a, vec2 b) {
    return vec2(a.x*b.x - a.y*b.y, a.x*b.y + a.y*b.x);
}

vec2 cexp(float theta) {
    return vec2(cos(theta), sin(theta));
}

float cabs2(vec2 z) {
    return z.x*z.x + z.y*z.y;
}

// Double-slit interference
vec2 doubleSlit(vec2 p, float wavelength, float slitSep, float time) {
    vec2 slit1 = vec2(-slitSep * 0.5, -0.5);
    vec2 slit2 = vec2(slitSep * 0.5, -0.5);

    float r1 = length(p - slit1);
    float r2 = length(p - slit2);
    float k = TAU / wavelength;

    // Amplitude from each slit
```

```
    vec2 psi1 = cexp(k * r1 - time * 2.0) / r1;
    vec2 psi2 = cexp(k * r2 - time * 2.0) / r2;

    return psi1 + psi2;
}

// Quantum harmonic oscillator wave function
// ψ_n(x) = (1/√(2^n n!)) · (mω/πℏ)^(1/4) · H_n(ξ) · e^(-ξ²/2)
// where ξ = √(mω/ℏ) · x
vec2 harmonicOscillator(vec2 p, float n, float time) {
    float x = p.x;
    float xi = x * 3.0; // scaled position

    // Hermite polynomials (first few)
    float H;
    if (n < 0.5) H = 1.0;                                    // H_0
    else if (n < 1.5) H = 2.0 * xi;                          // H_1
    else if (n < 2.5) H = 4.0 * xi * xi - 2.0;             // H_2
    else if (n < 3.5) H = 8.0 * xi * xi * xi - 12.0 * xi;  // H_3
    else H = 16.0 * pow(xi, 4.0) - 48.0 * xi * xi + 12.0;  // H_4

    float gaussian = exp(-xi * xi * 0.5);
    float amplitude = H * gaussian;

    // Time evolution: e^(-i E_n t/ℏ), E_n = n + 0.5
    float energy = n + 0.5;
    vec2 phase = cexp(-energy * time);

    return cmul(vec2(amplitude, 0.0), phase);
}

// Particle in a box (infinite square well)
// ψ_n(x) = √(2/L) · sin(nπx/L)
vec2 particleInBox(vec2 p, float n, float time) {
    float x = p.x;
    float L = 2.0; // box width

    // Map to box coordinates
    float boxX = (x + 1.0) * 0.5 * L;

    if (boxX < 0.0 || boxX > L) return vec2(0.0);

    float amplitude = sqrt(2.0 / L) * sin(n * PI * boxX / L);

    // Time evolution
    float energy = n * n; // E_n ∝ n²
    vec2 phase = cexp(-energy * time * 0.5);

    return cmul(vec2(amplitude, 0.0), phase);
}
```

```
// Gaussian wave packet
vec2 wavePacket(vec2 p, float spread, float time) {
    float x0 = sin(time * 0.5) * 0.5; // moving center
    float sigma = spread * (1.0 + time * 0.1); // spreading

    float amplitude = exp(-pow(p.x - x0, 2.0) / (2.0 * sigma * sigma));
    float k = 5.0; // wave number

    vec2 planeWave = cexp(k * p.x - time * 3.0);
    return vec2(amplitude, 0.0) * planeWave;
}

// Measurement: collapse to position eigenstate
float measurementFlash(vec2 p, float time, float lastMeasurement) {
    float flashTime = 0.3;
    float tSinceMeasure = fract(time / 3.0) * 3.0;

    if (tSinceMeasure < flashTime) {
        // Flash at measured position
        float measuredX = sin(lastMeasurement * 7.3) * 0.8;
        float d = abs(p.x - measuredX);
        return exp(-d * d * 50.0) * (1.0 - tSinceMeasure / flashTime);
    }

    return 0.0;
}

// Probability density → sparkle brightness
vec3 probabilityToSparkle(float prob, float phase, float mode) {
    float brightness = prob * 2.0;

    // Phase → hue
    float hue = fract(phase / TAU);
    vec3 color;
    float h = hue * 6.0;
    float i = floor(h);
    float f = h - i;
    float q = 1.0 - f;

    if (i == 0.0) color = vec3(1.0, f, 0.0);
    else if (i == 1.0) color = vec3(q, 1.0, 0.0);
    else if (i == 2.0) color = vec3(0.0, 1.0, f);
    else if (i == 3.0) color = vec3(0.0, q, 1.0);
    else if (i == 4.0) color = vec3(f, 0.0, 1.0);
    else color = vec3(1.0, 0.0, q);

    // Quantum modes have specific colors
    if (mode < 0.5) {
        // Double-slit: interference colors
        color = mix(vec3(0.2, 0.5, 1.0), vec3(1.0, 0.8, 0.2), brightness);
    } else if (mode < 1.5) {
```

```
        // Harmonic: energy level colors
        color = mix(vec3(0.1, 0.3, 0.8), vec3(0.9, 0.4, 0.1), brightness);
    } else if (mode < 2.5) {
        // Box: standing wave colors
        color = mix(vec3(0.2, 0.1, 0.5), vec3(0.8, 0.9, 1.0), brightness);
    } else if (mode < 3.5) {
        // Wave packet: dispersing cloud
        color = mix(vec3(0.1, 0.2, 0.4), vec3(0.6, 0.8, 1.0), brightness);
    } else {
        // Measurement: flash white
        color = vec3(1.0, 0.95, 0.9) * brightness;
    }

    return color * brightness;
}

void main() {
    vec2 uv = gl_FragCoord.xy / u_resolution.xy;
    vec2 p = uv * 2.0 - 1.0;
    p.x *= u_resolution.x / u_resolution.y;

    vec3 color = vec3(0.02, 0.02, 0.05);

    vec2 psi;
    float prob;
    float phase;

    if (u_mode < 0.5) {
        // Double-slit
        psi = doubleSlit(p, u_wavelength, u_slit_separation, u_time);
        prob = cabs2(psi);
        phase = atan(psi.y, psi.x);

    } else if (u_mode < 1.5) {
        // Harmonic oscillator
        psi = harmonicOscillator(p, u_quantum_n, u_time);
        prob = cabs2(psi);
        phase = atan(psi.y, psi.x);

    } else if (u_mode < 2.5) {
        // Particle in a box
        psi = particleInBox(p, u_quantum_n, u_time);
        prob = cabs2(psi);
        phase = atan(psi.y, psi.x);

    } else if (u_mode < 3.5) {
        // Wave packet
        psi = wavePacket(p, u_spread, u_time);
        prob = cabs2(psi);
        phase = atan(psi.y, psi.x);
```

```
    } else {
        // Measurement
        psi = wavePacket(p, u_spread, u_time);
        prob = cabs2(psi);
        phase = atan(psi.y, psi.x);

        float flash = measurementFlash(p, u_time, floor(u_time / 3.0));
        color += vec3(flash);
    }

    // Convert probability to sparkle
    vec3 sparkleColor = probabilityToSparkle(prob, phase, u_mode);

    // Add sparkle noise for quantum "graininess"
    float grain = hash(p * 100.0 + u_time) * 0.05;
    sparkleColor += vec3(grain);

    color += sparkleColor;

    // Tone mapping
    color = color / (1.0 + color * 0.3);

    // Vignette
    float vignette = 1.0 - smoothstep(0.5, 1.5, length(p));
    color *= vignette;

    gl_FragColor = vec4(color, 1.0);
}

float hash(vec2 p) {
    return fract(sin(dot(p, vec2(12.9898, 78.233))) * 43758.5453);
}
```

— JS MODULE —

```
// System 58: Quantum Interference Glitter — JS Module
// Wave functions, time evolution, measurement simulation

export class QuantumInterferenceGlitterSystem {
    constructor(gridSize = 256) {
        this.gridSize = gridSize;
        this.mode = 0;
        this.time = 0;

        // Quantum parameters
        this.wavelength = 0.1;
        this.slitSeparation = 0.3;
        this.quantumN = 2;
        this.spread = 0.1;

        // Wave function grid: [real, imag] for each point
```

```
        this.psi = new Float32Array(gridSize * gridSize * 2);

        // Measurement history
        this.measurements = [];
        this.lastMeasurement = 0;

        this.initWaveFunction();
    }

    // Initialize wave function based on mode
    initWaveFunction() {
        const N = this.gridSize;

        for (let y = 0; y < N; y++) {
            for (let x = 0; x < N; x++) {
                const idx = (y * N + x) * 2;
                const px = (x / N - 0.5) * 2;
                const py = (y / N - 0.5) * 2;

                let real, imag;

                if (this.mode === 0) {
                    // Double-slit: two sources
                    const s1 = {x: -this.slitSeparation * 0.5, y: -0.5};
                    const s2 = {x: this.slitSeparation * 0.5, y: -0.5};
                    const r1 = Math.sqrt((px - s1.x)**2 + (py - s1.y)**2);
                    const r2 = Math.sqrt((px - s2.x)**2 + (py - s2.y)**2);
                    const k = 2 * Math.PI / this.wavelength;

                    real = (Math.cos(k * r1) / r1 + Math.cos(k * r2) / r2) * 0.1;
                    imag = (Math.sin(k * r1) / r1 + Math.sin(k * r2) / r2) * 0.1;

                } else if (this.mode === 1) {
                    // Harmonic oscillator
                    const xi = px * 3;
                    const n = this.quantumN;
                    let H;
                    if (n === 0) H = 1;
                    else if (n === 1) H = 2 * xi;
                    else if (n === 2) H = 4 * xi**2 - 2;
                    else H = 8 * xi**3 - 12 * xi;

                    const amp = H * Math.exp(-xi**2 * 0.5);
                    real = amp * 0.3;
                    imag = 0;

                } else if (this.mode === 2) {
                    // Particle in box
                    const L = 2;
                    const boxX = (px + 1) * 0.5 * L;
                    if (boxX > 0 && boxX < L) {
```

```
                    real = Math.sqrt(2/L) * Math.sin(n * Math.PI * boxX / L) * 0.5;
                } else {
                    real = 0;
                }
                imag = 0;

            } else {
                // Wave packet / measurement
                const x0 = 0;
                const sigma = this.spread;
                const amp = Math.exp(-(px - x0)**2 / (2 * sigma**2));
                const k = 5;
                real = amp * Math.cos(k * px) * 0.5;
                imag = amp * Math.sin(k * px) * 0.5;
            }

            this.psi[idx] = real;
            this.psi[idx + 1] = imag;
        }
    }

    this.normalize();
}

// Normalize wave function
normalize() {
    let sum = 0;
    for (let i = 0; i < this.psi.length; i += 2) {
        sum += this.psi[i]**2 + this.psi[i+1]**2;
    }

    const norm = Math.sqrt(sum);
    if (norm > 0) {
        for (let i = 0; i < this.psi.length; i++) {
            this.psi[i] /= norm;
        }
    }
}

// Time evolution: multiply by phase factor
// ψ(t) = ψ(0) · e^(-iEt/ħ)
evolveTime(dt) {
    const N = this.gridSize;

    for (let y = 0; y < N; y++) {
        for (let x = 0; x < N; x++) {
            const idx = (y * N + x) * 2;
            const real = this.psi[idx];
            const imag = this.psi[idx + 1];

            // Energy depends on mode
```

```
            let E;
            if (this.mode === 1) {
                // Harmonic: E_n = n + 0.5
                E = this.quantumN + 0.5;
            } else if (this.mode === 2) {
                // Box: E_n ∝ n²
                E = this.quantumN**2;
            } else {
                E = 1.0;
            }

            const phase = -E * dt;
            const cos_p = Math.cos(phase);
            const sin_p = Math.sin(phase);

            // e^(-iEt) = cos(Et) - i·sin(Et)
            this.psi[idx] = real * cos_p - imag * (-sin_p);
            this.psi[idx + 1] = real * (-sin_p) + imag * cos_p;
        }
    }
}

// Simulate measurement: collapse to position eigenstate
measure() {
    // Compute probability density
    const probs = new Float32Array(this.gridSize * this.gridSize);
    let totalProb = 0;

    for (let i = 0; i < this.psi.length; i += 2) {
        const prob = this.psi[i]**2 + this.psi[i+1]**2;
        probs[i/2] = prob;
        totalProb += prob;
    }

    // Sample position from probability distribution
    let rand = Math.random() * totalProb;
    let cumsum = 0;
    let measuredIdx = 0;

    for (let i = 0; i < probs.length; i++) {
        cumsum += probs[i];
        if (cumsum >= rand) {
            measuredIdx = i;
            break;
        }
    }

    // Collapse wavefunction: delta function at measured position
    for (let i = 0; i < this.psi.length; i++) {
        this.psi[i] = 0;
    }
```

```
        this.psi[measuredIdx * 2] = 1;
        this.psi[measuredIdx * 2 + 1] = 0;

        this.lastMeasurement = measuredIdx;
        this.measurements.push({
            position: measuredIdx,
            time: this.time
        });

        // Re-spread after measurement
        setTimeout(() => this.initWaveFunction(), 100);
    }

    // Get probability density for shader
    getProbabilityData() {
        const data = new Float32Array(this.gridSize * this.gridSize * 4);

        for (let i = 0; i < this.psi.length; i += 2) {
            const prob = this.psi[i]**2 + this.psi[i+1]**2;
            const phase = Math.atan2(this.psi[i+1], this.psi[i]);

            data[i/2 * 4] = prob;
            data[i/2 * 4 + 1] = phase;
            data[i/2 * 4 + 2] = 0;
            data[i/2 * 4 + 3] = 1;
        }

        return data;
    }

    // Get uniforms for shader
    getUniforms() {
        return {
            u_time: this.time,
            u_mode: this.mode,
            u_wavelength: this.wavelength,
            u_slit_separation: this.slitSeparation,
            u_quantum_n: this.quantumN,
            u_spread: this.spread
        };
    }

    // Main update
    update(dt) {
        this.time += dt;
        this.evolveTime(dt);

        // Auto-measure in measurement mode
        if (this.mode === 4 && Math.floor(this.time / 3) > this.lastMeasurement) {
            this.measure();
        }
```

```
    }

    setMode(mode) {
        this.mode = mode;
        this.initWaveFunction();
    }

    setParams(wavelength, slitSep, n, spread) {
        this.wavelength = wavelength;
        this.slitSeparation = slitSep;
        this.quantumN = n;
        this.spread = spread;
        this.initWaveFunction();
    }
}
```

==========

SYSTEM 59: TOPOLOGY CHROME

==========

— DOCUMENTATION —

System 59: Topology Chrome

Overview

Chrome that knows its shape. Euler characteristic, genus, homology — topological invariants made visible. A torus reflects differently than a sphere. A Klein bottle has no inside or outside. The chrome IS the topology. You see the shape of space itself.

Mathematical Foundation

Euler Characteristic: `χ = V - E + F` (vertices - edges + faces)

- Sphere: = 2
- Torus: = 0
- Klein bottle: = 0 (but non-orientable)

Genus: Number of "holes". g = (2 -)/2 for orientable surfaces.

Fundamental Group: (M) — loops that can't be contracted. Torus has two independent loops.

Homology: H = connected components, H = 1D holes, H = 2D voids.

GLSL Shader

See `shader.glsl` — ray marching on implicit surfaces, topological coloring, fundamental group visualization.

JS Module

See `module.js` — implicit surface evaluation, topology computation, genus detection, shape morphing.

Showcase Builds

1. Sphere Mirror

= 2, g = 0. All loops contract. The chrome is simply connected. Reflection is "normal" — what you'd expect from a perfect sphere.

2. Torus Jewelry

= 0, g = 1. Two independent loop directions. The chrome has a grain — reflection depends on which loop you're on. The ring is a map of the torus.

3. Klein Bottle

= 0, non-orientable. No inside/outside. The chrome is one-sided. You can trace a path that returns mirrored. The surface drinks its own reflection.

4. Genus Garden

Multiple holes, multiple handles. = 2 - 2g. Each handle = a different reflection channel. The chrome is a switchboard of spaces.

5. Shape Morph

Continuous deformation between topologies. Sphere → torus (add handle) → sphere (remove different handle). The chrome passes through singularities. Topology change = catastrophe.

Implementation Notes

- Use implicit surfaces: f(x,y,z) = 0
- Torus: `(√(x²+y²) - R)² + z² = r²`
- Klein bottle: parametric or 4D immersion
- Ray marching with SDFs (signed distance functions)
- Topological coloring: different color per homology class

— GLSL SHADER —

```
// System 59: Topology Chrome
// Chrome that knows its shape — topological invariants as reflection

uniform float u_time;
uniform vec2 u_resolution;
uniform float u_mode;          // 0=sphere, 1=torus, 2=klein, 3=genus, 4=morph
uniform float u_genus;         // number of holes
uniform float u_morph_t;       // morph parameter

#define PI 3.14159265359
#define TAU 6.28318530718
```

```
// SDF primitives
float sdSphere(vec3 p, float r) {
    return length(p) - r;
}

float sdTorus(vec3 p, vec2 t) {
    vec2 q = vec2(length(p.xz) - t.x, p.y);
    return length(q) - t.y;
}

float sdBox(vec3 p, vec3 b) {
    vec3 d = abs(p) - b;
    return min(max(d.x, max(d.y, d.z)), 0.0) + length(max(d, 0.0));
}

// Klein bottle SDF (approximation)
float sdKlein(vec3 p, float scale) {
    p /= scale;
    float x = p.x, y = p.y, z = p.z;
    float a = 2.0;
    float r = a + cos(y * 0.5) * sin(x * 0.5) - sin(y * 0.5) * cos(x * 0.5) * 0.5;
    float dx = r * cos(y) - x;
    float dy = r * sin(y) - z;
    float dz = -sin(y * 0.5) * sin(x * 0.5) - cos(y * 0.5) * cos(x * 0.5) * 0.5 - y;
    return length(vec3(dx, dy, dz)) * scale;
}

// Genus-g surface (g holes)
float sdGenus(vec3 p, int g, float scale) {
    float d = sdSphere(p, scale);

    // Subtract tunnels
    for (int i = 0; i < 5; i++) {
        if (i >= g) break;
        float angle = float(i) * TAU / float(g);
        vec3 tunnelCenter = vec3(cos(angle), 0.0, sin(angle)) * scale * 0.6;
        float tunnel = sdTorus(p - tunnelCenter, vec2(scale * 0.3, scale * 0.15));
        d = max(d, -tunnel); // CSG subtraction
    }

    return d;
}

// Morph between shapes
float sdMorph(vec3 p, float t) {
    float sphere = sdSphere(p, 0.5);
    float torus = sdTorus(p, vec2(0.4, 0.15));

    // Smooth morph
    float morph = smoothstep(0.0, 1.0, t);
    return mix(sphere, torus, morph);
```

```
}

// Normal from SDF
vec3 calcNormal(vec3 p, float mode, float genus, float morph_t) {
    float eps = 0.001;
    vec2 h = vec2(eps, 0.0);

    float d;
    if (mode < 0.5) d = sdSphere(p, 0.5);
    else if (mode < 1.5) d = sdTorus(p, vec2(0.4, 0.15));
    else if (mode < 2.5) d = sdKlein(p, 0.5);
    else if (mode < 3.5) d = sdGenus(p, int(genus), 0.5);
    else d = sdMorph(p, morph_t);

    return normalize(vec3(
        d - sdSphere(p - h.xyy, 0.5),
        d - sdSphere(p - h.yxy, 0.5),
        d - sdSphere(p - h.yyx, 0.5)
    ));
}

// Evaluate SDF
float map(vec3 p, float mode, float genus, float morph_t) {
    if (mode < 0.5) return sdSphere(p, 0.5);
    else if (mode < 1.5) return sdTorus(p, vec2(0.4, 0.15));
    else if (mode < 2.5) return sdKlein(p, 0.5);
    else if (mode < 3.5) return sdGenus(p, int(genus), 0.5);
    else return sdMorph(p, morph_t);
}

// Topology-based coloring
vec3 topologyColor(vec3 p, vec3 normal, float mode, float genus) {
    vec3 color;

    if (mode < 0.5) {
        // Sphere: simply connected, uniform
        float lat = asin(normal.y) / PI + 0.5;
        float lon = atan(normal.z, normal.x) / TAU + 0.5;
        color = vec3(lon, lat, 0.5);

    } else if (mode < 1.5) {
        // Torus: two loop directions
        float major = atan(p.z, p.x) / TAU + 0.5; // major loop
        float minor = atan(p.y, length(p.xz) - 0.4) / TAU + 0.5; // minor loop
        color = vec3(major, minor, 0.5);

    } else if (mode < 2.5) {
        // Klein bottle: non-orientable
        float u = atan(p.z, p.x) / TAU + 0.5;
        float v = p.y * 0.5 + 0.5;
        // Mirror every other cycle (non-orientable)
```

```
        float mirror = step(0.5, fract(u * 2.0));
        color = mix(vec3(u, v, 0.5), vec3(1.0 - u, v, 0.5), mirror);

    } else if (mode < 3.5) {
        // Genus garden: color by hole
        float angle = atan(p.z, p.x);
        float hole = floor((angle / TAU + 0.5) * genus) / genus;
        color = vec3(hole, 0.5, 1.0 - hole);

    } else {
        // Morph: interpolate colors
        color = mix(vec3(0.5, 0.7, 1.0), vec3(1.0, 0.5, 0.3), morph_t);
    }

    return color;
}

// Ray marching
vec4 rayMarch(vec3 ro, vec3 rd, float mode, float genus, float morph_t) {
    float t = 0.0;
    float d;

    for (int i = 0; i < 100; i++) {
        vec3 p = ro + rd * t;
        d = map(p, mode, genus, morph_t);

        if (d < 0.001 || t > 5.0) break;
        t += d * 0.5;
    }

    return vec4(ro + rd * t, d);
}

// Chrome reflection
vec3 chromeReflect(vec3 p, vec3 normal, vec3 rd, float mode, float genus) {
    // Reflect view direction
    vec3 reflectDir = reflect(rd, normal);

    // Environment
    vec3 envColor = vec3(
        0.5 + 0.5 * sin(reflectDir.x * 3.0 + u_time * 0.3),
        0.5 + 0.5 * sin(reflectDir.y * 2.0 + u_time * 0.5 + 1.0),
        0.5 + 0.5 * sin(reflectDir.z * 4.0 + u_time * 0.7 + 2.0)
    );

    // Topology color
    vec3 topoColor = topologyColor(p, normal, mode, genus);

    // Fresnel
    float fresnel = pow(1.0 - max(dot(-rd, normal), 0.0), 3.0);
```

```
    return mix(topoColor * 0.5, envColor, fresnel);
}

void main() {
    vec2 uv = gl_FragCoord.xy / u_resolution.xy;
    vec2 p = uv * 2.0 - 1.0;
    p.x *= u_resolution.x / u_resolution.y;

    // Camera
    vec3 ro = vec3(0.0, 0.0, 2.0);
    vec3 rd = normalize(vec3(p, -1.0));

    // Rotate camera
    float camAngle = u_time * 0.2;
    ro = vec3(
        ro.x * cos(camAngle) - ro.z * sin(camAngle),
        ro.y,
        ro.x * sin(camAngle) + ro.z * cos(camAngle)
    );

    // Ray march
    vec4 hit = rayMarch(ro, rd, u_mode, u_genus, u_morph_t);
    vec3 hitPoint = hit.xyz;
    float dist = hit.w;

    vec3 color = vec3(0.02, 0.02, 0.05);

    if (dist < 0.001) {
        vec3 normal = calcNormal(hitPoint, u_mode, u_genus, u_morph_t);
        color = chromeReflect(hitPoint, normal, rd, u_mode, u_genus);
    }

    // Vignette
    float vignette = 1.0 - smoothstep(0.5, 1.5, length(p));
    color *= vignette;

    gl_FragColor = vec4(color, 1.0);
}
```

— JS MODULE —

```
// System 59: Topology Chrome — JS Module
// Implicit surfaces, topology computation, shape morphing

export class TopologyChromeSystem {
    constructor() {
        this.mode = 0;
        this.time = 0;
        this.genus = 1;
        this.morphT = 0;
```

```
        // Surface parameters
        this.sphereRadius = 0.5;
        this.torusMajor = 0.4;
        this.torusMinor = 0.15;

        // Topology invariants (computed)
        this.eulerCharacteristic = 2;
        this.currentGenus = 0;
    }

    // SDF: Sphere
    sdSphere(p, r) {
        const len = Math.sqrt(p.x**2 + p.y**2 + p.z**2);
        return len - r;
    }

    // SDF: Torus
    sdTorus(p, R, r) {
        const qx = Math.sqrt(p.x**2 + p.z**2) - R;
        const qy = p.y;
        return Math.sqrt(qx**2 + qy**2) - r;
    }

    // SDF: Box
    sdBox(p, b) {
        const d = [
            Math.abs(p.x) - b.x,
            Math.abs(p.y) - b.y,
            Math.abs(p.z) - b.z
        ];
        const inside = Math.min(Math.max(d[0], Math.max(d[1], d[2])), 0);
        const outside = Math.sqrt(
            Math.max(d[0], 0)**2 +
            Math.max(d[1], 0)**2 +
            Math.max(d[2], 0)**2
        );
        return inside + outside;
    }

    // Evaluate SDF at point
    evaluateSDF(p) {
        switch(this.mode) {
            case 0: return this.sdSphere(p, this.sphereRadius);
            case 1: return this.sdTorus(p, this.torusMajor, this.torusMinor);
            case 2: return this.sdKlein(p, 0.5);
            case 3: return this.sdGenus(p, this.genus, 0.5);
            case 4: return this.morphT * this.sdTorus(p, this.torusMajor, this.torusMinor) +
                           (1 - this.morphT) * this.sdSphere(p, this.sphereRadius);
            default: return this.sdSphere(p, this.sphereRadius);
        }
    }
```

```
// Approximate normal via central differences
calcNormal(p) {
    const eps = 0.001;
    return {
        x: this.evaluateSDF({x: p.x + eps, y: p.y, z: p.z}) -
           this.evaluateSDF({x: p.x - eps, y: p.y, z: p.z}),
        y: this.evaluateSDF({x: p.x, y: p.y + eps, z: p.z}) -
           this.evaluateSDF({x: p.x, y: p.y - eps, z: p.z}),
        z: this.evaluateSDF({x: p.x, y: p.y, z: p.z + eps}) -
           this.evaluateSDF({x: p.x, y: p.y, z: p.z - eps})
    };
}

// Compute Euler characteristic (sampling approach)
computeEulerCharacteristic() {
    // Sample grid of points
    const samples = 20;
    let inside = 0;
    let surface = 0;

    for (let x = -1; x <= 1; x += 2/samples) {
        for (let y = -1; y <= 1; y += 2/samples) {
            for (let z = -1; z <= 1; z += 2/samples) {
                const d = this.evaluateSDF({x, y, z});
                if (d < 0) inside++;
                if (Math.abs(d) < 0.05) surface++;
            }
        }
    }

    // Rough approximation: χ ≈ 2 for sphere, 0 for torus
    const volume = inside / (samples**3);
    this.eulerCharacteristic = volume > 0.5 ? 2 : 0;
    this.currentGenus = (2 - this.eulerCharacteristic) / 2;

    return this.eulerCharacteristic;
}

// Morph between shapes
morph(targetMode, duration = 2.0) {
    const startT = this.morphT;
    const startTime = this.time;

    const animate = () => {
        const elapsed = this.time - startTime;
        const progress = Math.min(elapsed / duration, 1.0);

        if (this.mode === 4) {
            this.morphT = startT + (targetMode === 1 ? 1 : -1) * progress;
            this.morphT = Math.max(0, Math.min(1, this.morphT));
```

```
        }

        if (progress < 1.0) {
            requestAnimationFrame(animate);
        } else {
            this.mode = targetMode;
        }
    };

    this.mode = 4; // morph mode
    animate();
}

// Get topology info
getTopologyInfo() {
    return {
        eulerCharacteristic: this.eulerCharacteristic,
        genus: this.currentGenus,
        orientable: this.mode !== 2, // Klein is non-orientable
        name: this.getShapeName()
    };
}

getShapeName() {
    const names = ['Sphere', 'Torus', 'Klein Bottle', 'Genus-g', 'Morphing'];
    return names[this.mode] || 'Unknown';
}

// Get uniforms for shader
getUniforms() {
    return {
        u_time: this.time,
        u_mode: this.mode,
        u_genus: this.genus,
        u_morph_t: this.morphT
    };
}

// Main update
update(dt) {
    this.time += dt;
}

setMode(mode) {
    this.mode = mode;
    this.computeEulerCharacteristic();
}

setGenus(g) {
    this.genus = Math.max(1, Math.floor(g));
    if (this.mode === 3) {
```

```
            this.computeEulerCharacteristic();
        }
    }
}

// Klein bottle SDF (simplified parametric)
TopologyChromeSystem.prototype.sdKlein = function(p, scale) {
    p = {x: p.x/scale, y: p.y/scale, z: p.z/scale};
    const a = 2.0;
    const r = a + Math.cos(p.y * 0.5) * Math.sin(p.x * 0.5) -
              Math.sin(p.y * 0.5) * Math.cos(p.x * 0.5) * 0.5;
    const dx = r * Math.cos(p.y) - p.x;
    const dy = r * Math.sin(p.y) - p.z;
    const dz = -Math.sin(p.y * 0.5) * Math.sin(p.x * 0.5) -
               Math.cos(p.y * 0.5) * Math.cos(p.x * 0.5) * 0.5 - p.y;
    return Math.sqrt(dx*dx + dy*dy + dz*dz) * scale;
};

// Genus-g surface SDF
TopologyChromeSystem.prototype.sdGenus = function(p, g, scale) {
    let d = this.sdSphere(p, scale);

    for (let i = 0; i < g; i++) {
        const angle = (i / g) * Math.PI * 2;
        const tc = {
            x: Math.cos(angle) * scale * 0.6,
            y: 0,
            z: Math.sin(angle) * scale * 0.6
        };
        const tunnel = this.sdTorus(
            {x: p.x - tc.x, y: p.y - tc.y, z: p.z - tc.z},
            scale * 0.3,
            scale * 0.15
        );
        d = Math.max(d, -tunnel); // CSG subtraction
    }

    return d;
};
```

==========

SYSTEM 60: INFORMATION THEORY SPARKLE

==========

— DOCUMENTATION —

System 60: Information Theory Sparkle

Overview

Entropy made visible. Shannon entropy = sparkle density. Mutual information = connected sparkles. KL divergence = color shift between distributions. The chrome surface is a communication channel, and the sparkles are the message. Information IS the glitter.

Mathematical Foundation

Shannon Entropy: `H(X) = -Σ p(x) · log₂p(x)`

- High entropy = uniform distribution = dense, even sparkles
- Low entropy = concentrated = sparse, clustered sparkles

Mutual Information: `I(X;Y) = H(X) - H(X|Y)`

- Measures dependence between variables
- High MI = sparkles are connected/aligned
- Low MI = independent, random

KL Divergence: `D_KL(P||Q) = Σ p(x) · log(p(x)/q(x))`

- Measures difference between distributions
- Visualized as color shift from reference

Channel Capacity: `C = max_p I(X;Y)`

- Maximum information throughput
- Chrome "bandwidth" = sparkle update rate

GLSL Shader

See `shader.glsl` — entropy field computation, mutual information visualization, divergence coloring.

JS Module

See `module.js` — probability distribution estimation, entropy calculation, information dynamics, compression ratio tracking.

Showcase Builds

1. Entropy Field

Real-time entropy of pixel neighborhoods. Uniform regions = bright even sparkles. Edges/texture = chaotic sparkle patterns. The image reveals its own information content.

2. Mutual Information Web

Connected regions share information = connected sparkles. The web of mutual information IS the web of sparkles. Cut a connection = two separate constellations.

3. KL Divergence Chrome

Two chrome surfaces with different statistics. The divergence between them = color difference. As they converge, colors merge. Information distance made chromatic.

4. Compression Art

Sparkle density = compression ratio. Highly compressible = few sparkles (low entropy). Incompressible = dense sparkles (high entropy). The art IS the compression limit.

5. Channel Noise

Add noise to the sparkle channel. Capacity decreases = sparkles blur. At channel capacity = maximum sparkle clarity. Shannon limit made visible.

Implementation Notes

- Estimate local probability distributions from pixel neighborhoods
- Compute entropy via histogram binning
- Mutual information: joint histogram vs product of marginals
- KL divergence: compare current frame to reference
- Use logarithmic scaling for entropy visualization

— GLSL SHADER —

```
// System 60: Information Theory Sparkle
// Entropy, mutual information, KL divergence — information IS the glitter

uniform float u_time;
uniform vec2 u_resolution;
uniform float u_mode;           // 0=entropy, 1=mutual info, 2=KL div, 3=compression, 4=channel noise
uniform sampler2D u_prevFrame;  // Previous frame for temporal analysis
uniform sampler2D u_reference;  // Reference distribution
uniform float u_noise_level;    // channel noise parameter

#define PI 3.14159265359

// Hash for sparkle variation
float hash(vec2 p) {
    return fract(sin(dot(p, vec2(12.9898, 78.233))) * 43758.5453);
}

// Local entropy estimation (simplified)
float localEntropy(vec2 uv, vec2 resolution) {
    // Sample 3x3 neighborhood
    float hist[8];
    for (int i = 0; i < 8; i++) hist[i] = 0.0;

    vec2 offset = 1.0 / resolution;

    for (int x = -1; x <= 1; x++) {
        for (int y = -1; y <= 1; y++) {
```

```
            vec2 sampleUV = uv + vec2(float(x), float(y)) * offset;
            float val = texture2D(u_prevFrame, sampleUV).r;
            int bin = int(val * 7.0);
            hist[bin] += 1.0;
        }
    }

    // Compute entropy
    float entropy = 0.0;
    float total = 9.0;
    for (int i = 0; i < 8; i++) {
        float p = hist[i] / total;
        if (p > 0.0) {
            entropy -= p * log2(p);
        }
    }

    return entropy / 3.0; // normalize to [0,1]
}

// Mutual information between two regions
float mutualInformation(vec2 uv1, vec2 uv2, vec2 resolution) {
    // Simplified: correlation between two points
    float val1 = texture2D(u_prevFrame, uv1).r;
    float val2 = texture2D(u_prevFrame, uv2).r;

    float joint = (val1 + val2) * 0.5;
    float marginal1 = val1;
    float marginal2 = val2;

    // MI ≈ joint - marginals (simplified)
    float mi = abs(joint - marginal1 * marginal2);
    return mi;
}

// KL divergence from reference
float klDivergence(vec2 uv, sampler2D reference) {
    float p = texture2D(u_prevFrame, uv).r;
    float q = texture2D(reference, uv).r;

    if (p > 0.0 && q > 0.0) {
        return p * log2(p / q);
    }
    return 0.0;
}

// Entropy → sparkle density
vec3 entropyToSparkle(float entropy, vec2 uv, float time) {
    // High entropy = dense sparkles
    float density = entropy;
```

```
    // Sparkle threshold
    float sparkle = step(1.0 - density, hash(uv * 100.0 + time));

    // Color from entropy level
    vec3 lowEntropyColor = vec3(0.2, 0.1, 0.5);   // ordered: deep purple
    vec3 highEntropyColor = vec3(1.0, 0.9, 0.3);  // chaotic: warm gold

    vec3 color = mix(lowEntropyColor, highEntropyColor, entropy);

    return color * sparkle;
}

// Mutual information → connection lines
vec3 mutualInfoToSparkle(vec2 uv, vec2 resolution, float time) {
    vec3 color = vec3(0.0);

    // Check connections to neighbors
    vec2 offset = 2.0 / resolution;

    for (int x = -2; x <= 2; x++) {
        for (int y = -2; y <= 2; y++) {
            if (x == 0 && y == 0) continue;

            vec2 neighborUV = uv + vec2(float(x), float(y)) * offset;
            float mi = mutualInformation(uv, neighborUV, resolution);

            if (mi > 0.3) {
                // Draw connection
                vec2 mid = (uv + neighborUV) * 0.5;
                float d = length(uv - mid);
                float line = exp(-d * d * 500.0) * mi;

                vec3 connectionColor = vec3(0.3, 0.7, 1.0) * line;
                color += connectionColor;
            }
        }
    }

    return color;
}

// KL divergence → color shift
vec3 klDivToSparkle(float kl, vec2 uv, float time) {
    // KL > 0 = different from reference = color shift
    float shift = smoothstep(0.0, 1.0, kl);

    vec3 referenceColor = vec3(0.5, 0.6, 0.7);
    vec3 shiftedColor = vec3(0.9, 0.3, 0.2);

    vec3 color = mix(referenceColor, shiftedColor, shift);
```

```
    // Add sparkle based on divergence magnitude
    float sparkle = step(1.0 - shift * 0.5, hash(uv * 100.0 + time));

    return color * sparkle;
}

// Compression ratio → sparkle density
vec3 compressionToSparkle(vec2 uv, vec2 resolution, float time) {
    // Estimate local compressibility via smoothness
    vec2 offset = 1.0 / resolution;
    float center = texture2D(u_prevFrame, uv).r;
    float n = texture2D(u_prevFrame, uv + vec2(0.0, offset.y)).r;
    float s = texture2D(u_prevFrame, uv - vec2(0.0, offset.y)).r;
    float e = texture2D(u_prevFrame, uv + vec2(offset.x, 0.0)).r;
    float w = texture2D(u_prevFrame, uv - vec2(offset.x, 0.0)).r;

    // Variance = incompressibility
    float variance = abs(center - n) + abs(center - s) + abs(center - e) + abs(center - w);
    float compressibility = 1.0 - smoothstep(0.0, 1.0, variance);

    // Low compressibility = many sparkles (high entropy)
    float density = 1.0 - compressibility;
    float sparkle = step(1.0 - density, hash(uv * 100.0 + time));

    vec3 color = mix(vec3(0.1, 0.4, 0.8), vec3(1.0, 0.8, 0.2), density);

    return color * sparkle;
}

// Channel noise → blurred sparkles
vec3 channelNoiseSparkle(vec2 uv, float noiseLevel, float time) {
    // Add noise to position
    vec2 noisyUV = uv + vec2(
        hash(uv + time) * noiseLevel * 0.1,
        hash(uv + time + 1.0) * noiseLevel * 0.1
    );

    float val = texture2D(u_prevFrame, noisyUV).r;

    // Capacity = 1 - noiseLevel
    float capacity = 1.0 - noiseLevel;
    float sparkle = step(1.0 - capacity * 0.5, hash(uv * 100.0 + time));

    // Blur proportional to noise
    float blur = noiseLevel * 0.5;
    vec3 color = vec3(val * (1.0 - blur) + 0.5 * blur);

    return color * sparkle;
}

void main() {
```

```
    vec2 uv = gl_FragCoord.xy / u_resolution.xy;
    vec2 p = uv * 2.0 - 1.0;
    p.x *= u_resolution.x / u_resolution.y;

    vec3 color = vec3(0.02, 0.02, 0.05);

    if (u_mode < 0.5) {
        // Entropy Field
        float entropy = localEntropy(uv, u_resolution);
        color += entropyToSparkle(entropy, uv, u_time);

    } else if (u_mode < 1.5) {
        // Mutual Information Web
        color += mutualInfoToSparkle(uv, u_resolution, u_time);

        // Add node sparkles
        float val = texture2D(u_prevFrame, uv).r;
        float node = step(0.7, val) * hash(uv * 50.0 + u_time);
        color += vec3(0.8, 0.9, 1.0) * node;

    } else if (u_mode < 2.5) {
        // KL Divergence
        float kl = klDivergence(uv, u_reference);
        color += klDivToSparkle(kl, uv, u_time);

    } else if (u_mode < 3.5) {
        // Compression Art
        color += compressionToSparkle(uv, u_resolution, u_time);

    } else {
        // Channel Noise
        color += channelNoiseSparkle(uv, u_noise_level, u_time);
    }

    // Tone mapping
    color = color / (1.0 + color * 0.3);

    // Vignette
    float vignette = 1.0 - smoothstep(0.5, 1.5, length(p));
    color *= vignette;

    gl_FragColor = vec4(color, 1.0);
}
```

— JS MODULE —

```
// System 60: Information Theory Sparkle — JS Module
// Entropy, mutual information, KL divergence, compression

export class InformationTheorySparkleSystem {
    constructor(width, height) {
```

```
        this.width = width;
        this.height = height;
        this.mode = 0;
        this.time = 0;

        // Frame data
        this.currentFrame = new Float32Array(width * height);
        this.prevFrame = new Float32Array(width * height);
        this.referenceFrame = new Float32Array(width * height);

        // Information metrics
        this.entropyField = new Float32Array(width * height);
        this.mutualInfoField = new Float32Array(width * height);
        this.klField = new Float32Array(width * height);

        // Channel parameters
        this.noiseLevel = 0.1;
        this.capacity = 1.0;

        // History for entropy estimation
        this.frameHistory = [];
        this.maxHistory = 10;
    }

    // Compute Shannon entropy of a distribution
    computeEntropy(histogram) {
        let entropy = 0;
        const total = histogram.reduce((a, b) => a + b, 0);

        for (let count of histogram) {
            if (count > 0) {
                const p = count / total;
                entropy -= p * Math.log2(p);
            }
        }

        return entropy;
    }

    // Local entropy from neighborhood
    computeLocalEntropy(x, y, radius = 2) {
        const bins = 8;
        const histogram = new Array(bins).fill(0);

        for (let dy = -radius; dy <= radius; dy++) {
            for (let dx = -radius; dx <= radius; dx++) {
                const px = (x + dx + this.width) % this.width;
                const py = (y + dy + this.height) % this.height;
                const val = this.currentFrame[py * this.width + px];
                const bin = Math.min(Math.floor(val * bins), bins - 1);
                histogram[bin]++;
```

```
        }
    }

    return this.computeEntropy(histogram);
}

// Compute full entropy field
updateEntropyField() {
    for (let y = 0; y < this.height; y++) {
        for (let x = 0; x < this.width; x++) {
            const idx = y * this.width + x;
            this.entropyField[idx] = this.computeLocalEntropy(x, y);
        }
    }
}

// Mutual information between two regions
computeMutualInformation(x1, y1, x2, y2, radius = 2) {
    const bins = 4;
    const jointHist = new Array(bins * bins).fill(0);
    const marginal1 = new Array(bins).fill(0);
    const marginal2 = new Array(bins).fill(0);

    for (let dy = -radius; dy <= radius; dy++) {
        for (let dx = -radius; dx <= radius; dx++) {
            const px1 = (x1 + dx + this.width) % this.width;
            const py1 = (y1 + dy + this.height) % this.height;
            const px2 = (x2 + dx + this.width) % this.width;
            const py2 = (y2 + dy + this.height) % this.height;

            const val1 = this.currentFrame[py1 * this.width + px1];
            const val2 = this.currentFrame[py2 * this.width + px2];

            const bin1 = Math.min(Math.floor(val1 * bins), bins - 1);
            const bin2 = Math.min(Math.floor(val2 * bins), bins - 1);

            jointHist[bin1 * bins + bin2]++;
            marginal1[bin1]++;
            marginal2[bin2]++;
        }
    }

    // Compute MI
    let mi = 0;
    const total = (2 * radius + 1) ** 2;

    for (let i = 0; i < bins; i++) {
        for (let j = 0; j < bins; j++) {
            const joint = jointHist[i * bins + j] / total;
            const m1 = marginal1[i] / total;
            const m2 = marginal2[j] / total;
```

```
            if (joint > 0 && m1 > 0 && m2 > 0) {
                mi += joint * Math.log2(joint / (m1 * m2));
            }
        }
    }

    return mi;
}

// KL divergence from reference
computeKLDivergence(x, y) {
    const idx = y * this.width + x;
    const p = this.currentFrame[idx];
    const q = this.referenceFrame[idx];

    if (p > 0 && q > 0) {
        return p * Math.log2(p / q);
    }
    return 0;
}

// Update KL field
updateKLField() {
    for (let y = 0; y < this.height; y++) {
        for (let x = 0; x < this.width; x++) {
            const idx = y * this.width + x;
            this.klField[idx] = this.computeKLDivergence(x, y);
        }
    }
}

// Compression ratio estimation
estimateCompressibility(x, y) {
    // Simple: variance in neighborhood = incompressibility
    const radius = 2;
    let sum = 0;
    let sumSq = 0;
    let count = 0;

    for (let dy = -radius; dy <= radius; dy++) {
        for (let dx = -radius; dx <= radius; dx++) {
            const px = (x + dx + this.width) % this.width;
            const py = (y + dy + this.height) % this.height;
            const val = this.currentFrame[py * this.width + px];
            sum += val;
            sumSq += val * val;
            count++;
        }
    }
```

```
        const mean = sum / count;
        const variance = (sumSq / count) - mean * mean;

        // High variance = low compressibility
        return 1.0 - Math.min(variance * 10, 1.0);
    }

    // Channel capacity with noise
    computeCapacity() {
        // Shannon: C = B * log2(1 + S/N)
        // Simplified: capacity = 1 - noiseLevel
        this.capacity = Math.max(0, 1.0 - this.noiseLevel);
        return this.capacity;
    }

    // Update frame from input (e.g., video, noise, pattern)
    updateFrame(inputData) {
        // Shift history
        this.prevFrame.set(this.currentFrame);
        this.currentFrame.set(inputData);

        // Add to history
        this.frameHistory.push(new Float32Array(inputData));
        if (this.frameHistory.length > this.maxHistory) {
            this.frameHistory.shift();
        }
    }

    // Generate synthetic frame
    generateSyntheticFrame() {
        for (let i = 0; i < this.currentFrame.length; i++) {
            const x = (i % this.width) / this.width;
            const y = Math.floor(i / this.width) / this.height;

            // Moving pattern
            this.currentFrame[i] = (
                Math.sin(x * 10 + this.time) *
                Math.cos(y * 10 + this.time * 0.7) * 0.5 + 0.5
            );
        }
    }

    // Set reference frame
    setReference(frameData) {
        this.referenceFrame.set(frameData);
    }

    // Get metrics data for shader
    getMetricsData() {
        // Pack entropy, MI, KL into RGBA
        const data = new Float32Array(this.width * this.height * 4);
```

```
        for (let i = 0; i < this.width * this.height; i++) {
            data[i * 4] = this.entropyField[i] / 3.0; // normalize
            data[i * 4 + 1] = this.mutualInfoField[i];
            data[i * 4 + 2] = this.klField[i];
            data[i * 4 + 3] = this.currentFrame[i];
        }

        return data;
    }

    // Get uniforms for shader
    getUniforms() {
        return {
            u_time: this.time,
            u_mode: this.mode,
            u_noise_level: this.noiseLevel
        };
    }

    // Main update
    update(dt) {
        this.time += dt;
        this.generateSyntheticFrame();
        this.updateEntropyField();
        this.updateKLField();
        this.computeCapacity();
    }

    setMode(mode) {
        this.mode = mode;
    }

    setNoiseLevel(level) {
        this.noiseLevel = Math.max(0, Math.min(1, level));
    }
}
```